

Paper 2: Algorithms, programming and logic

Chapter 7: Algorithm design and problem-solving

By: Mr. Nouredine Tadjerout

Learning Objectives:

Candidates should be able to:

- 1 Understand the program development life cycle, limited to: analysis, design, coding and testing
- 2
 - (a) Understand that every computer system is made up of sub-systems, which are made up of further sub-systems
 - (b) Understand how a problem can be decomposed into its component parts
 - (c) Use different methods to design and construct a solution to a problem
- 3 Explain the purpose of a given algorithm

Notes and guidance

- Including identifying each stage and performing these tasks for each stage:
 - analysis: abstraction, decomposition of the problem, identification of the problem and requirements
 - design: decomposition, structure diagrams, flowcharts, pseudocode
 - coding: writing program code and iterative testing
 - testing: testing program code with the use of test data
- Including:
 - inputs
 - processes
 - outputs
 - storage
- Including:
 - structure diagrams
 - flowcharts
 - pseudocode
- Including:
 - stating the purpose of an algorithm
 - describing the processes involved in an algorithm

Learning Objectives:

- 4 Understand standard methods of solution
 - Limited to:
 - linear search
 - bubble sort
 - totalling
 - counting
 - finding maximum, minimum and average values
 - Including:
 - range check
 - length check
 - type check
 - presence check
 - format check
 - check digit
 - the purpose of each validation check and writing algorithms to implement each validation check
- 5 (a) Understand the need for validation checks to be made on input data and the different types of validation check
 - Including:
 - visual check
 - double entry check
 - The purpose of each verification check
- (b) Understand the need for verification checks to be made on input data and the different types of verification check

Learning Objectives:

6 Suggest and apply suitable test data

- Limited to:
 - normal
 - abnormal
 - extreme
 - boundary
- Extreme data is the largest/smallest acceptable value
- Boundary data is the largest/smallest acceptable value and the corresponding smallest/largest rejected value
- Including, at each step in an algorithm:
 - variables
 - outputs
 - user prompts

7 Complete a trace table to document a dry-run of an algorithm

8 Identify errors in given algorithms and suggest ways of correcting these errors

9 Write and amend algorithms for given problems or scenarios, using: pseudocode, program code and flowcharts

- Precision is required when writing algorithms, e.g. `x > y` is acceptable but `x is greater than y` is not acceptable
- See section 4 for flowchart symbols
- See section 4 for pseudocode

Resources:



<https://www.hoddereducation.co.uk/subjects/general/products/14-16/cambridge-igcse-and-o-level-computer-science-second-edition>

Resources:



Craig 'n' Dave For Students

CRAIG 'N' DAVE

To explore the entire series,
visit <https://craigndave.org/free-videos>

Learning Objectives:

Candidates should be able to:

- 1 Understand the program development life cycle, limited to: analysis, design, coding and testing
- 2
 - (a) Understand that every computer system is made up of sub-systems, which are made up of further sub-systems
 - (b) Understand how a problem can be decomposed into its component parts
 - (c) Use different methods to design and construct a solution to a problem
- 3 Explain the purpose of a given algorithm

Notes and guidance

- Including identifying each stage and performing these tasks for each stage:
 - analysis: abstraction, decomposition of the problem, identification of the problem and requirements
 - design: decomposition, structure diagrams, flowcharts, pseudocode
 - coding: writing program code and iterative testing
 - testing: testing program code with the use of test data
- Including:
 - inputs
 - processes
 - outputs
 - storage
- Including:
 - structure diagrams
 - flowcharts
 - pseudocode
- Including:
 - stating the purpose of an algorithm
 - describing the processes involved in an algorithm

7.1 Understand the program development life cycle, limited to: analysis, design, coding and testing

Click the link below to watch the video for the Software development life cycle

<https://www.youtube.com/watch?v=cY0GjVzmo8s&list=PLCiOXwirraUCKOH2duohyLRk9O75bgUdI>

7 Algorithm design and problem-solving

CAMBRIDGE
IGCSE
(0478/0984)

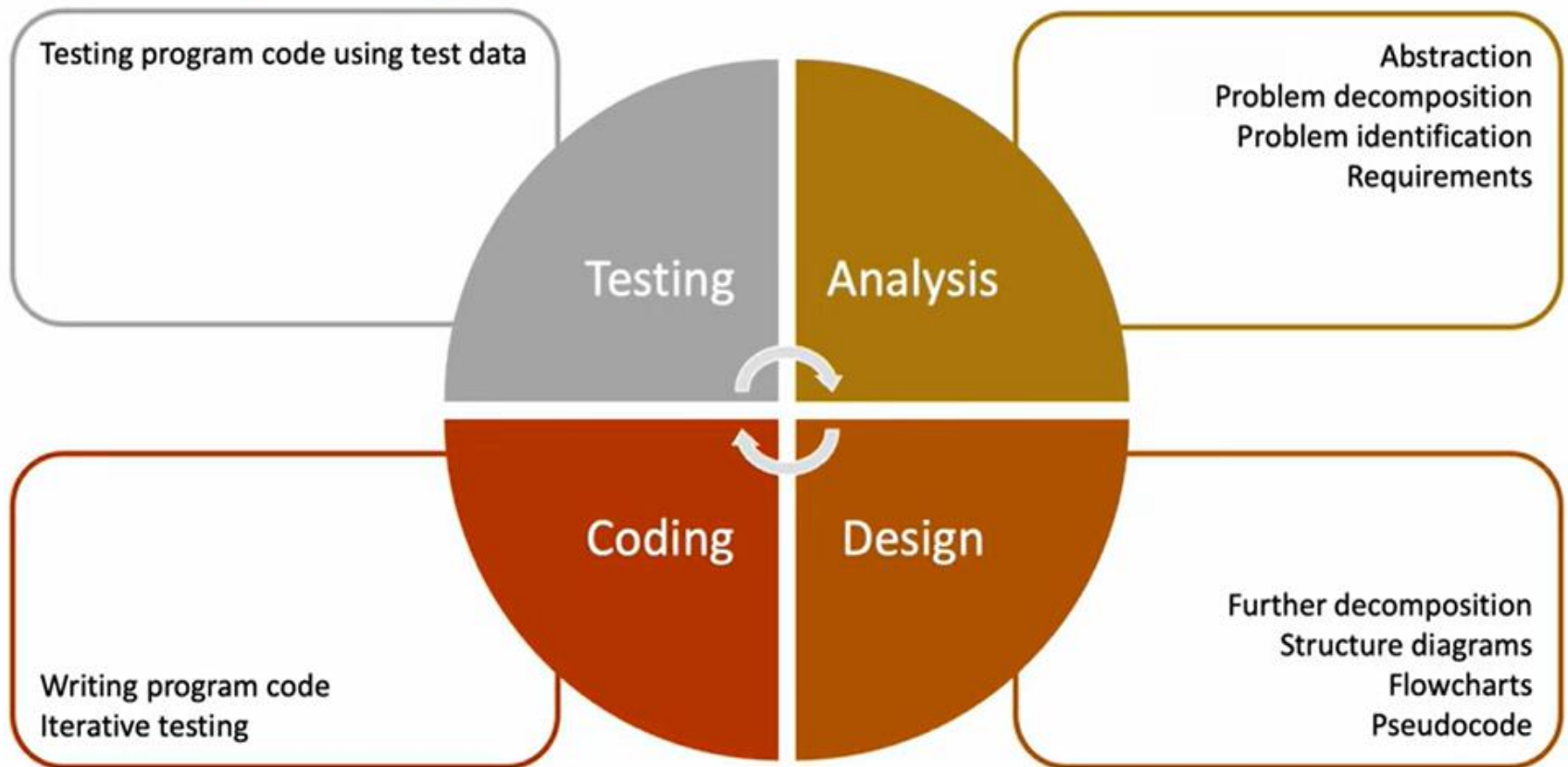
Software development life cycle



3:10
minutes

7.1 Understand the program development life cycle, limited to: analysis, design, coding and testing

Software development lifecycle



7.1 Understand the program development life cycle, limited to: analysis, design, coding and testing

Software development lifecycle

When developing a solution to a problem for a computer system, programmers work through a number of distinct phases, collectively known as the **software development lifecycle (SDLC)**.

There are four main stages, during which various key tasks need to be performed:

- **Analysis**
 - Abstraction
 - Problem decomposition
 - Problem identification
 - Requirements
- **Design**
 - Further decomposition
 - Structure diagrams
 - Flowcharts
 - Pseudocode
- **Coding**
 - Writing program code
 - Iterative testing
- **Testing**
 - Testing program code using test data



7.1 Understand the **program development life cycle**, limited to: **analysis, design, coding and testing**

1. Analysis:

Abstraction: Identifying the essential details of the problem and ignoring non-essential information.

Decomposition of the problem: Breaking down the problem into smaller, manageable sub-problems.

Identification of the problem and requirements: Clearly defining the problem statement and understanding the requirements from an end-user perspective.

2. Design:

Decomposition: Breaking down the problem into modules or components, each with a specific function.

Structure Diagrams: Creating diagrams like UML (Unified Modeling Language) class diagrams or entity-relationship diagrams to visualize the structure of the program.

Flowcharts: Creating flowcharts to represent the flow of control within the program, showing decision points, loops, and actions taken based on conditions.

Pseudocode: Writing pseudocode, which is a high-level description of the program logic using simple, human-readable language without worrying about specific syntax.

7.1 Understand the **program development life cycle**, limited to: **analysis, design, coding and testing**

3. Coding:

- **Writing Program Code:** Translating the design into actual code using a programming language, following the syntax and rules of that language.
- **Iterative Testing:** Testing the code in an iterative manner, identifying and fixing errors or bugs as they are found. This involves debugging and making necessary revisions to the code.

4. Testing:

- **Testing Program Code:** Using test data to validate the program's functionality and ensure it produces the expected output under various conditions.
- **Use of Test Data:** Creating different sets of input data to cover a wide range of scenarios, including normal cases, edge cases, and invalid inputs, to thoroughly test the program.

7.1 Understand the program development life cycle, limited to: analysis, design, coding and testing

Example:

Let's consider the development of a simple program to calculate the **area of a rectangle**.

Analysis:

Abstraction: Focus on the necessary information - **length** and **width** of the rectangle.

Decomposition: Divide the problem into sub-problems: **input validation**, **calculation of area**.

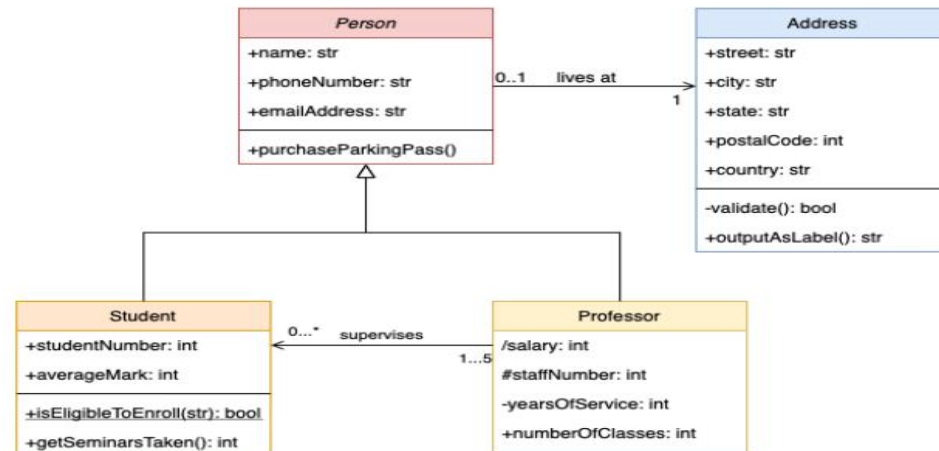
Problem Identification: Calculate the **area of a rectangle** given its **length** and **width**.

Design:

Decomposition: Identify modules - **input validation module**, **area calculation module**.

Structure Diagrams: Create a UML class diagram with classes like Rectangle and methods like **calculateArea()**.

Example: Class UML diagram.



7.1 Understand the program development life cycle, limited to: analysis, design, coding and testing

Example:

Flowcharts: Draw a flowchart depicting the flow of control from input to output.

Pseudocode: Write pseudocode for input validation and area calculation logic

1. Start
2. Read input for length from the user
3. Read input for width from the user
4. Calculate area as length multiplied by width and store it in the variable "area"
5. Display "The area of the rectangle is: " concatenated with the value of "area"
6. End

7.1 Understand the program development life cycle, limited to: analysis, design, coding and testing

Coding:

- Write code in a programming language (e.g., Python, Java) based on the design specifications.
- Implement functions/methods for input validation and area calculation.
- Use appropriate syntax and follow coding conventions.

```
main.py ×  
1 # Taking user input for length and width  
2 length = float(input("Enter the length of the rectangle: "))  
3 width = float(input("Enter the width of the rectangle: "))  
4  
5 # Calculating area of the rectangle  
6 area = length * width  
7  
8 # Displaying the area of the rectangle  
9 print("The area of the rectangle is:", area)
```

```
C:\Users\n.tadgerout\PycharmProjects\pythonProject  
Enter the length of the rectangle: 200  
Enter the width of the rectangle: 50  
The area of the rectangle is: 10000.0
```

7.1 Understand the program development life cycle, limited to: analysis, design, coding and testing

Testing:

- Prepare test data, including valid and invalid inputs (**length** and **width**).
- Test the program with different sets of input data.
- Verify that the calculated area matches the expected results for all test cases.
- Debug and fix any errors encountered during testing.

By following these stages and tasks, developers can systematically analyze, design, code, and test programs, ensuring the development process is organized and effective.

7.2 - (a) Understand that every computer system is made up of sub-systems, which are made up of further sub-systems

[Click the video to watch Computer sub-systems](#)

<https://www.youtube.com/watch?v=j51R215dNs0&list=PLCiOXwirraUCKOH2duohyLRk9O75bgUdI&index=2>

7 Algorithm design and problem-solving

Computer sub-systems

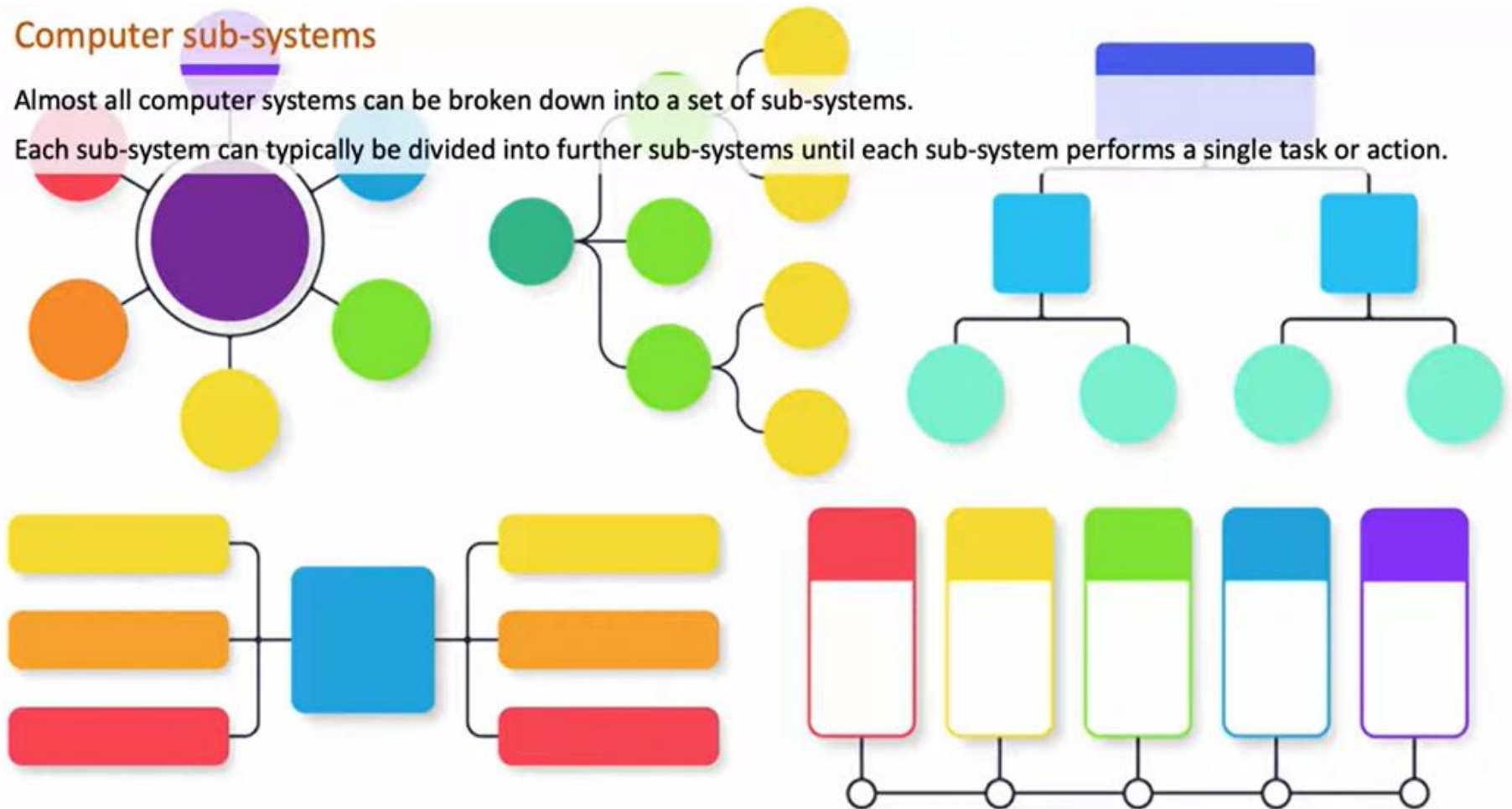
1:57

7.2 - (a) Understand that every computer system is made up of **sub-systems**, which are made up of **further sub-systems**

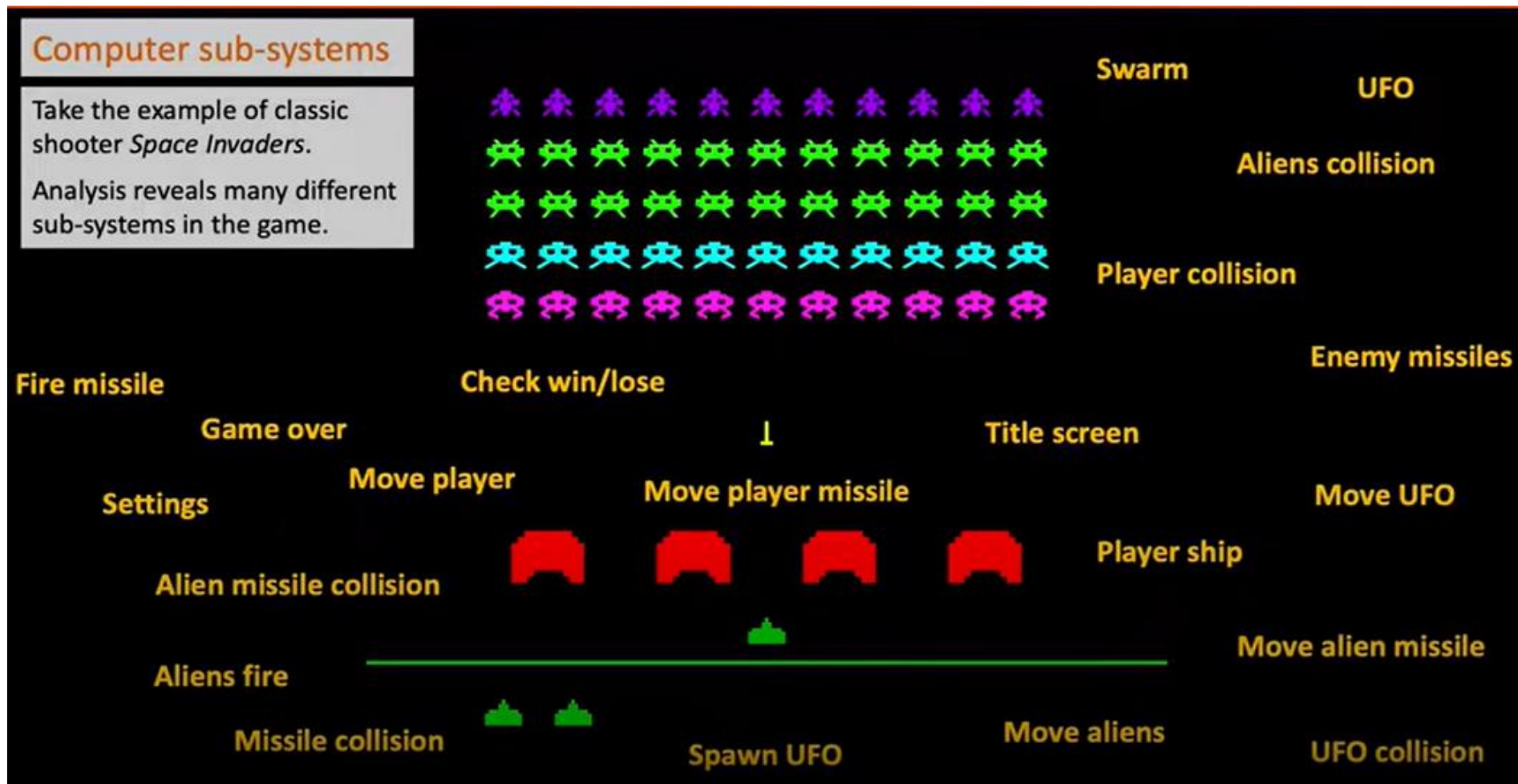
Computer sub-systems

Almost all computer systems can be broken down into a set of sub-systems.

Each sub-system can typically be divided into further sub-systems until each sub-system performs a single task or action.

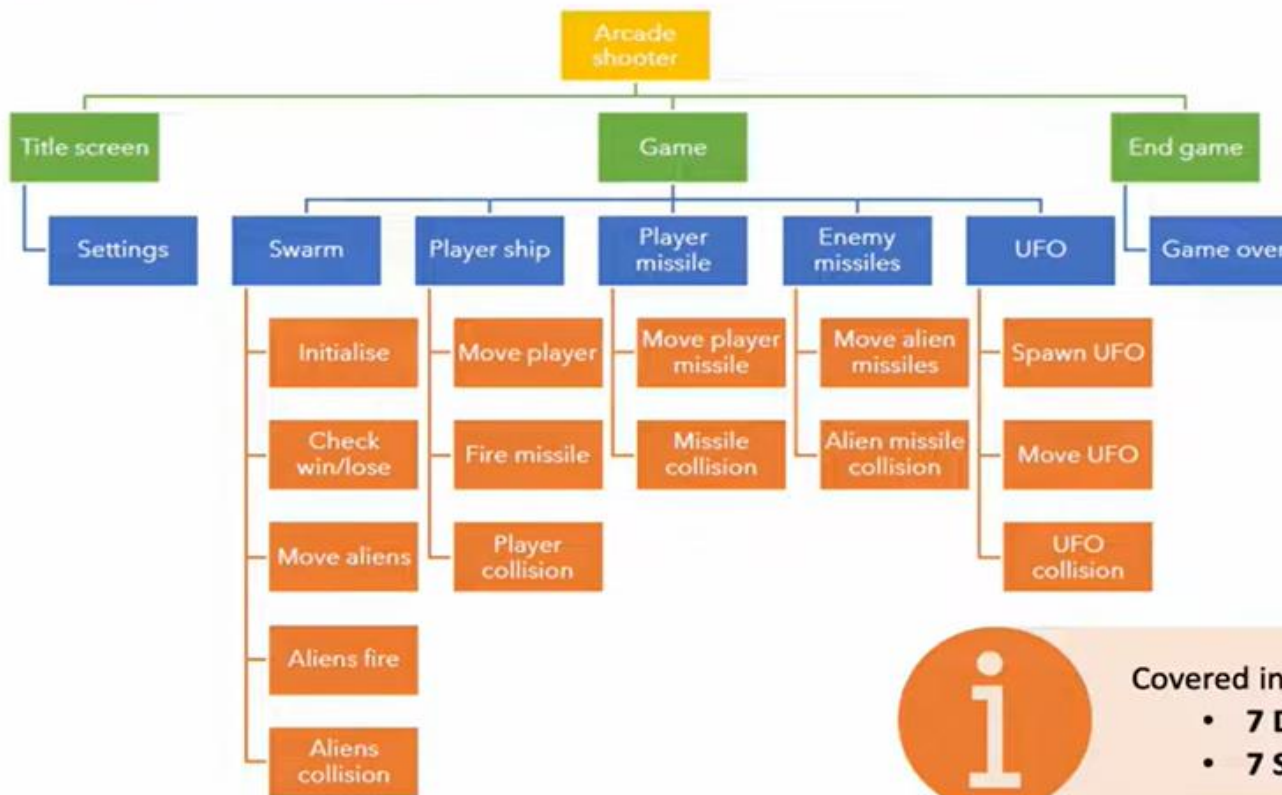


7.2 - (a) Understand that every computer system is made up of sub-systems, which are made up of further sub-systems



7.2 - (a) Understand that every computer system is made up of **sub-systems**, which are made up of **further sub-systems**

Computer sub-systems



Here is a **structure diagram**.

This is one just example of applying the analysis and design skill of **decomposition** to break a problem down into its various sub-problems.



Covered in more detail in the following videos:

- **7 Decomposition overview**
- **7 Structure diagrams**

7.2 - (a) Understand that every computer system is made up of **sub-systems**, which are made up of **further sub-systems**



Computer sub-systems

Computer systems range from small and simple to large and complex.

Almost all computer systems can be broken down into sub-systems. These sub-systems can typically be broken down into further sub-systems.

Eventually, each sub-system performs a single task or action.

7.2 - (a) Understand that every computer system is made up of sub-systems, which are made up of further sub-systems

In computer science and engineering, the concept of hierarchical systems is fundamental. Every computer system, from a simple electronic device to a complex network, is composed of subsystems, and these subsystems are further composed of smaller subsystems, forming a hierarchical structure. Each level of this hierarchy represents a different level of abstraction and functionality.

Example for 7.2 (a):

Consider a personal computer: **In this example, a personal computer is a computer system composed of several primary subsystems. The CPU, Memory, Storage, Input Devices, and Output Devices are primary subsystems, and each of them contains further subsystems, creating a hierarchical structure within the computer system**

Primary Subsystems: Central Processing Unit (CPU), Memory (RAM), Storage (Hard Drive or SSD), Input Devices (Keyboard, Mouse), Output Devices (Monitor, Printer).

- Subsystems of CPU: Arithmetic Logic Unit (ALU), Control Unit, Registers.

- **Subsystems of Memory:** Cache Memory, Main Memory.

- **Subsystems of Storage:** File System, Disk Controller.

- **Subsystems of Input Devices:** Keyboard Controller, Mouse Interface.

- **Subsystems of Output Devices:** Graphics Controller, Printer Interface.

7.2 - (b) Understand how a problem can be decomposed into its component parts

[Click the link below to watch the video for Decomposition overview](#)

<https://www.youtube.com/watch?v=UR4ZWB1B8cU&list=PLCiOXwirraUCKOH2duohyLRk9O75bgUdl&index=3>

7 Algorithm design and problem-solving

Decomposition overview



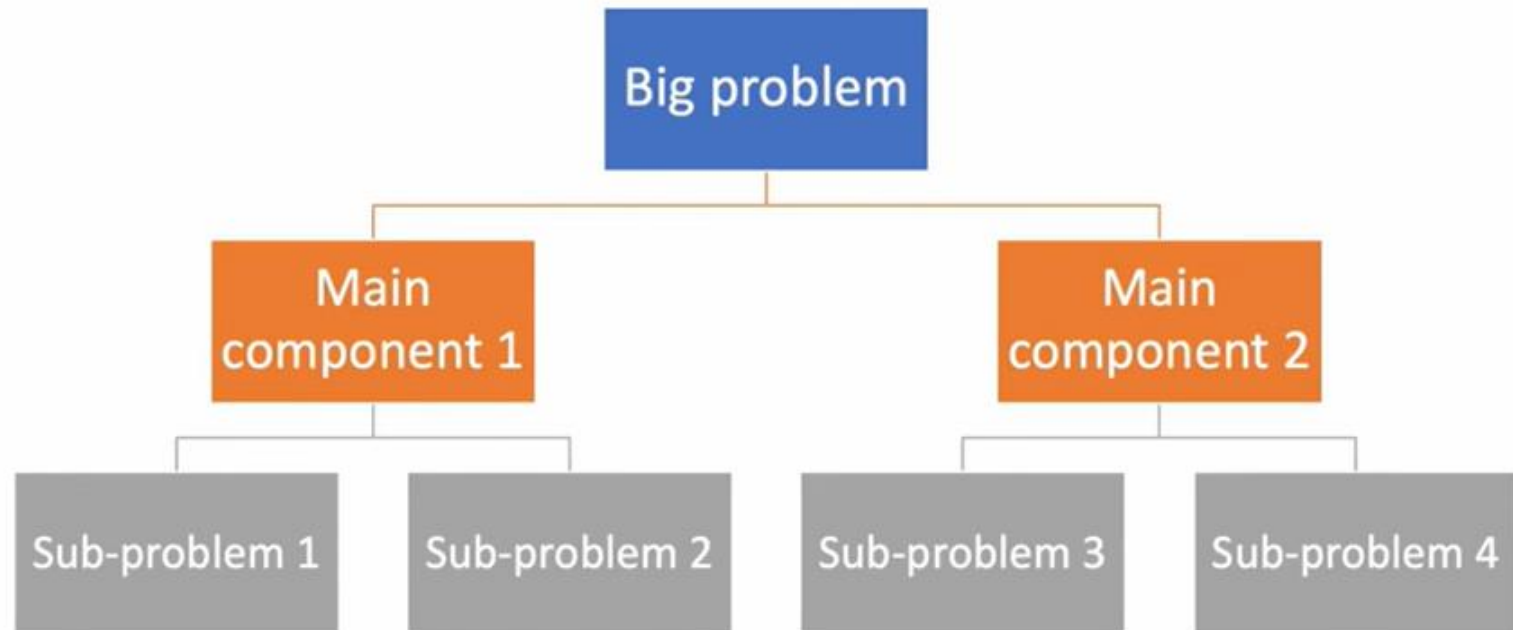
3:35

7.2 - (b) Understand how a problem can be **decomposed** into its component parts

Decomposition in daily life

Decomposition means breaking a complex problem down into smaller, more manageable parts.

Dealing with many different stages of a large problem at once is much more difficult than breaking it down into a number of smaller problems and solving them one at a time.



7.2 - (b) Understand how a problem can be **decomposed** into its component parts

Decomposition in daily life

You do many tasks on a daily basis:

- Getting up in the morning
- Brushing your teeth
- Travelling to school
- Doing your homework

These are all complex tasks with many steps involved, but you break them down and execute them without even thinking about it.



7.2 - (b) Understand how a problem can be **decomposed** into its component parts

Decomposition in daily life

Even an everyday problem like crossing the road can be broken down into sub-problems.

Cross a road

- Stop before crossing.
- Look left and right.
- Is the road clear?
- If not, press the button.
- Wait for the lights to turn red.
- Cross the road when it's safe.



7.2 - (b) Understand how a problem can be **decomposed** into its component parts

Decomposition

Breakout clone

How the game works:

- The player controls the paddle at the bottom of the screen. The paddle can be moved left and right.
- When the ball bounces off the paddle, it is redirected towards the blocks above. If the ball hits a block, that block is removed.
- The aim is to remove all the blocks. If the ball leaves the bottom of the screen, the player loses a life.

How could we use decomposition to help us break down the process of programming this game?



7.2 - (b) Understand how a problem can be **decomposed** into its component parts

Decomposition

Breakout clone

Breaking down the problem:

- Display the paddle on-screen.



7.2 - (b) Understand how a problem can be **decomposed** into its component parts

Decomposition

Breakout clone

Breaking down the problem:

- Display the paddle on-screen.
- Allow it to move left and right.



7.2 - (b) Understand how a problem can be **decomposed** into its component parts

Decomposition

Breakout clone

Breaking down the problem:

- Display the paddle on-screen.
- Allow it to move left and right.
- Prevent it from moving off the left or right edge of the screen.



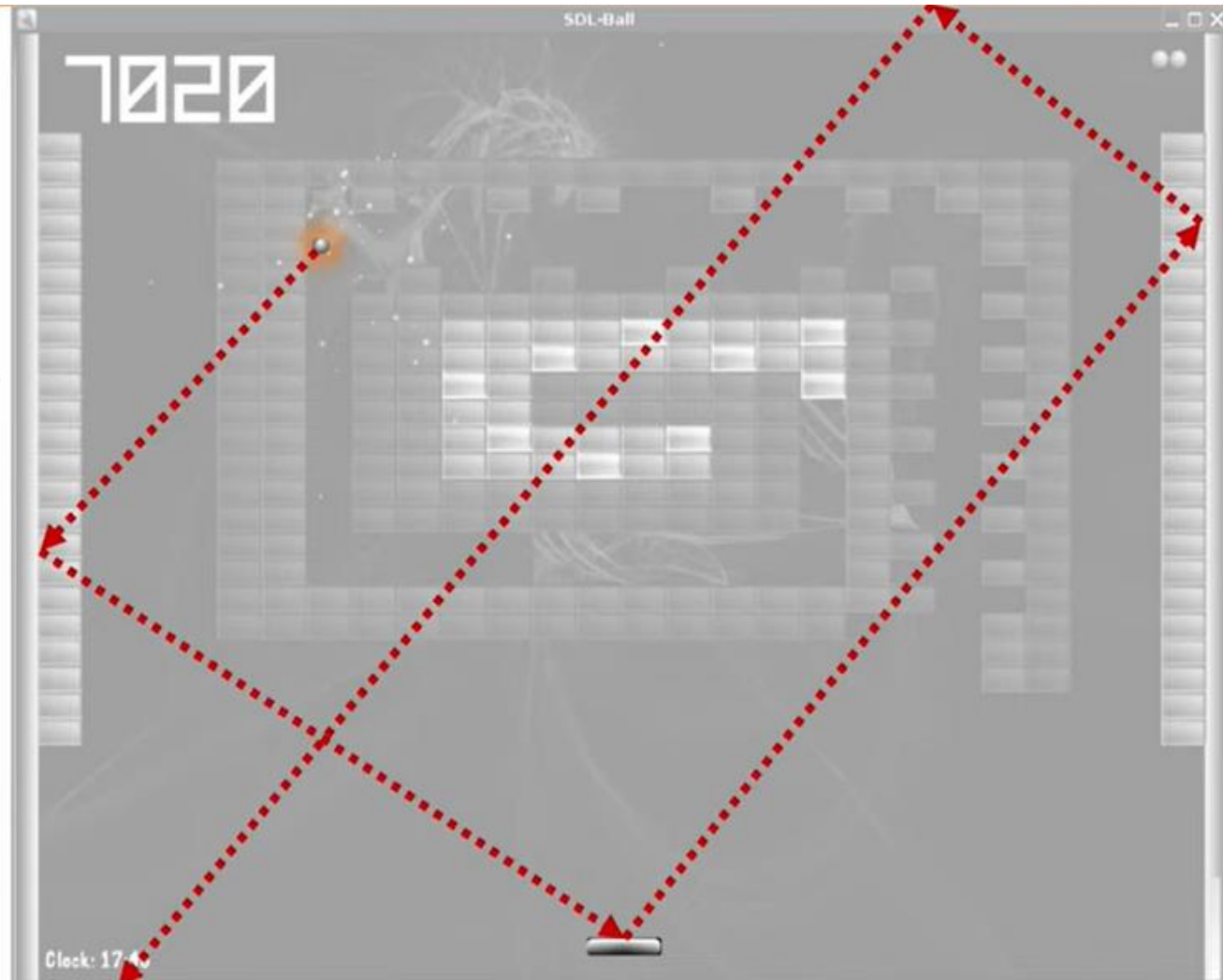
7.2 - (b) Understand how a problem can be **decomposed** into its component parts

Decomposition

Breakout clone

Breaking down the problem:

- Display the paddle on-screen.
- Allow it to move left and right.
- Prevent it from moving off the left or right edge of the screen.
- Display the ball on-screen.
- Program its movement.



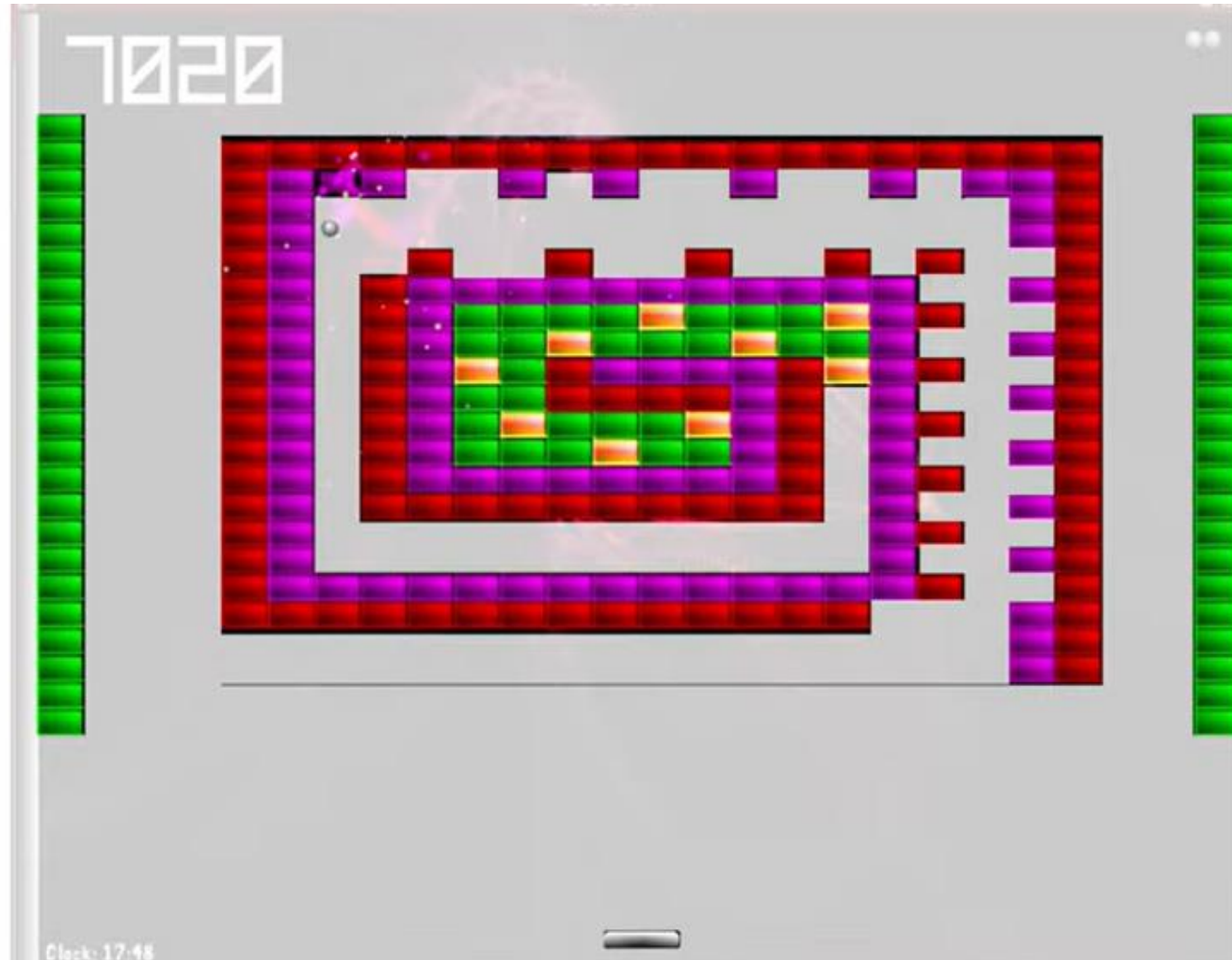
7.2 - (b) Understand how a problem can be **decomposed** into its component parts

Decomposition

Breakout clone

Breaking down the problem:

- Display the paddle on-screen.
- Allow it to move left and right.
- Prevent it from moving off the left or right edge of the screen.
- Display the ball on-screen.
- Program its movement.
- Display the blocks on-screen.



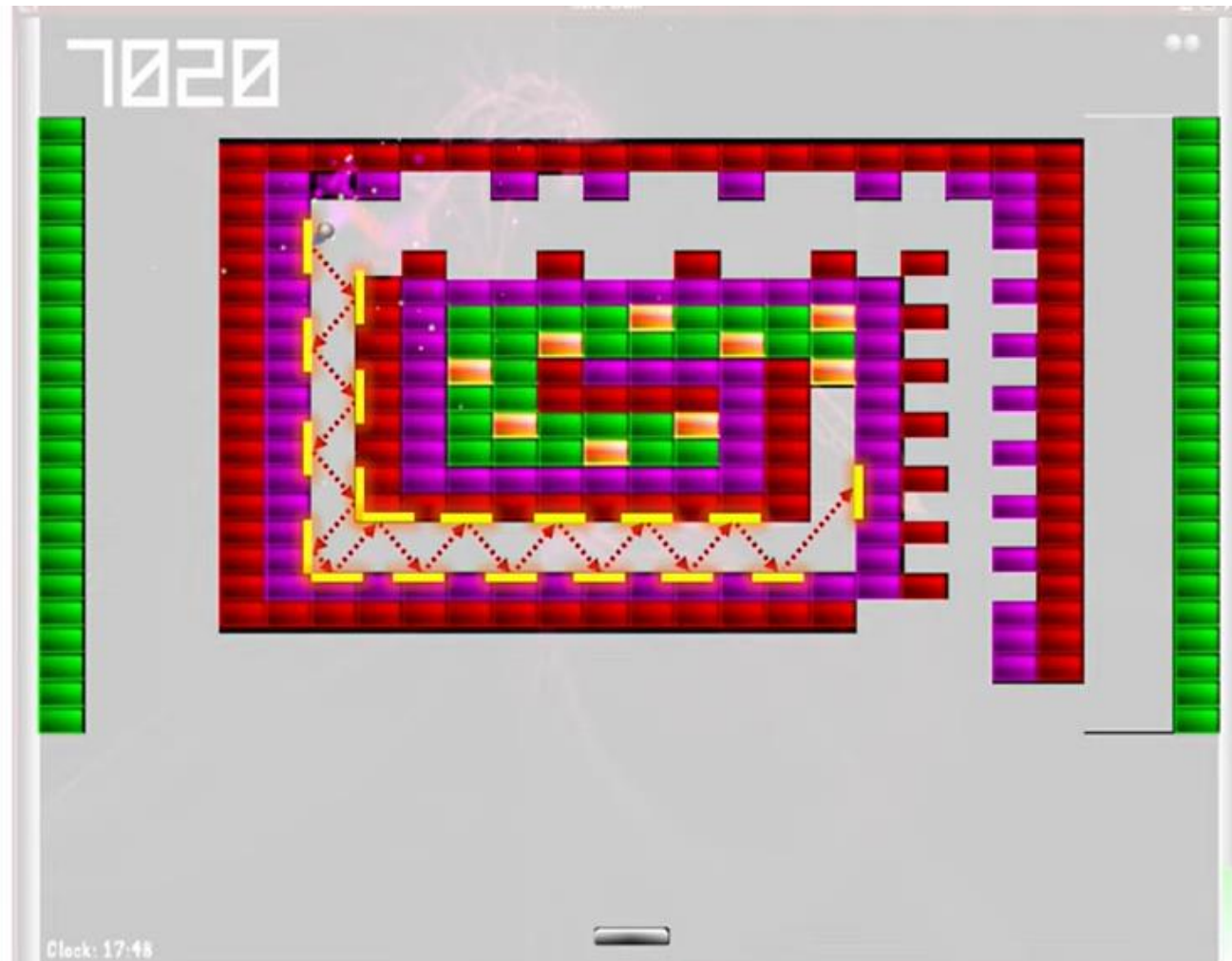
7.2 - (b) Understand how a problem can be **decomposed** into its component parts

Decomposition

Breakout clone

Breaking down the problem:

- Display the paddle on-screen.
- Allow it to move left and right.
- Prevent it from moving off the left or right edge of the screen.
- Display the ball on-screen.
- Program its movement.
- Display the blocks on-screen.
- Program collision detection.



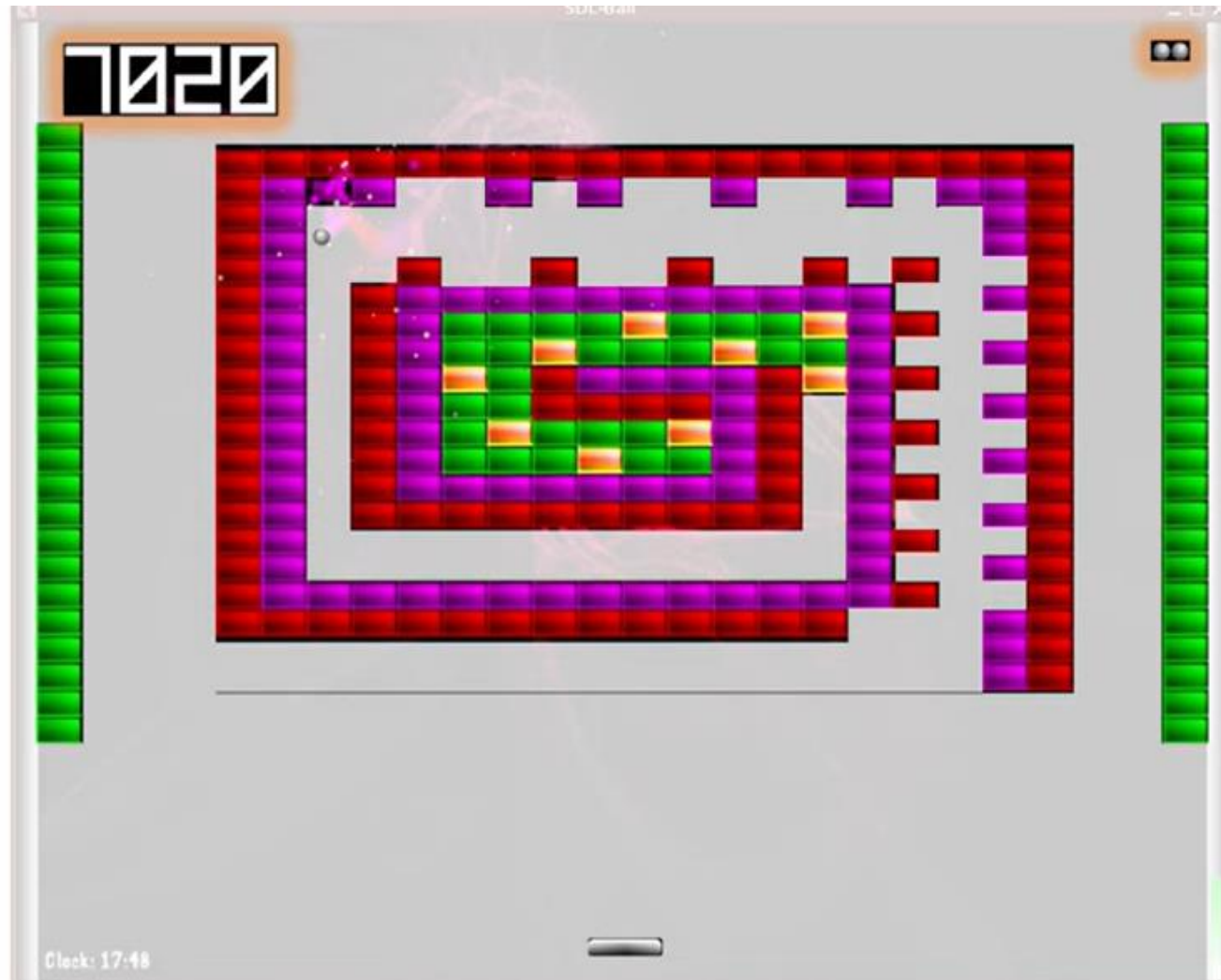
7.2 - (b) Understand how a problem can be decomposed into its component parts

Decomposition

Breakout clone

Breaking down the problem:

- Display the paddle on-screen.
- Allow it to move left and right.
- Prevent it from moving off the left or right edge of the screen.
- Display the ball on-screen.
- Program its movement.
- Display the blocks on-screen.
- Program collision detection.
- Implement winning and losing.



7.2 - (b) Understand how a problem can be **decomposed** into its component parts

Decomposition

Breakout clone

Breaking down the problem:

- Display the paddle on-screen.
- Allow it to move left and right.
- Prevent it from moving off the left or right edge of the screen.
- Display the ball on-screen.
- Program its movement.
- Display the blocks on-screen.
- Program collision detection.
- Implement winning and losing.
- Add extras: sounds, effects, graphics, background, etc.



7.2 - (b) Understand how a problem can be **decomposed** into its component parts



Decomposition

Decomposition means breaking a complex problem down into smaller, more manageable parts.

Advantages of problem decomposition

- Makes problems easier to solve.
- Different people can work on different parts of a problem at the same time, reducing development time.
- Program components developed in one program can easily be used in other programs.

7.2 - (b) Understand how a problem can be decomposed into its component parts

Click the link below to watch the video for **Inputs, processes, outputs and storage**

<https://www.youtube.com/watch?v=Cc9VFuTKggo&list=PLCiOXwirraUCKOH2duohyLRk9O75bgUdI&index=4>

7 Algorithm design and problem-solving

Inputs, processes,
outputs and storage



4:05
Stude

7.2 - (b) Understand how a problem can be decomposed into its component parts Inputs, processes, outputs and storage

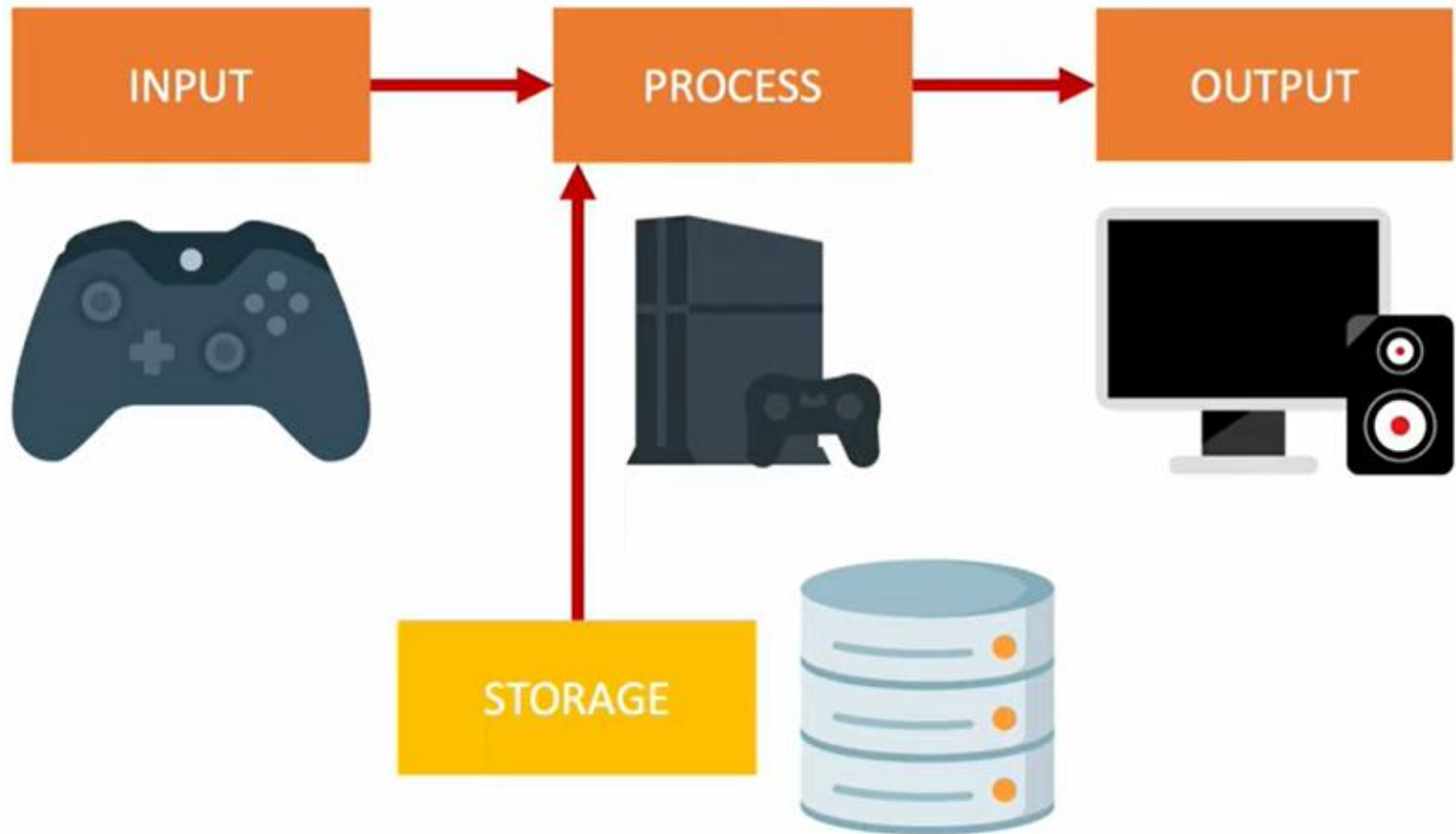
Inputs, processes and outputs



An electronic device that takes input, processes data and delivers output.

7.2 - (b) Understand how a problem can be **decomposed** into its component parts

Inputs, processes, outputs and storage



7.2 - (b) Understand how a problem can be **decomposed** into its component parts

Inputs, processes, outputs and storage

When writing a computer program, we need to be able to identify the inputs, processes, outputs and storage requirements of the problem.

Under exam conditions, this will usually relate to a provided scenario.



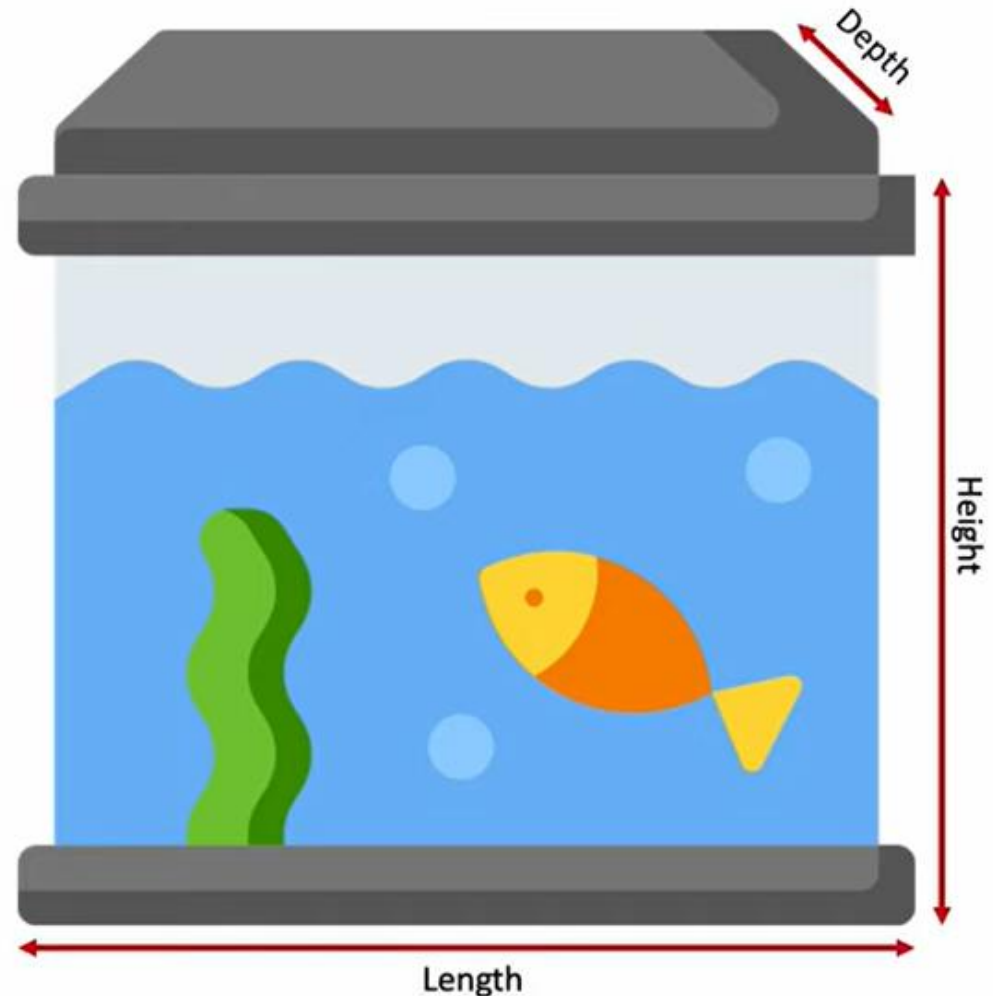
```
121 'Procedure to move the worm
122 Sub move_worm()
123   Dim wormHead() As Integer
124   Dim wormTail() As Integer
125   'calculate new head position
126   wormHead() = worm(wormHeadPointer, 1)
127   wormHead() = worm(wormHeadPointer, 2)
128   wormHeadPointer = wormHeadPointer + 1
129   If (wormHeadPointer = 100) Then wormHeadPointer = 0
130   Select Case wormDirection
131     Case "u"
132       wormHead() = wormHead() + 1
133     Case "d"
134       wormHead() = wormHead() - 1
135     Case "l"
136       wormHead() = wormHead() - 1
137     Case "r"
138       wormHead() = wormHead() + 1
139   End Select
140   'wrap worm around screen if needed
141   If (wormHead() = 50) Then wormHead() = 0
142   If (wormHead() = -1) Then wormHead() = 49
143   If (wormHead() = -1) Then wormHead() = 23
144   If (wormHead() = 24) Then wormHead() = 0
145   'set new worm head position
146   worm(wormHeadPointer, 1) = wormHead()
147   worm(wormHeadPointer, 2) = wormHead()
148   collision_detection_and_store()
149   arena(wormHead(), wormHead()) = "W"
150   'draw worm head
151   Console.SetCursorPosition(worm(wormHeadPointer, 1), worm(wormHeadPointer, 2))
152   Console.WriteLine("W")
153   'delete segment from tail
154   If wormDraw = False Then
155     Console.SetCursorPosition(worm(wormTailPointer, 1), worm(wormTailPointer, 2))
156     Console.WriteLine(" ")
157     arena(worm(wormTailPointer, 1), worm(wormTailPointer, 2)) = " "
158     wormTailPointer = wormTailPointer + 1
159     If (wormTailPointer = 100) Then wormTailPointer = 0
160   Else
161     wormDraw = False
162   End If
163 End Sub
164
165 'Procedure to display the game over message and get high score name
166 Sub game_over_message()
167   Dim key As Char
168   Console.SetCursorPosition(17, 12)
169   Console.WriteLine("G A M E O V E R")
170   System.Threading.Thread.Sleep(3000)
171   Console.SetCursorPosition(14, 14)
172   Console.WriteLine("Press a key to continue")
173   key = Console.ReadKey.KeyChar
174 End Sub
175
176 'Procedure to compute and display the high score table
177 Sub high_scores()
178   Dim counter1, counter2 As Integer
179   Dim key As Char
180   Dim isHighScore As Boolean
181   Dim name As String
182
183   'check if score is a high score
184   isHighScore = False
185   counter1 = 0
186
187   'go through scores and check if score is a high score
```

7.2 - (b) Understand how a problem can be **decomposed** into its component parts

Inputs, processes, outputs and storage

"Write a program to calculate the volume of a fish tank based on its dimensions, and report the result to the user."

Input	Length: Real/float Height: Real/float Depth: Real/float
Process	Length x Height x Depth
Output	Volume: Real/float

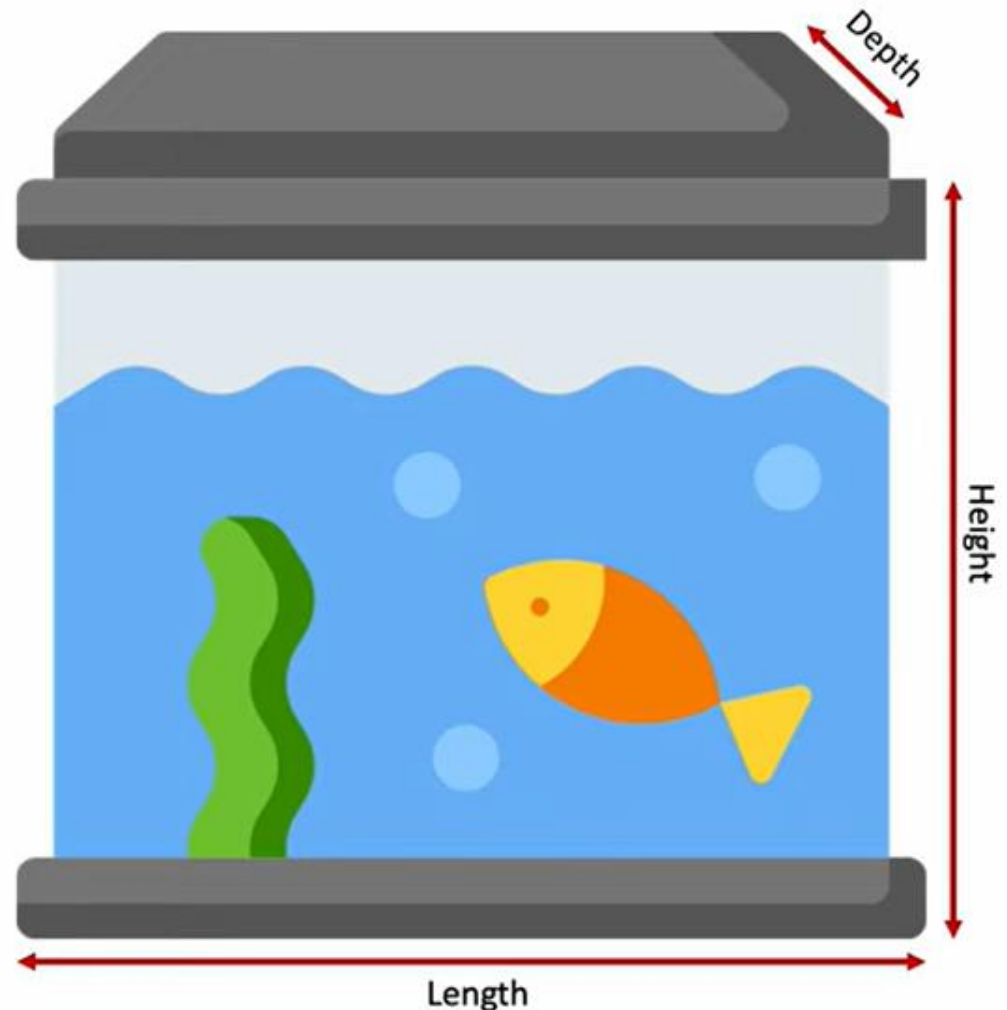


7.2 - (b) Understand how a problem can be **decomposed** into its component parts

Inputs, processes, outputs and storage

*"Write a program to calculate the volume of a fish tank based on its dimensions, report the result to the user **and write the results to a text file.**"*

Input	Length: Real/float Height: Real/float Depth: Real/float
Process	Length x Height x Depth
Output	Volume: Real/float
Storage	Volume: Real/float

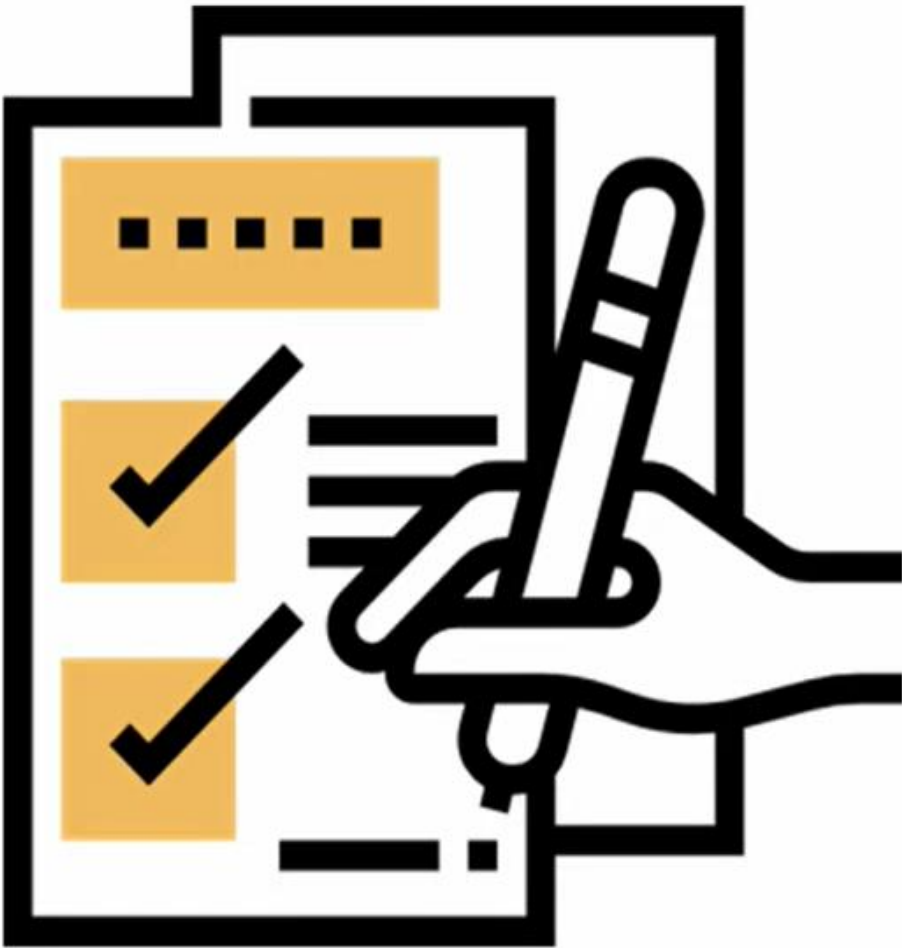


7.2 - (b) Understand how a problem can be **decomposed** into its component parts

Inputs, processes, outputs and storage

“Write a program that asks the user for the number of students in their class and then prompts the user to enter each student’s test score within a range of 0–100. The program should then output the highest, lowest and average score to the user as well as saving there highest score to a file for future retrieval.”

Input	
Process	
Output	
Storage	

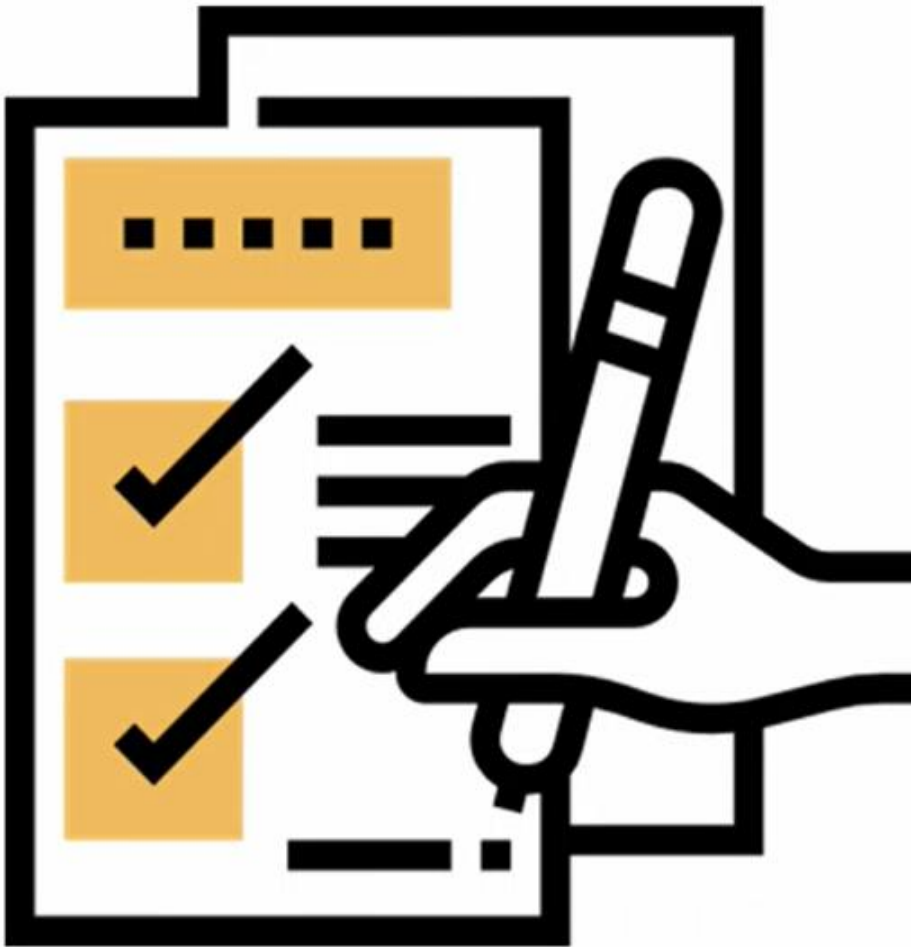


7.2 - (b) Understand how a problem can be **decomposed** into its component parts

Inputs, processes, outputs and storage

“Write a program that asks the user for the number of students in their class and then prompts the user to enter each student’s test score within a range of 0–100. The program should then output the highest, lowest and average score to the user as well as saving there highest score to a file for future retrieval.”

Input	NumOfStudents: Integer CurrentScore: Integer
Process	TotalScore = TotalScore + CurrentScore AverageScore = TotalScore / NumOfStudents Store the list of scores in an array: • Loop through and return the lowest score • Loop through and return the highest score
Output	LowScore: Integer HighScore: Integer AverageScore: Real/float
Storage	HighScore: Integer



7.2 - (b) Understand how a problem can be **decomposed** into its component parts



Inputs, processes and outputs

When tackling a problem, you need to consider any inputs, processes and outputs.

Inputs

- Anything that needs to be supplied to the program so it can meet its goals – often supplied by the user.
- Consider an appropriate variable name and data type for inputs.

Processes

- Calculations that need to be performed while the program is running.
- Consider whether any data need to change format or data type.

Outputs

- Consider what your program needs to output to the user.
- Consider what form outputs should take.
- Consider an appropriate variable name and data type for outputs.

Storage

- Consider any data that needs to be stored in files on an appropriate medium for use or retrieval in the future.

7.2 - (b) Understand how a problem can be decomposed into its component parts

Problem decomposition is a fundamental concept in computer science and problem-solving. It involves breaking down a complex problem into smaller, more manageable components, including inputs, processes, outputs, and storage requirements. Here's a breakdown of each component:

- **Inputs:** Inputs are the data or information provided to the system for processing. They are the raw materials needed to solve the problem. Inputs can come from various sources such as user input, sensors, files, or databases. Understanding the types and formats of inputs is crucial for problem-solving.

- **Processes:** Processes refer to the operations or actions performed on the inputs to produce the desired outputs. Processes involve algorithms, calculations, manipulations, and transformations applied to the input data. Efficient and logical processes are key to solving a problem effectively.

- **Outputs:** Outputs are the results or outcomes produced by processing the inputs. They represent the solution to the problem. Outputs can take various forms, including visual displays, reports, files, or signals. Understanding the expected outputs helps in verifying the correctness of the solution.

- **Storage:** Storage involves the temporary or permanent holding of data during the problem-solving process. Data may need to be stored for various reasons such as intermediate results, user preferences, or historical records. Proper storage management ensures that data is accessible, accurate, and secure when needed.

7.2 - (b) Understand how a problem can be **decomposed** into its component parts

Example for 7.2 (b):

Consider the problem of calculating and **displaying** the **total price** of items purchased by a customer in a store. Here's how the problem can be decomposed into its component parts:

Inputs:

- Item prices (from a database or input by the cashier)
- Quantities of items purchased (entered by the cashier or scanned from barcodes)

Processes:

- Calculate the total price for each item by multiplying its price by the quantity purchased.
- Sum up the individual item totals to compute the overall total price.

Outputs:

Display the total price to the customer (output on the cash register display).

Generate a receipt with itemized details and total price (physical or digital output for the customer).

Storage:

Temporary storage of intermediate results (individual item totals) in variables during calculations.

Optionally, store transaction details, including item names, quantities, prices, and total price, in a database for record-keeping purposes.

7.2 - (c) Use different methods to **design** and **construct** a solution to a problem

Using **structure diagrams**, **flowcharts**, and **pseudocode** are effective methods for designing and constructing solutions to problems.

7.2 - (c) Use different methods to design and construct a solution to a problem

Click the link below to watch the video for Structure diagrams

<https://www.youtube.com/watch?v=gFY9QdxMInI&list=PLCiOXwirraUCKOH2duohyLRk9O75bgUdI&index=5>

7 Algorithm design and problem-solving

Structure diagrams



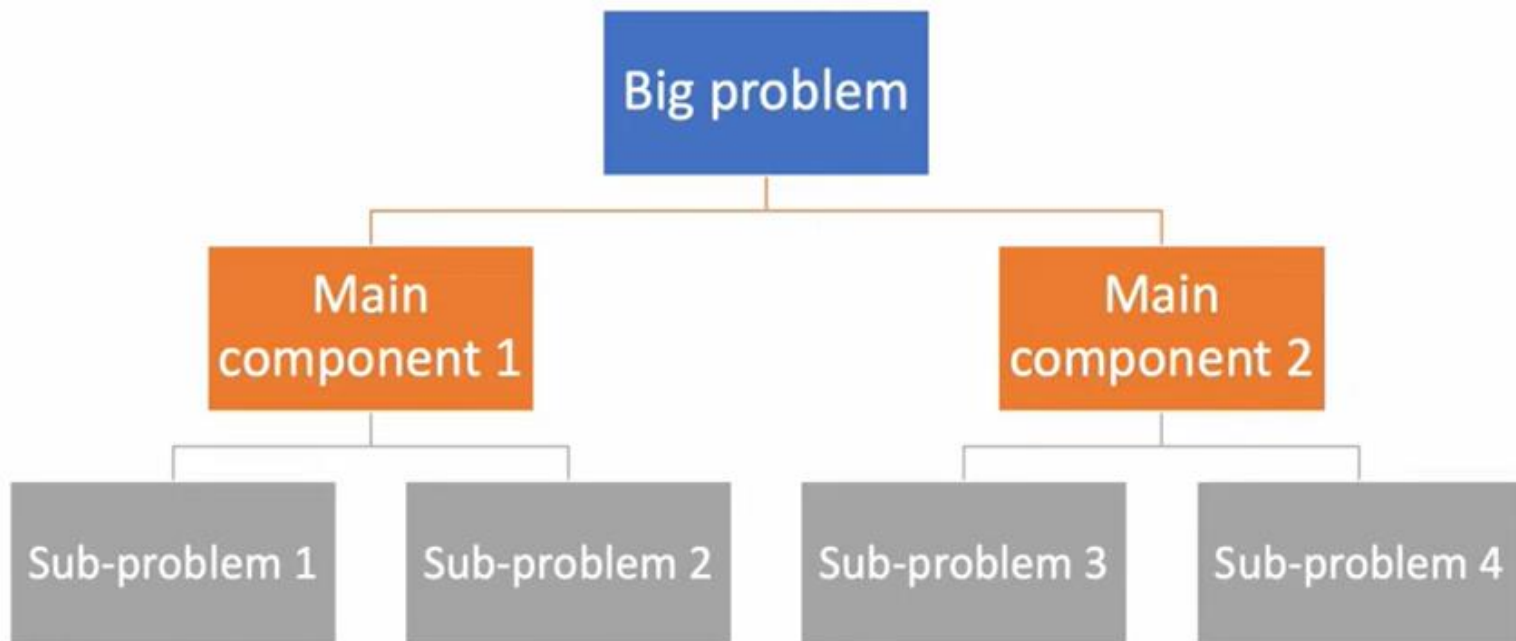
2:44
Study

7.2 - (c) Use different methods to **design** and **construct** a solution to a problem

Decomposition recap

Decomposition means breaking a complex problem down into smaller, more manageable parts.

Dealing with many different stages of a problem at once is much more difficult than breaking it down into a number of smaller problems and solving them one at time.

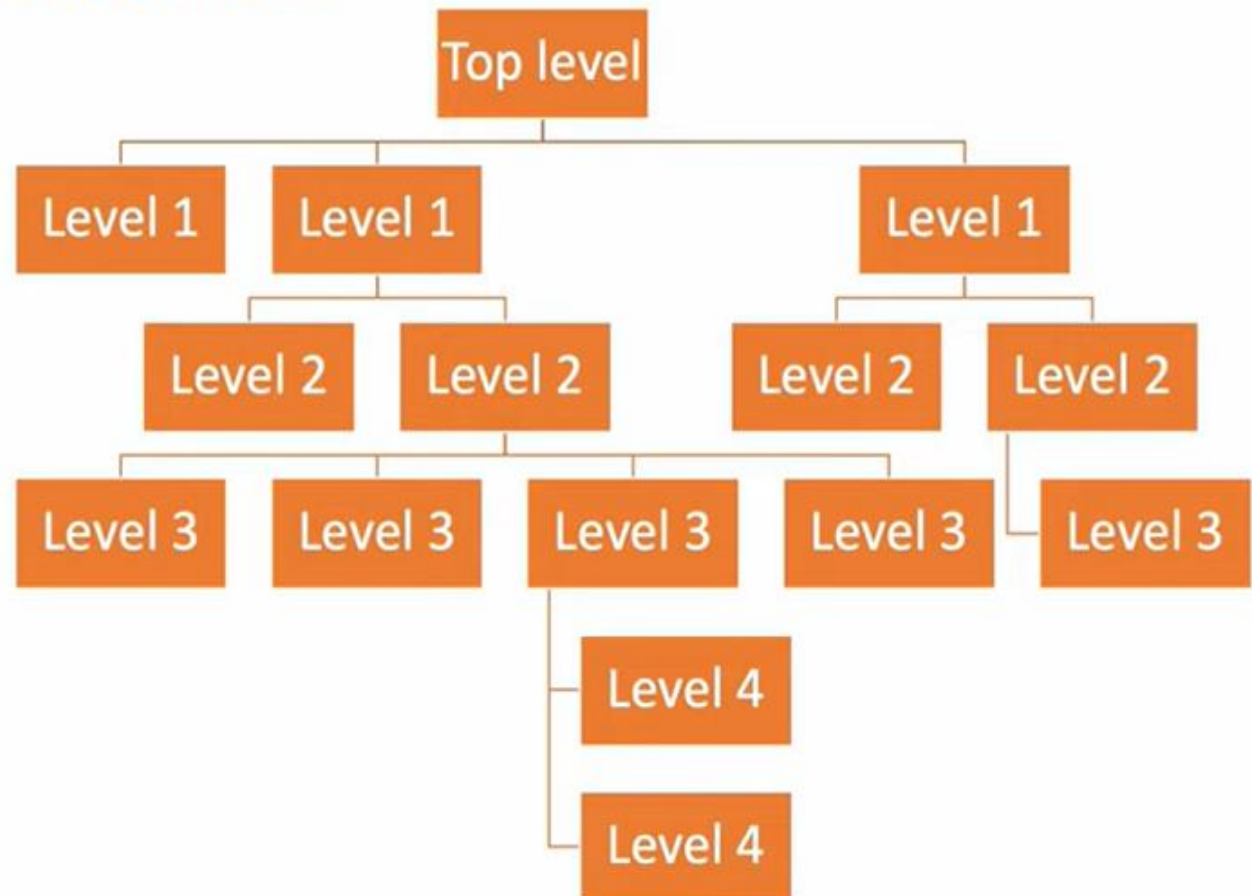


7.2 - (c) Use different methods to **design** and **construct** a solution to a problem

Step-wise refinement and structure diagrams

We can use **structure diagrams** to help **decompose** a problem.

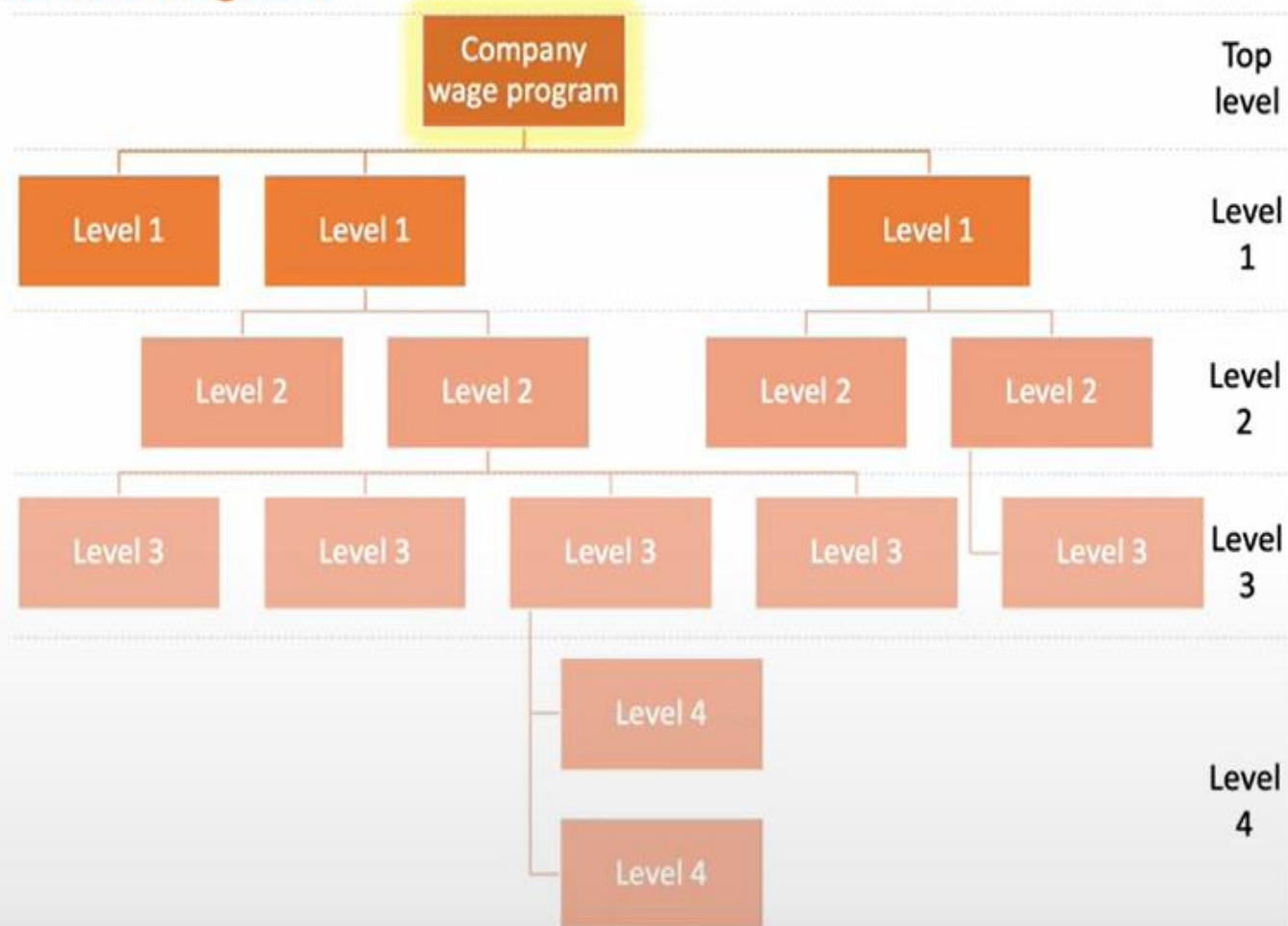
We create structure diagrams using an approach known as *step-wise refinement*.



7.2 - (c) Use different methods to **design** and **construct** a solution to a problem

Step-wise refinement and structure diagrams

At the top level, we have the name of the program.



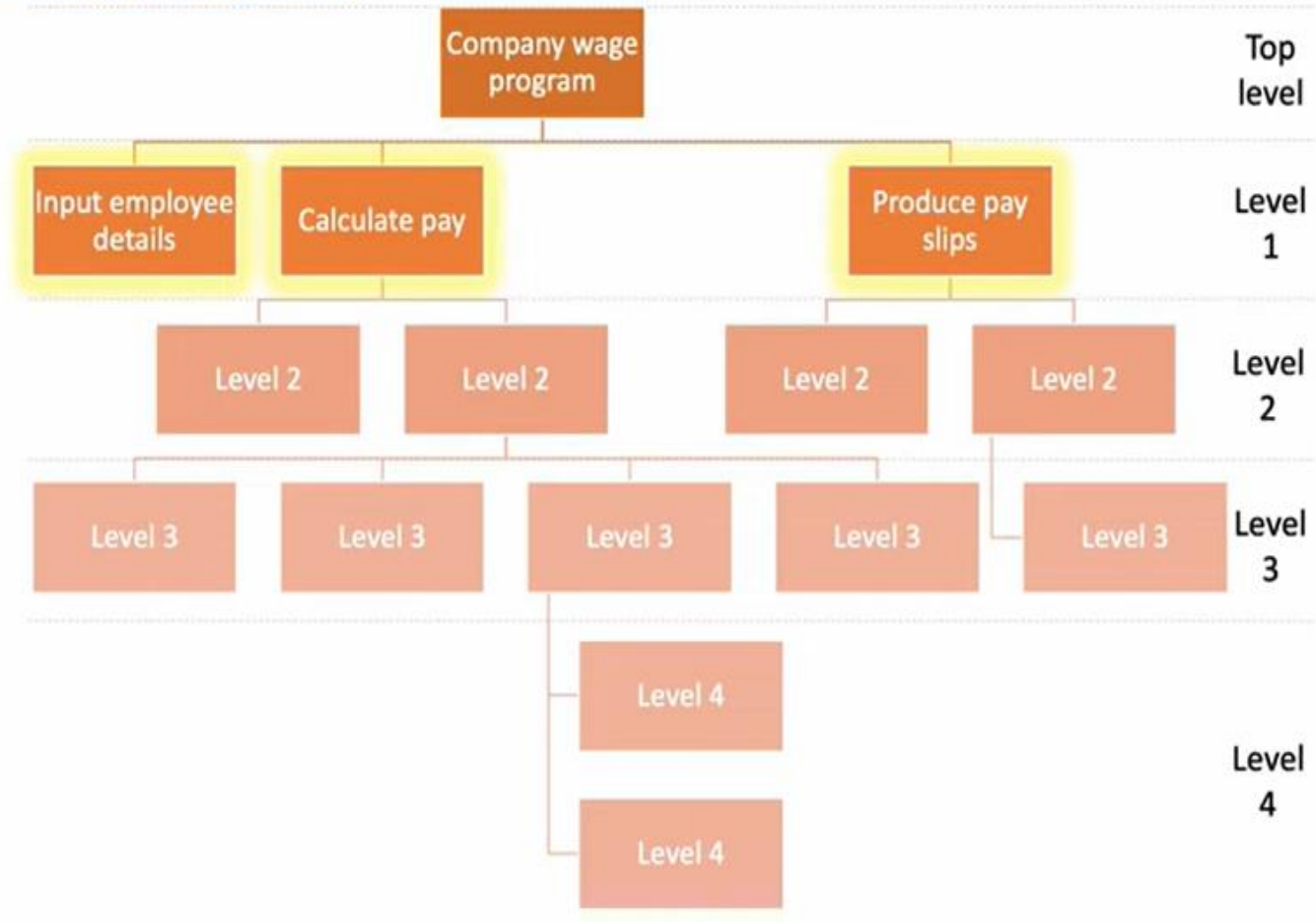
7.2 - (c) Use different methods to **design** and **construct** a solution to a problem

Step-wise refinement and structure diagrams

After some initial analysis, we can break the program down into the main tasks we need it to perform:

- Input employee details
- Calculate pay
- Produce pay slips

These would go in the next level of the structure diagram.



7.2 - (c) Use different methods to **design** and **construct** a solution to a problem

Step-wise refinement and structure diagrams

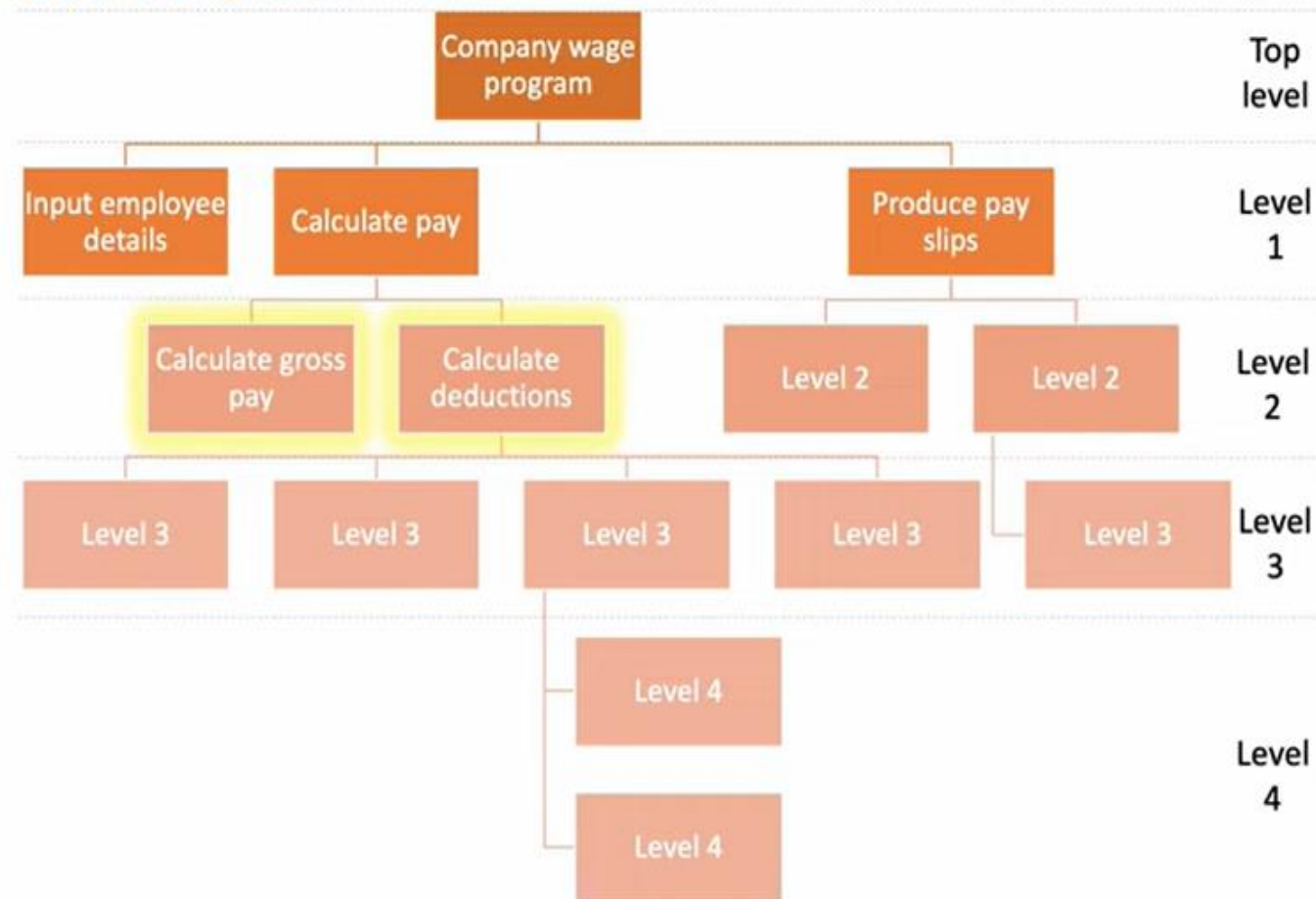
Some Level 1 modules may not need breaking down any further.

Some may still be quite complex and need breaking down further.

Calculate pay could be broken down again into:

- Gross pay
- Total deductions

These would go in the next level of the structure diagram.



7.2 - (c) Use different methods to **design** and **construct** a solution to a problem

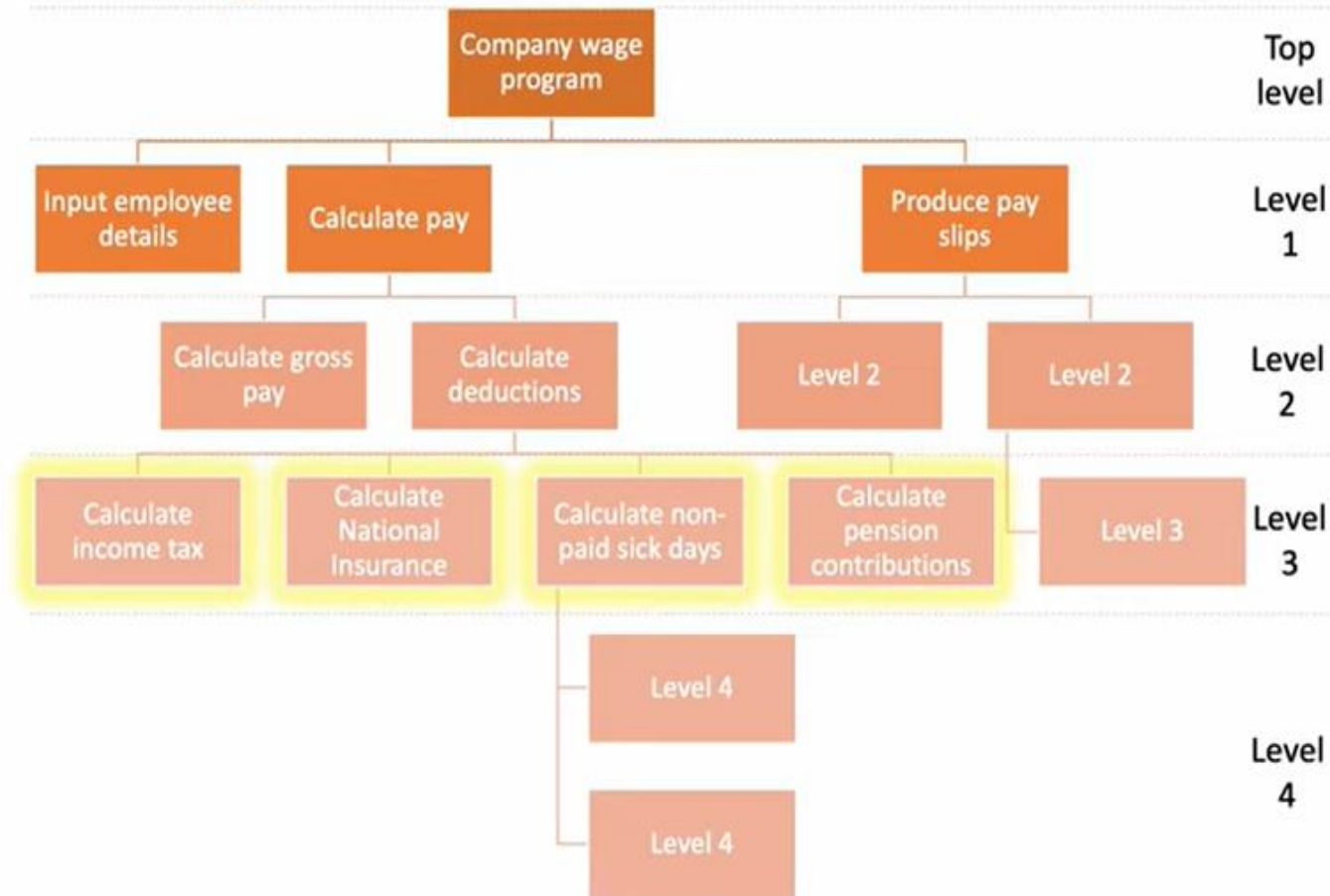
Step-wise refinement and structure diagrams

Carry this process on, breaking down any stage that is still too complex into smaller, more manageable chunks.

Calculate deductions could be broken down into:

- Income tax
- National Insurance
- Non-paid sick days
- Pension contributions

These would go in the next level of the structure diagram.

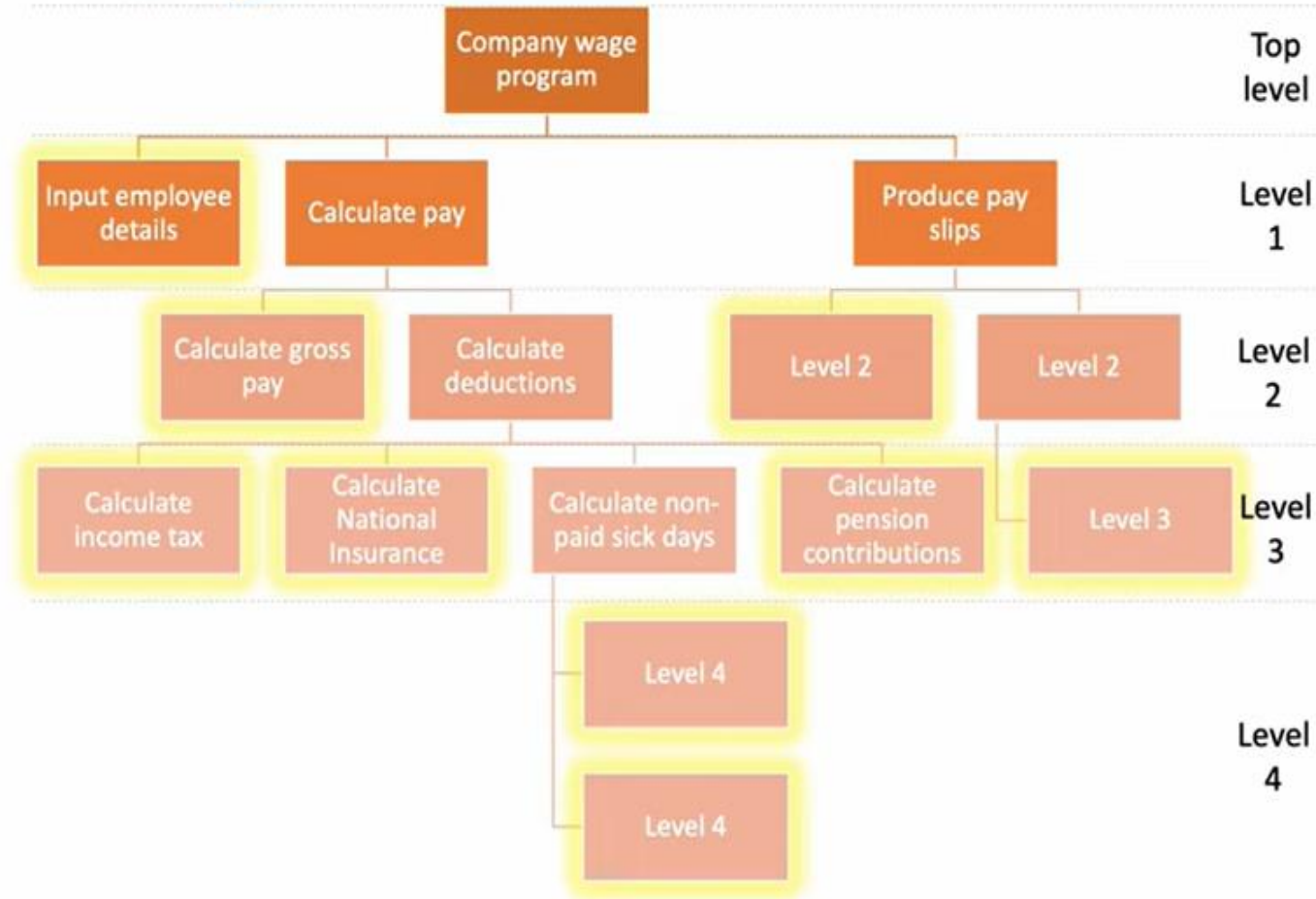


7.2 - (c) Use different methods to **design** and **construct** a solution to a problem

Step-wise refinement and structure diagrams

Different sections of the program will be broken down to varying degrees depending on how complex they are.

However, the end result should be the same – that each node at the lowest level achieves one task and can be coded as a single module or sub-program.



7.2 - (c) Use different methods to **design** and **construct** a solution to a problem



Structure diagrams

Structure diagrams

- A method of designing a solution to a problem.
- Produced using a method known as step-wise refinement.
- Break a problem down into smaller sub-problems using decomposition.
- Some areas of the program will need to be broken down more than others.
- Lowest-level nodes should achieve one task – and therefore, be coded as a single module or sub-program.

7.2 - (c) Use different methods to design and construct a solution to a problem

Click link below to watch video How to produce algorithms using pseudocode and flowcharts

<https://www.youtube.com/watch?v=kSpUGDYBnww&list=PLCiOXwirraUCKOH2duohyLRk9O75bgUdl&index=6>

7 Algorithm design and problem-solving

How to produce algorithms using
pseudocode and flowcharts

5:05
Study

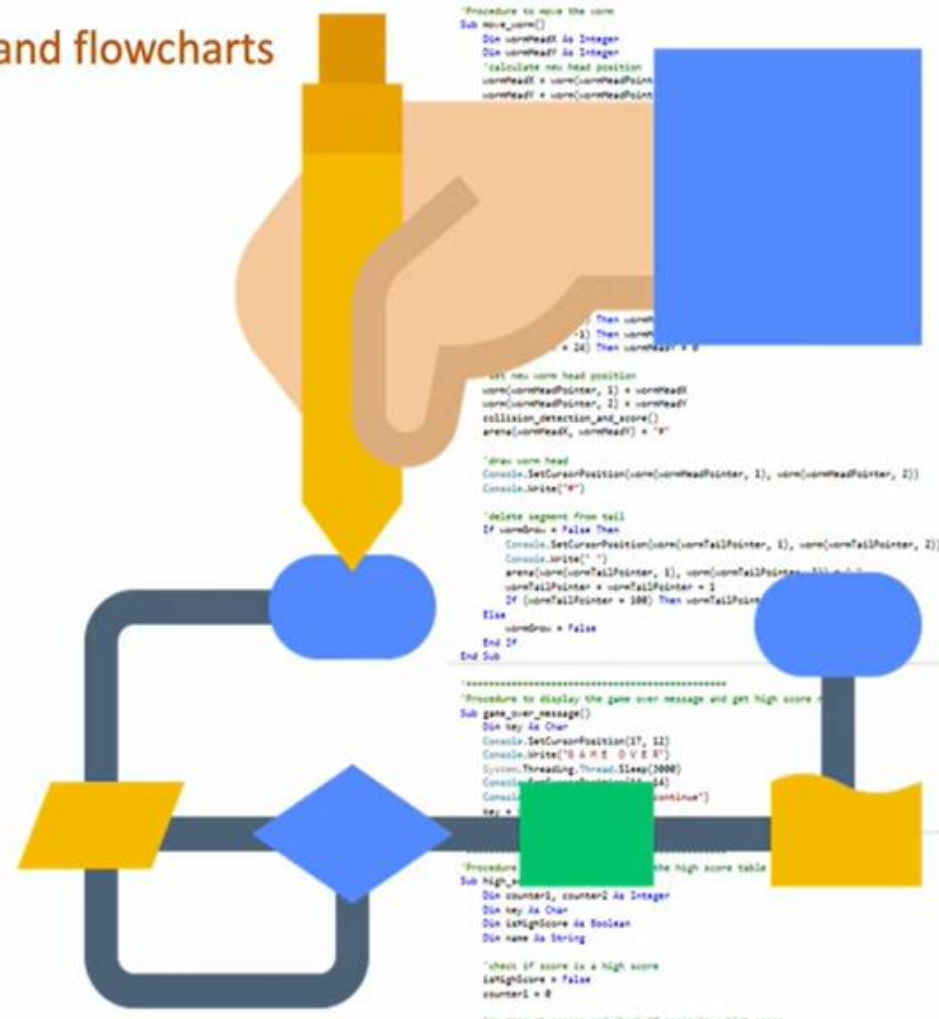


7.2 - (c) Use different methods to **design** and **construct** a solution to a problem

How to produce algorithms using pseudocode and flowcharts

In the exam you need to be able to create, interpret, correct and refine **algorithms** using:

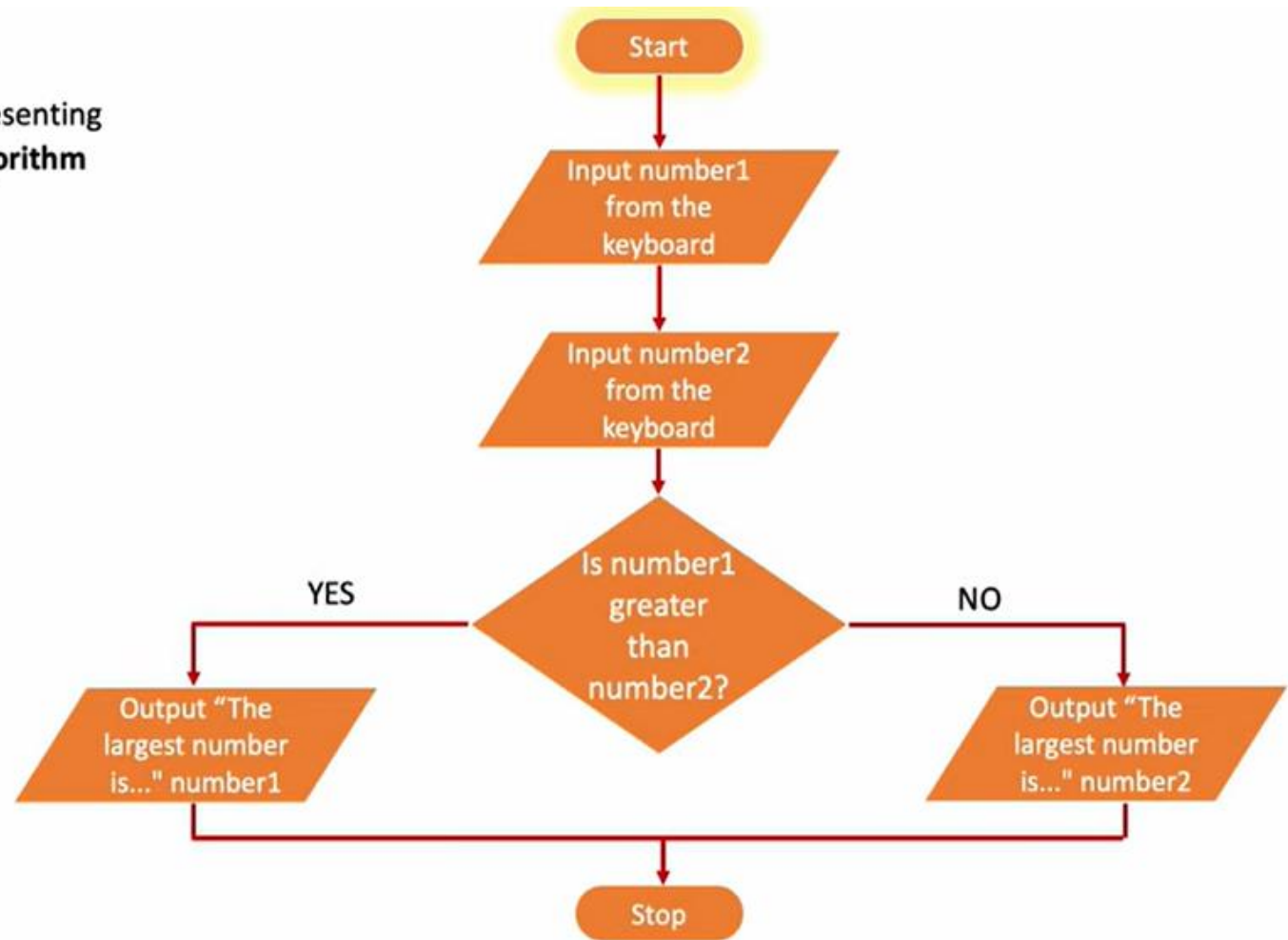
- **Pseudocode**
- **Flowcharts**



7.2 - (c) Use different methods to design and construct a solution to a problem

Flowcharts

A **flowchart** is a method of representing the sequences of steps in an **algorithm** in the form of a diagram.



7.2 - (c) Use different methods to design and construct a solution to a problem



Pseudocode

Pseudocode is an alternative, text-based way of representing the sequence of steps in an **algorithm**.

The prefix *pseudo* means *false* or *not genuine*.

Pseudocode can be thought of as a simplified form of programming code.

It allows us to lay down the logic of a problem in an *almost-like-real-code* way without worrying about the actual rules and syntax of a particular language.



7.2 - (c) Use different methods to design and construct a solution to a problem

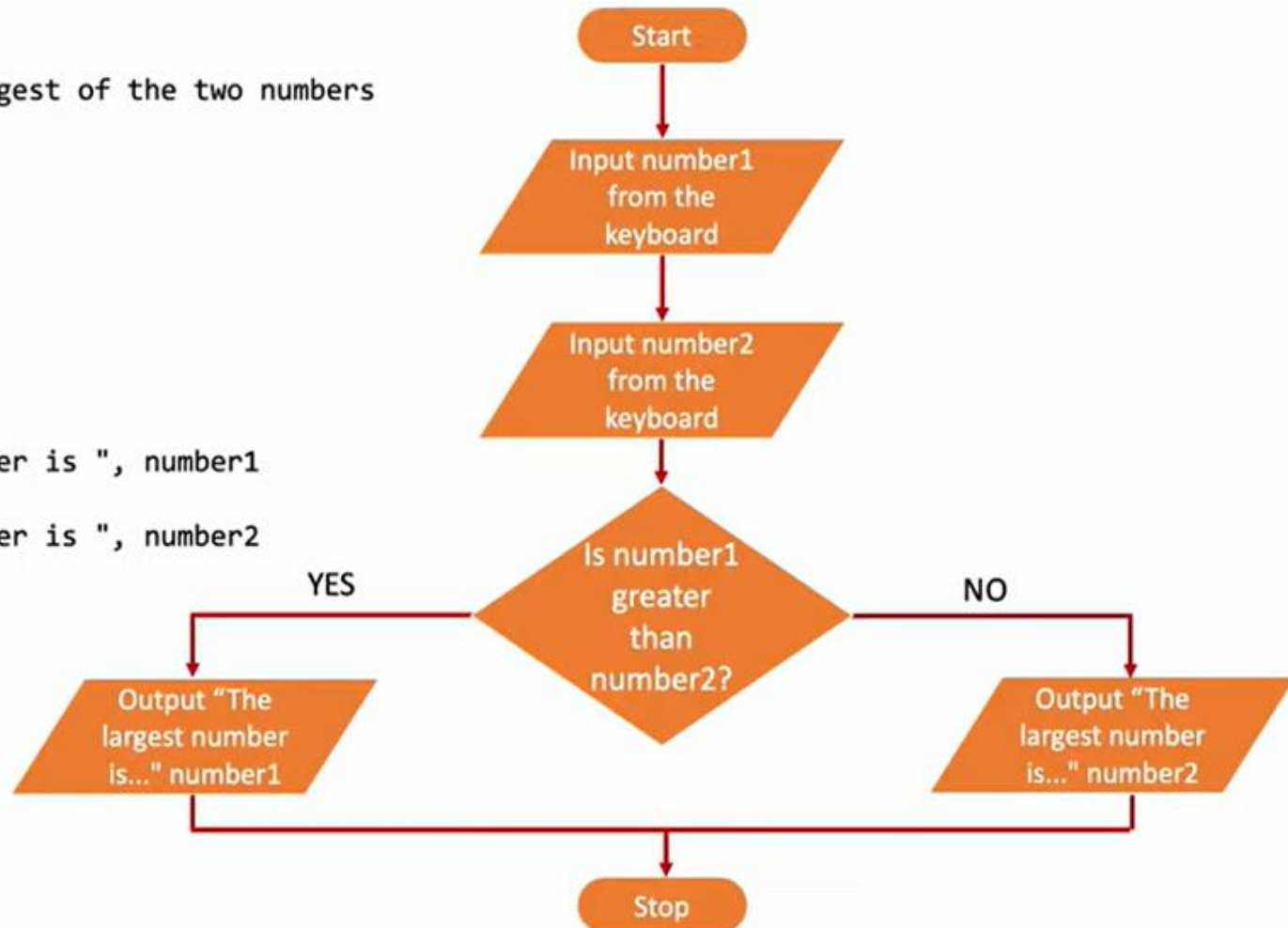
Pseudocode

//Program to output the largest of the two numbers

```
DECLARE number1 : INTEGER  
DECLARE number2 : INTEGER
```

```
INPUT number1  
INPUT number2
```

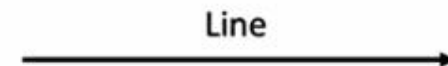
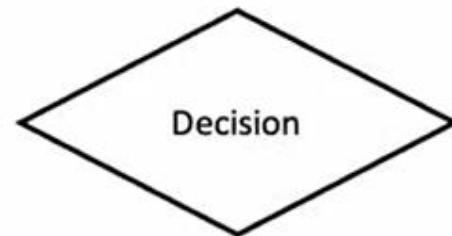
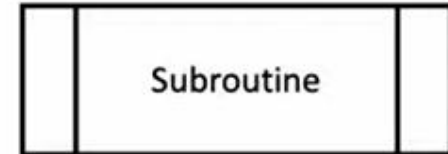
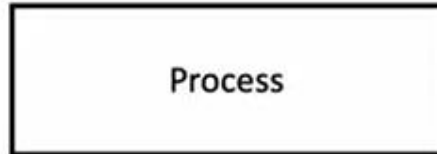
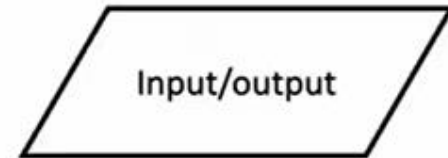
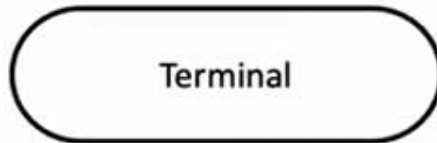
```
IF number1 > number2  
  THEN  
    OUTPUT "The largest number is ", number1  
  ELSE  
    OUTPUT "The largest number is ", number2  
ENDIF
```



7.2 - (c) Use different methods to **design** and **construct** a solution to a problem



Flowchart symbols



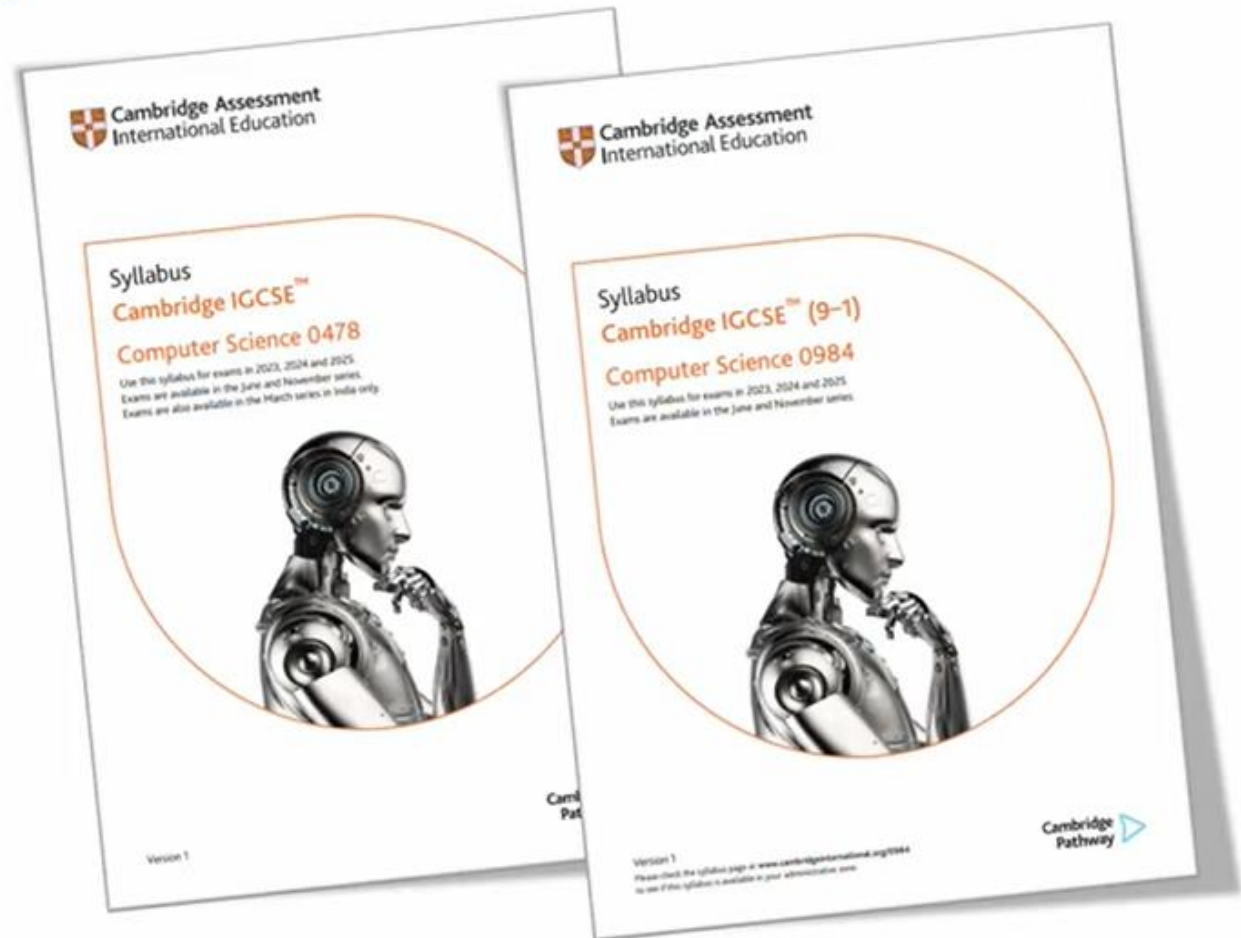
7.2 - (c) Use different methods to **design** and **construct** a solution to a problem

Cambridge IGCSE pseudocode

The pseudocode in these videos follows the exact style which will be used in your exams.

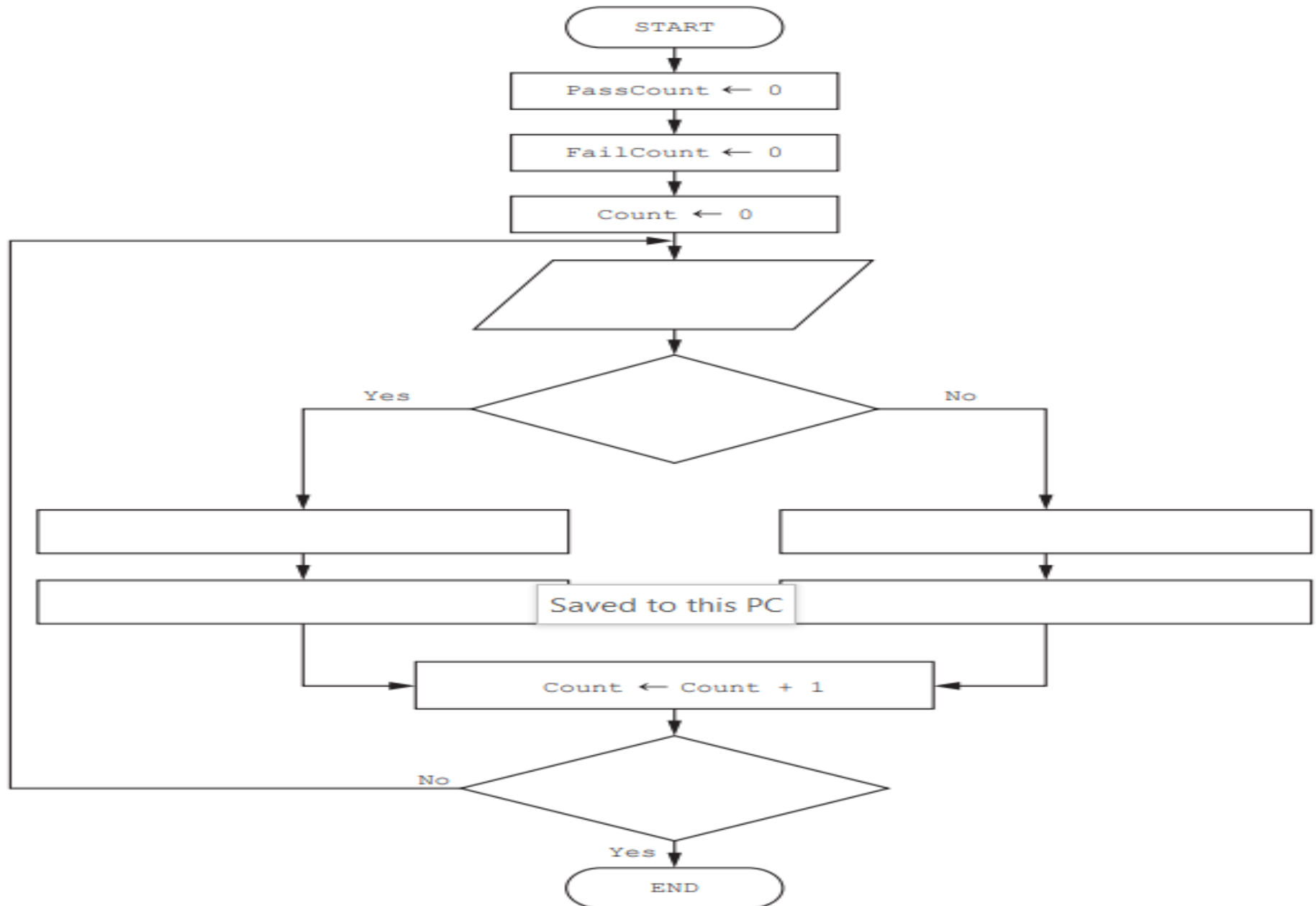
The full set of pseudocode is defined in the syllabus:

- Cambridge IGCSE (9-1)
Computer Science – 0984
- Cambridge IGCSE (A*-G)
Computer Science – 0478



- 4 The flowchart shows an algorithm that should allow 60 test results to be entered into the variable *Score*. Each test result is checked to see if it is 50 or more. If it is, the test result is assigned to the *Pass* array. Otherwise, it is assigned to the *Fail* array.

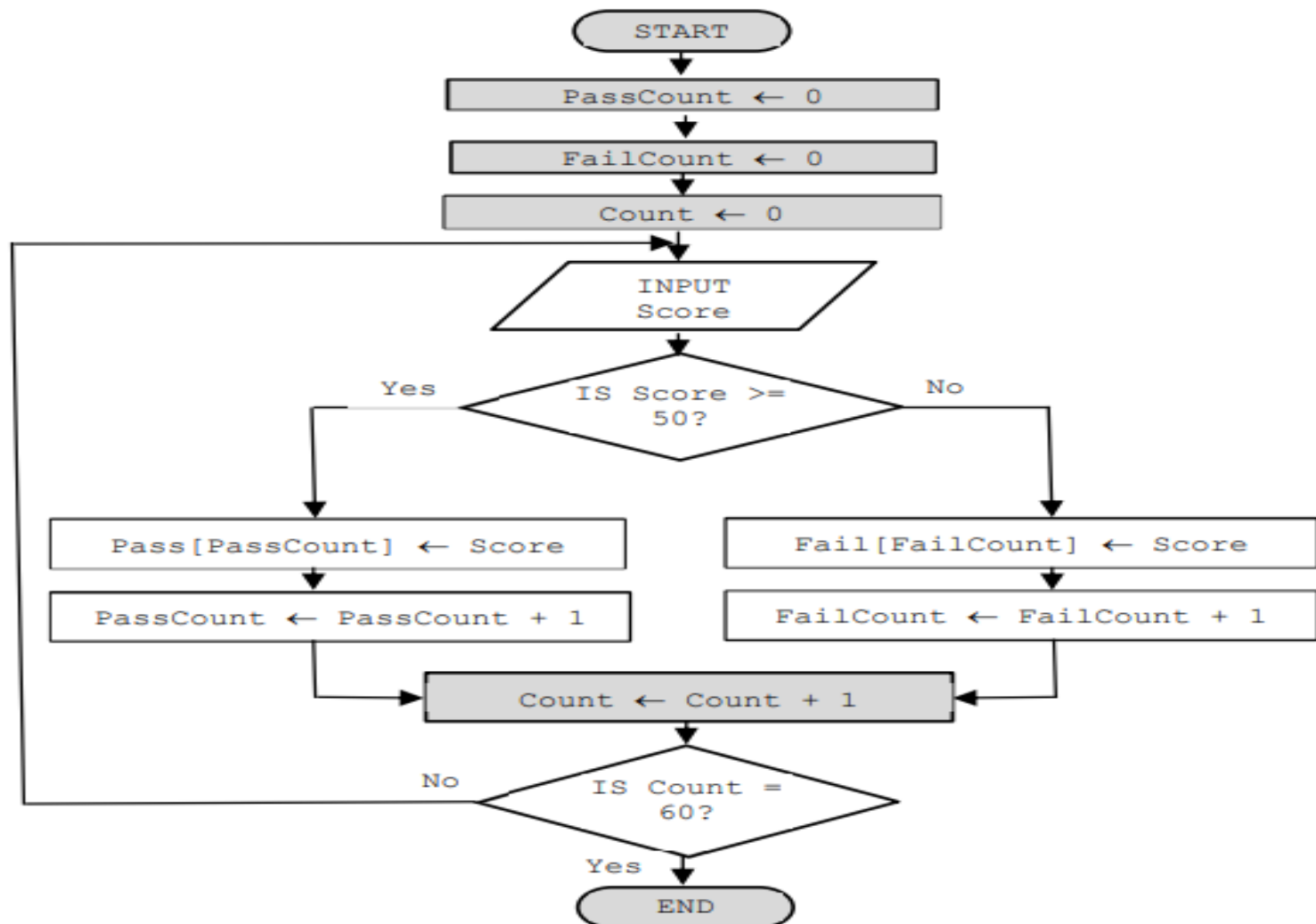
(a) Complete this flowchart:



4(a)

One mark per mark point, max six

- MP1 input box
MP2 correct check of Score
MP3 assign Score to Pass correctly
MP4 assign Score to Fail correctly
MP5 increment **both** array counters
MP6 correct check of number of scores



7.2 - (c) Use different methods to design and construct a solution to a problem

Past paper exams Question 2:

- (b) Write a pseudocode routine that will check that each test result entered into the algorithm is between 0 and 100 inclusive.

[4]

7.2 - (c) Use different methods to **design** and **construct** a solution to a problem

Past paper exams Answer 2:

Question	Answer	Marks
4(b)	One mark per mark point, max four MP1 appropriate conditional loop structure MP2 correct identification of invalid input MP3 appropriate error message MP4 repeated input of score until correct <pre>WHILE Score < 0 OR Score > 100 (DO) OUTPUT "Your entry must be between 0 and 100, inclusive, please try again " INPUT Score ENDWHILE</pre> Or: <pre>REPEAT IF Score < 0 OR Score > 100 THEN OUTPUT "Your entry must be between 0 and 100, inclusive, please try again " INPUT Score ENDIF UNTIL Score >= 0 AND Score <= 100</pre>	4

2 An algorithm has been written to:

- input the ages of 100 students
- count and output the number of students aged 7 and under 12
- count and output the number of students aged 12 and under 18
- count and output the number of students aged 18 and over.

(a) Complete the pseudocode algorithm:

```
01 Count7to12 ← 0
02 Count12to18 ← 0
03 CountOver18 ← 0
04 FOR Student ← 1 TO .....
05     OUTPUT "Please enter student's age in years "
06     INPUT Age
07     IF Age >= 7 .....
08         THEN
09             Count7to12 ← Count7to12 + 1
10     ENDIF
11     IF Age >= 12 AND Age < 18
12         THEN
13             Count12to18 ← .....
14     ENDIF
15     IF Age >= 18
16         THEN
17             CountOver18 ← CountOver18 + 1
18     ENDIF
19 NEXT Student
20 OUTPUT "There are ", Count7to12, " students aged 7 and under 12."
21 OUTPUT "There are ", Count12to18, " students aged 12 and under 18."
22 OUTPUT "There are ", ....., " students aged 18 and over."
```

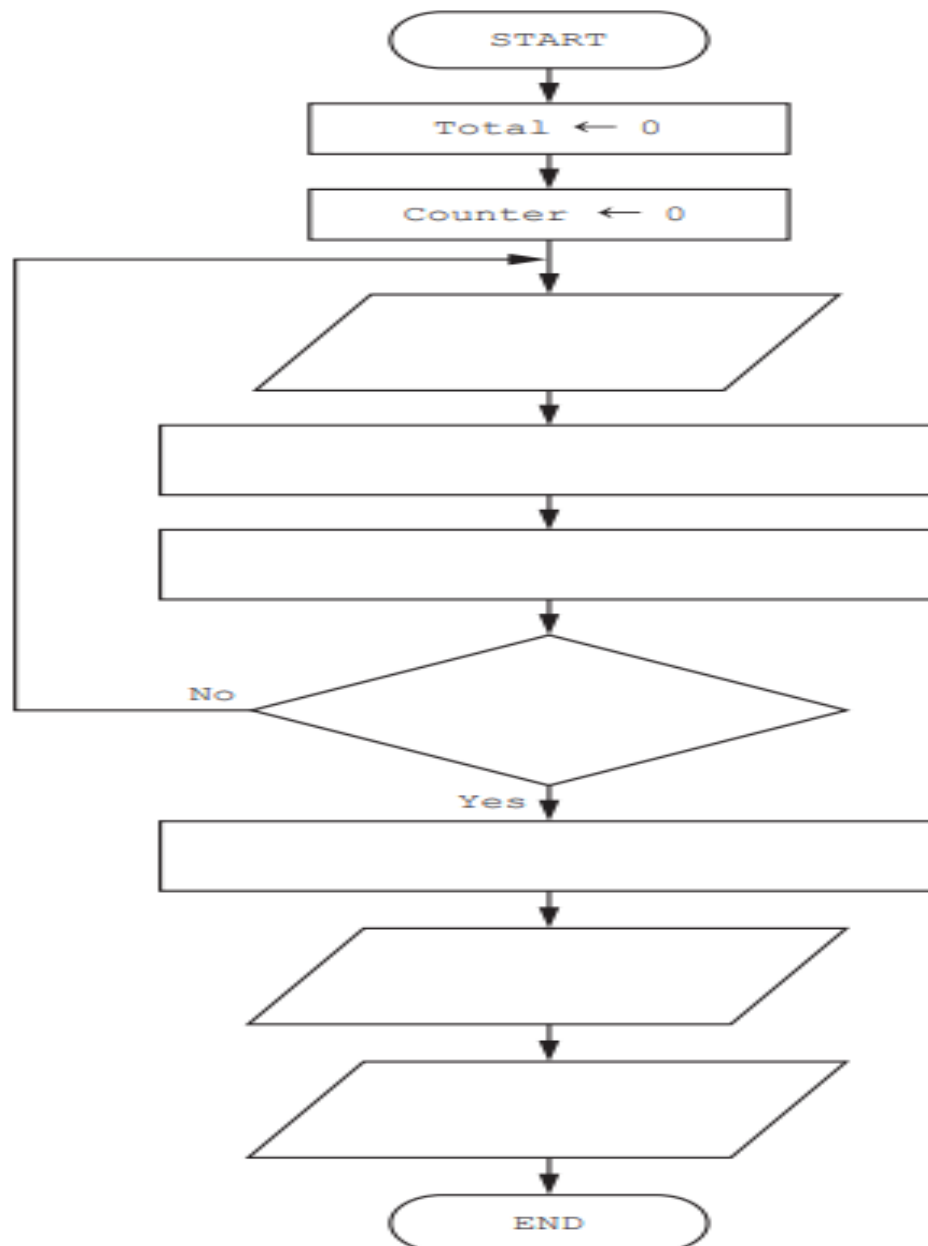
7.2 - (c) Use different methods to **design** and **construct** a solution to a problem

Past paper exams Answer 3:

Question	Answer	Marks
Section B		
2(a)	One mark per mark point, max four • 100 • AND Age < 12 • Count12to18 + 1 • CountOver18	4
2(b)	One mark suitable IF construct, one mark correct assignment statement, for example IF Age < 7 THEN CountUnder7 ← CountUnder7 + 1 ENDIF One mark suitable message, one mark correct use of countUnder7 variable, for example OUTPUT "There are ", CountUnder7, " students aged under 7."	4

- 5 The flowchart shows an algorithm that should:
- allow 100 numbers to be entered into the variable `Number`
 - total the numbers as they are entered
 - output the total and average of the numbers after they have all been entered.

Complete this flowchart:



5

One mark per mark point, max **six**

MP1 input box

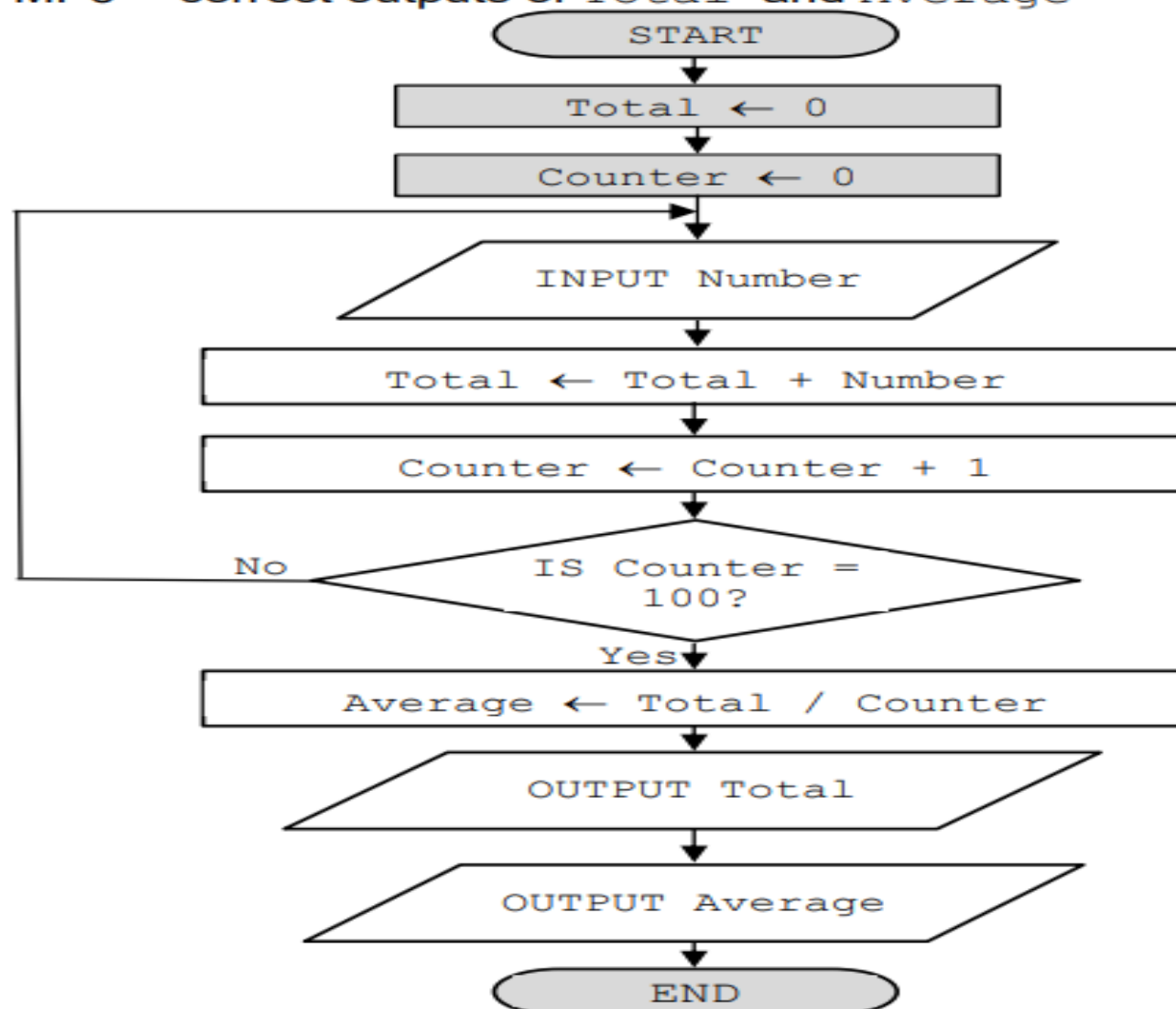
MP2 correct totalling using Total

MP3 correct counting using Counter

MP4 correct conditional statement for Counter

MP5 correct calculation of Average

MP6 correct outputs of Total and Average



7.3 - Explain the **purpose** of a given **algorithm**

Click the link below to watch the video for what is an algorithm

<https://www.youtube.com/watch?v=qiMpOYKlruc&list=PLCiOXwirraUCKOH2duohyLRk9O75bgUdl&index=7>

7 Algorithm design and problem-solving

What is an algorithm?

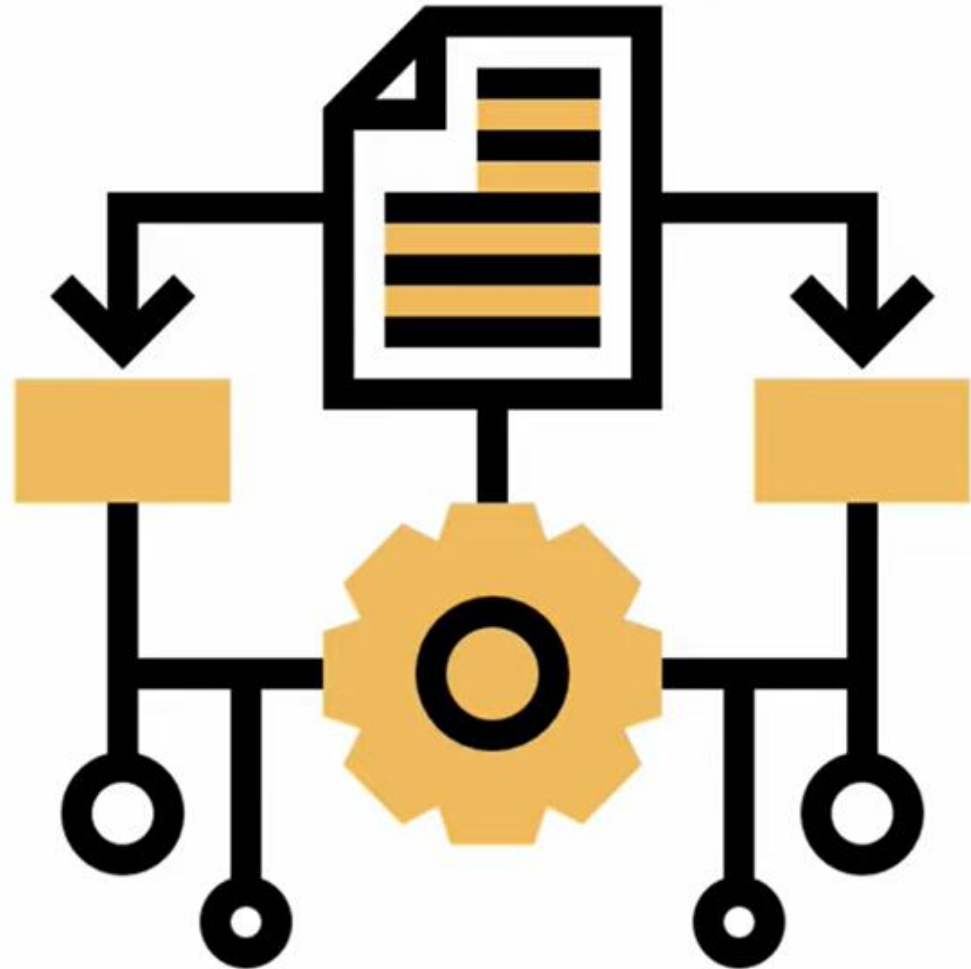


2:07
stude

7.3 - Explain the purpose of a given algorithm

What is an algorithm?

An **algorithm** is nothing more than a specific sequence of steps that can be followed to complete a task.



7.3 - Explain the purpose of a given algorithm

What is an algorithm?

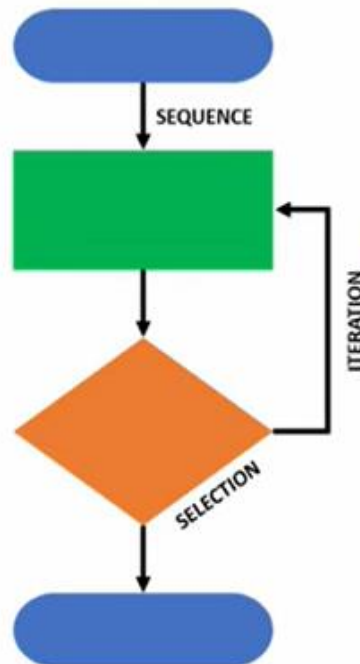
An **algorithm** is nothing more than a specific sequence of steps that can be followed to complete a task.

Making a sandwich

1. White or brown bread?
2. Spread?
 - a. Butter or margarine?
3. Sauce?
4. Filling
 - a. Lettuce?
 - b. Tomatoes?
 - c. Bacon?

A computer program is an implementation of an **algorithm**.

An **algorithm** is *not* a computer program in itself.



7.3 - Explain the **purpose** of a given **algorithm**

Explaining the purpose of an algorithm

In the exam, you will need to be able to explain the purpose of a specific **algorithm**.

The **algorithm** might be presented as:

- A **flowchart**
- **Pseudocode**



Covered in more detail in the following video:

- **7 How to produce algorithms using pseudocode and flowcharts**



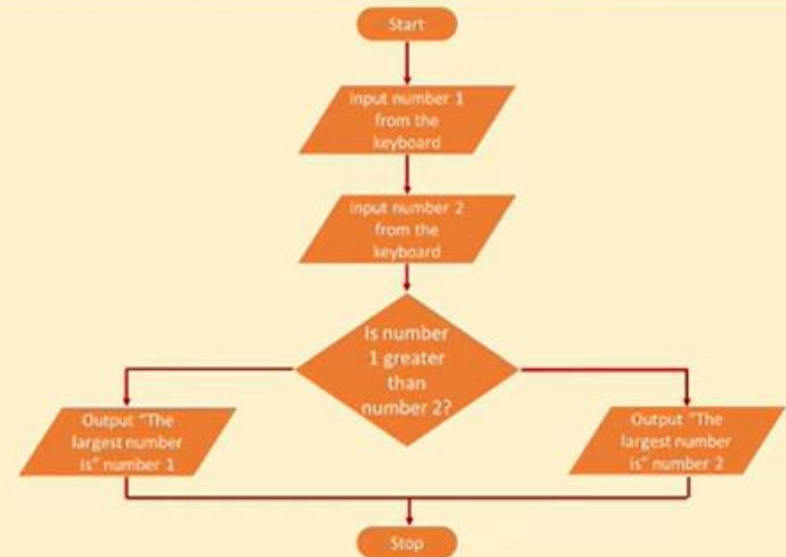
7.3 - Explain the **purpose** of a given **algorithm**

Explaining the purpose of an algorithm

Algorithm presented as pseudocode

```
//Program to output the largest of the two numbers  
  
DECLARE number1 : INTEGER  
DECLARE number2 : INTEGER  
  
INPUT number1  
INPUT number2  
  
IF number1 > number2  
    THEN  
        OUTPUT "The largest number is ", number1  
    ELSE  
        OUTPUT "The largest number is ", number2  
    ENDIF
```

Algorithm presented as a flowchart



The **pseudocode** and **flowchart** above both represent the same algorithm.

Implementing this algorithm as a solution will produce a computer program that asks the user to input two numbers and outputs which of the two is larger.

7.3 - Explain the **purpose** of a given **algorithm**



Algorithms

What is an algorithm?

- An **algorithm** is a specific sequence of steps that can be followed to complete a task.
- A computer program is an implementation of an **algorithm**.
- An **algorithm** is *not* a computer program in itself.

7.3 - Explain the **purpose** of a given **algorithm**

Stating the Purpose of an Algorithm:

An algorithm is a step-by-step procedure or formula for solving a problem or completing a task. Algorithms are designed to achieve a specific objective or solve a particular problem efficiently and accurately. When stating the purpose of an algorithm, it is essential to clearly define what problem it addresses and what the expected outcome is. The purpose should be specific and concise, outlining the problem-solving goal the algorithm aims to achieve

Example of Stating the Purpose:

"The purpose of the algorithm is to sort a list of integers in ascending order."

In this example, the purpose is clear: the algorithm aims to arrange a given list of integers in ascending order, ensuring that the numbers are sorted from smallest to largest.

Describing the Processes Involved in an Algorithm:

When describing the processes involved in an algorithm, it is crucial to detail the individual steps or operations that the algorithm performs. This description should provide a clear and precise explanation of how the algorithm achieves its purpose. It often involves breaking down the problem into smaller subproblems, defining the sequence of steps to be executed, and specifying any conditions or loops involved.

7.3 - Explain the **purpose** of a given **algorithm**

Example of Describing Processes: Consider the problem of finding the factorial of a non-negative integer. The factorial of a number is the product of all positive integers from 1 up to that number.

Algorithm Purpose: *"The purpose of the algorithm is to calculate the factorial of a non-negative integer."*

Algorithm Processes:

- 1-Input: Receive a non-negative integer n as input.
- 2-Initialization: Set factorial to **1** (initial factorial value).
- 3-Loop: For each integer i from **1** to **n** , repeat steps **4** and **5**.
- 4-Multiplication: Multiply factorial by **i** .
- 4-Update Factorial: Update factorial with the result of the multiplication.
- 5-Output: Return the calculated factorial as the output.

This description breaks down the algorithm into its individual processes, clearly outlining the steps involved in calculating the factorial of a non-negative integer.

Past Paper 2 Exam Questions and Answers

Question 1:

1 (a) Four descriptions of stages in the program development life cycle are shown.

Draw **one** line to link each description to its most appropriate program development life cycle stage.

Not all program development life cycle stages will be used.

Program development life cycle description

develop an algorithm to solve the problem by using structure diagrams, flowcharts or pseudocode

detect and fix the errors in the program

identify the problem and its requirements

write and implement the instructions to solve the problem

Program development life cycle stage

analysis

coding

design

evaluation

testing

[4]

Past Paper 2 Exam Questions and Answers

Question 1:

(b) Identify **three** of the component parts after a problem has been decomposed.

1

.....

2

.....

3

.....

[3]

Past Paper 2 Exam Questions and Answers

Answer 1:

Question	Answer	Marks
1(a)	One mark for each correct line. Program development life cycle description develop an algorithm to solve the problem using structure diagrams, flowcharts or pseudocode detect and fix the errors in the program identify the problem and its requirements write and implement the instructions to solve the problem Program development life cycle stage analysis coding design evaluation testing	4
1(b)	One mark for naming or describing each component part, max three For example: inputs // what is put into the system processes // actions taken to achieve a result outputs // what is taken out of the system storage // what needs to be kept for future use	3

Past Paper 2 Exam Questions and Answers

Question 2:

1 Tick (✓) **one** box to identify the first stage of the program development life cycle.

- | | | |
|----------|----------|--------------------------|
| A | Analysis | ☐ |
| B | Coding | ☐ |
| C | Design | ☐ |
| D | Testing | ☐ |

3 Identify **three** different ways that the design of a solution to a problem can be presented.

- 1
-
- 2
-
- 3
-

Past Paper 2 Exam Questions and Answers

Answer 2:

Question	Answer	Marks
1	A	1

Question	Answer	Marks
3	One mark for each correct answer • structure diagram / chart • flowchart • pseudocode	3

Past Paper 2 Exam Questions and Answers

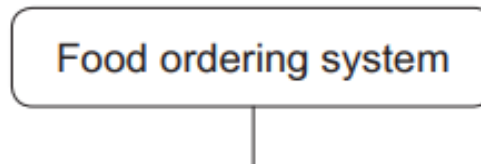
Question 3:

- 5 A food ordering system is an example of a computer system that is made up of sub-systems.

The food ordering system:

- allows the user to enter the details of the food they want to order and to pay for the order
- displays food available as pictures or as a list.

Complete the structure diagram for the given parts of the food ordering system.



Past Paper 2 Exam Questions and Answers

Answer 3:

Question	Answer	Marks
5	One mark for a suitable hierarchical structure One mark for suitable names for the sub systems for user input and display options One mark for sub systems for user inputs, (choice of display,) food order and payment One mark for sub systems for display output types, pictures and list For example: <pre>graph TD; A[Food ordering system] --> B[User Input]; A --> C[Display options]; B --> D[Display choice]; B --> E[Food order]; B --> F[Payment]; C --> G[Pictures]; C --> H[List];</pre>	4

Learning Objectives:

- 4 Understand standard methods of solution
 - Limited to:
 - linear search
 - bubble sort
 - totalling
 - counting
 - finding maximum, minimum and average values
 - Including:
 - range check
 - length check
 - type check
 - presence check
 - format check
 - check digit
 - the purpose of each validation check and writing algorithms to implement each validation check
- 5 (a) Understand the need for validation checks to be made on input data and the different types of validation check
 - Including:
 - visual check
 - double entry check
 - The purpose of each verification check
- (b) Understand the need for verification checks to be made on input data and the different types of verification check

7.4 - Understand **standard** methods of solution

Various **standard methods** of solution used to include:

1– linear search

2– bubble sort

3– totalling

4– counting

5– finding maximum, minimum and average values

7.4 - Understand **standard** methods of solution

[Click the video to watch Linear search](#)

<https://www.youtube.com/watch?v=CX2CYIJLwfg>

```
linearSearch(key, array[]):  
    for (i = 0; i < length(array); i++):  
        if (array[i] == key):  
            return i  
  
    return -1
```

7.4 - Understand **standard** methods of solution

1- Linear Search:

The linear search(or sequential search) algorithm is a simple search algorithm that starts at the left-hand side of an array (index 0) and moves through the array one item at a time. Once the item being searched for is found the algorithm returns the index of the item in question. If the algorithm reaches the end of the array without finding the item, then it either returns an error or it returns a non-valid index depending on the implementation.

Linear search is the most basic kind of search algorithm. It involves checking each item of the list (beginning to end), until the desired item is found.

Characteristic: Linear search involves checking each element in the array one by one until the target element is found or the entire array is searched.

Example: Let's say you have an array of numbers [3, 7, 1, 9, 5], and you want to find the number '9'. Linear search would start from the beginning and find '9' after checking the third element.

7.4 - Understand standard methods of solution

1. Linear Search: Example 3

```
main.py ×  
1 # Given list of names  
2 NAMES = ["Nuriya", "Artur", "Danial", "Inaaya", "Harsh", "Ivan", "Artur", "Ivan", "Beiza", "Nicholas", "Darius", "Samina", "Emir"]  
3  
4 # Function to check if the first letter of the name is 'I'  
5 usage  
6 def firstLetter(name):  
7     return name[0].upper() == 'I'  
8  
9 # Linear search implementation based on pseudocode  
10 index = 0  
11 while index < len(NAMES):  
12     name = NAMES[index]  
13     if firstLetter(name):  
14         print(name)  
15     index += 1
```

```
C:\Users\n.tadjerout\PycharmProjects  
Inaaya  
Ivan  
Ivan
```

-**index** is a variable used to keep track of the current position while iterating through the **NAMES** list.
-**index += 1** increments the value of **index** by **1** in each iteration of the loop. It moves the index to the next position in the list, allowing the loop to progress to the next element.

7.4 - Understand **standard** methods of solution

1-Linear Search: **Pseudocode for Linear search example 3:**

**NAMES = "Nuriya", " Artur", " Danial", " Inaaya", " Harsh", " Ivan", " Artur", " Ivan", " Beiza",
Nicholas", " Darius", " Samina", " Emir"**

output "These names start with I"

loop while NAMES.hasNext()

NAME = NAMES.getNext()

if firstLetter(NAME) = "I" then

output NAME

end if

end loop

7.4 - Understand standard methods of solution

1. Linear Search: Example 1

Open Python IDLE and copy the code below for more practical activities

Example 1:

area.py - C:/Users/v90001010/AppData/Local/Programs/Python/Python37-32/area.py (3.7

File Edit Format Run Options Window Help

```
def linearsearch(arr, x):  
    for i in range(len(arr)):  
        if arr[i] == x:  
            return i  
    return -1  
  
arr = ['N', 'O', 'U', 'R', 'E', 'D', 'D', 'I', 'N', 'E']  
x = 'I'  
print("element found at index "+str(linearsearch(arr,x)))
```

```
...  
RESTART: C:/Users/v900010  
element found at index 7  
.
```

In Python programming, **def** is a keyword used to define a function. Functions in Python are blocks of code that perform a specific task and can be **reused** throughout the program. Inside the **parentheses ()**, you can specify **parameters (input values)**, **def function_name(parameters):** # Function body (code to be executed)

7.4 - Understand standard methods of solution

1. Linear Search: Example 2

```
main.py ×  
1 usage  
2 def linear_search(names, target_name):  
3     for index, name in enumerate(names):  
4         if name == target_name:  
5             return index # Returns the index of the target name if found  
6         return -1 # Returns -1 if the target name is not in the list  
7  
8 # List of names  
9 names = ["Nuriya", "Artur", "Danial", "Inaaya", "Harsh", "Ivan", "Artur", "Ivan", "Beiza", "Nicholas", "Darius", "Samina", "Emir"]  
10  
11 # Target name to search for  
12 target_name = "Harsh"  
13  
14 # Perform linear search  
15 result = linear_search(names, target_name)  
16  
17 if result != -1:  
18     print(f"{target_name} found at index {result}")  
19 else:  
20     print(f"{target_name} not found in the list")
```

```
C:\Users\n.tadgerout\PycharmProjects\  
Harsh found at index 4
```

7.4 - Understand **standard** methods of solution

1. **Linear Search:** Example 2

In Python, **enumerate** is a built-in function that is used to iterate over a sequence (such as a list, tuple, or string) along with keeping track of the index of the current item. It returns pairs of the form (index, element).

Example:

```
main.py ×  
1 # Example list  
2 fruits = ['apple', 'banana', 'cherry']  
3  
4 # Using enumerate to iterate over the list with both index and element  
5 for index, fruit in enumerate(fruits):  
6     print(f"Index {index}: {fruit}")  
7
```

```
C:\Users\n.tadgerout\Pycho  
Index 0: apple  
Index 1: banana  
Index 2: cherry
```

7.4 - Understand **standard** methods of solution

1-Linear Search: Pseudocode for Linear search for example 2:

FUNCTION LinearSearch(names: ARRAY[1: n] OF STRING, target_name: STRING) RETURNS INTEGER

FOR index $\leftarrow$ 1 TO n

IF names[index] = target_name **THEN**

RETURN index // Target name found, return the index

END IF

ENDFOR

RETURN -1 // Target name not found, return -1

END FUNCTION

In this pseudocode:

- **FUNCTION** LinearSearch(names: ARRAY[1: n] OF STRING, target_name: STRING) RETURNS INTEGER: This line defines a function named **LinearSearch** that takes an array of strings (**names**) and a target name (**target_name**) as parameters. It returns an integer representing the index of the target name in the array.
- **FOR** index $\leftarrow$ 1 TO n: This line starts a loop that iterates over the indices of the names array from **1** to **n**.
- **IF** names[index] = target_name **THEN**: Inside the loop, it checks if the current name at index **index** is equal to the **target_name**.
- **RETURN** index: If the names match, it returns the index **index** indicating that the target name was found.
- **RETURN** -1: If the loop finishes without finding the target name, it **returns -1**, indicating that the target name was not found in the list.

7.4 - Understand **standard** methods of solution

Click the link below to watch the video for Linear search

<https://www.youtube.com/watch?v=OqOb7hQ1cMw&list=PLCiOXwirraUCKOH2duohyLRk9O75bgUdI&index=9>

7 Algorithm design and problem-solving

Linear search

6:04



7.4 - Understand **standard** methods of solution



Linear search

Starting from the beginning of a data set, each item is checked in turn to see if it is the one being searched for.

A linear search:

- Doesn't require data to be in order.
- Works on any type of storage device.
- Is efficient for small data sets.
- Is very inefficient for large data sets.



7.4 - Understand standard methods of solution

Linear search

```
FUNCTION linearSearch(items, item_to_find) RETURNS STRING
  DECLARE index : INTEGER
  DECLARE found : BOOLEAN

  index ← 0
  found ← False

  WHILE (found = FALSE) AND (NOT end of data set) DO
    IF items[index] = item_to_find
      THEN
        found ← True
      ELSE
        index ← index + 1
    ENDIF
  ENDWHILE

  IF found = True
    THEN
      RETURN "Item found at position ", index
    ELSE
      RETURN "Item not found"
    ENDIF
ENDFUNCTION
```

We have declared a function called *linearSearch* that takes in two parameters.

The first is an array of data items called *items*.

The second is the item we are going to be searching for in the array, called *item_to_find*.

7.4 - Understand standard methods of solution

Linear search

```
FUNCTION linearSearch(items, item_to_find) RETURNS STRING
  DECLARE index : INTEGER
  DECLARE found : BOOLEAN

  index ← 0
  found ← False

  WHILE (found = FALSE) AND (NOT end of data set) DO
    IF items[index] = item_to_find
      THEN
        found ← True
      ELSE
        index ← index + 1
      ENDIF
    ENDWHILE

    IF found = True
      THEN
        RETURN "Item found at position ", index
      ELSE
        RETURN "Item not found"
      ENDIF
    ENDFUNCTION
```

We declare two additional local variables to use during the *linearSearch* function.

index: An integer that will be used to keep track of where we are in the *items* array.

found: A Boolean flag that we will set to *true* if we find the item we are searching for.

7.4 - Understand standard methods of solution

Linear search

```
FUNCTION linearSearch(items, item_to_find) RETURNS STRING
  DECLARE index : INTEGER
  DECLARE found : BOOLEAN
```

```
  index ← 0
  found ← False
```

```
  WHILE (found = FALSE) AND (NOT end of data set) DO
    IF items[index] = item_to_find
      THEN
        found ← True
      ELSE
        index ← index + 1
    ENDIF
  ENDWHILE
```

```
  IF found = True
    THEN
      RETURN "Item found at position ", index
    ELSE
      RETURN "Item not found"
    ENDIF
  ENDFUNCTION
```

Next, we initialise starting states for local variables.

We set *index* to 0, as we will be starting at the beginning of the array, position zero.

We set *found* to *false*, as we are yet to find the item we will be searching for.

7.4 - Understand standard methods of solution

Linear search

```
FUNCTION linearSearch(items, item_to_find) RETURNS STRING
  DECLARE index : INTEGER
  DECLARE found : BOOLEAN

  index ← 0
  found ← False

  WHILE (found = FALSE) AND (NOT end of data set) DO
    IF items[index] = item_to_find
      THEN
        found ← True
      ELSE
        index ← index + 1
      ENDIF
    ENDWHILE

  IF found = True
    THEN
      RETURN "Item found at position ", index
    ELSE
      RETURN "Item not found"
    ENDIF
  ENDFUNCTION
```

We then enter a WHILE loop until we find the item we are looking for or reach the end of the data set.

7.4 - Understand standard methods of solution

Linear search

```
FUNCTION linearSearch(items, item_to_find) RETURNS STRING
  DECLARE index : INTEGER
  DECLARE found : BOOLEAN

  index ← 0
  found ← False

  WHILE (found = FALSE) AND (NOT end of data set) DO
    IF items[index] = item_to_find
      THEN
        found ← True
      ELSE
        index ← index + 1
      ENDIF
    ENDWHILE

  IF found = True
    THEN
      RETURN "Item found at position ", index
    ELSE
      RETURN "Item not found"
    ENDIF
  ENDFUNCTION
```

We check if the item we are currently looking at in the array is the one we are searching for.

7.4 - Understand standard methods of solution

Linear search

```
FUNCTION linearSearch(items, item_to_find) RETURNS STRING
  DECLARE index : INTEGER
  DECLARE found : BOOLEAN

  index ← 0
  found ← False

  WHILE (found = FALSE) AND (NOT end of data set) DO
    IF items[index] = item_to_find
      THEN
        found ← True
      ELSE
        index ← index + 1
      ENDIF
    ENDWHILE

    IF found = True
      THEN
        RETURN "Item found at position ", index
      ELSE
        RETURN "Item not found"
      ENDIF
    ENDFUNCTION
```

If it is, then we set the *found* flag to *true*.

7.4 - Understand standard methods of solution

Linear search

```
FUNCTION linearSearch(items, item_to_find) RETURNS STRING
  DECLARE index : INTEGER
  DECLARE found : BOOLEAN

  index ← 0
  found ← False

  WHILE (found = FALSE) AND (NOT end of data set) DO
    IF items[index] = item_to_find
      THEN
        found ← True
      ELSE
        index ← index + 1
      ENDIF
    ENDWHILE

    IF found = True
      THEN
        RETURN "Item found at position ", index
      ELSE
        RETURN "Item not found"
      ENDIF
    ENDFUNCTION
```

If it isn't, we increment the *index* variable...

7.4 - Understand standard methods of solution

Linear search

```
FUNCTION linearSearch(items, item_to_find) RETURNS STRING
  DECLARE index : INTEGER
  DECLARE found : BOOLEAN

  index ← 0
  found ← False

  WHILE (found = FALSE) AND (NOT end of data set) DO
    IF items[index] = item_to_find
      THEN
        found ← True
      ELSE
        index ← index + 1
      ENDIF
    ENDWHILE

  IF found = True
    THEN
      RETURN "Item found at position ", index
    ELSE
      RETURN "Item not found"
    ENDIF
  ENDFUNCTION
```

...and repeat the process.

7.4 - Understand standard methods of solution

Linear search

```
FUNCTION linearSearch(items, item_to_find) RETURNS STRING
  DECLARE index : INTEGER
  DECLARE found : BOOLEAN

  index ← 0
  found ← False

  WHILE (found = FALSE) AND (NOT end of data set) DO
    IF items[index] = item_to_find
      THEN
        found ← True
      ELSE
        index ← index + 1
      ENDIF
    ENDWHILE

    IF found = True
      THEN
        RETURN "Item found at position ", index
      ELSE
        RETURN "Item not found"
      ENDIF
    ENDFUNCTION
```

Eventually, we will exit the WHILE loop.

At this point, we use an IF statement to check the value held in the *found* variable.

If it contains *true*, we return a message that the item was found along with its position in the array.

7.4 - Understand standard methods of solution

Linear search

```
FUNCTION linearSearch(items, item_to_find) RETURNS STRING
  DECLARE index : INTEGER
  DECLARE found : BOOLEAN

  index ← 0
  found ← False

  WHILE (found = FALSE) AND (NOT end of data set) DO
    IF items[index] = item_to_find
      THEN
        found ← True
      ELSE
        index ← index + 1
      ENDIF
    ENDWHILE

    IF found = True
      THEN
        RETURN "Item found at position ", index
      ELSE
        RETURN "Item not found"
      ENDIF
    ENDFUNCTION
```

If it contains *false*, we return a message that the item could not be found.

7.4 - Understand **standard** methods of solution

Cambridge IGCSE pseudocode

The pseudocode in these videos follows the exact style that will be used in the exams.

The full set of pseudocode is defined in the syllabus:

- Cambridge IGCSE (9-1)
Computer Science – 0984
- Cambridge IGCSE (A*-G)
Computer Science – 0478



7.4 - Understand **standard** methods of solution

2- Bubble Sort:

Bubble sort unsorted list of items and it is a simple sorting algorithm that repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order.

How Bubble Sort Works example 1?

We take an unsorted array for our example.



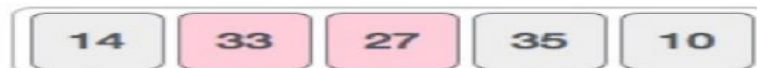
Bubble sort starts with very first two elements, comparing them to check which one is greater.



In this case, value 33 is greater than 14, so it is already in sorted locations. Next, we compare 33 with 27.



We find that 27 is smaller than 33 and these two values must be swapped.



The new array should look like this –



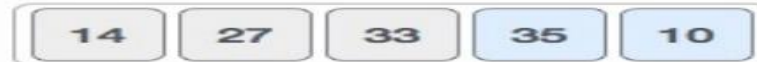
7.4 - Understand **standard** methods of solution

2- Bubble Sort:

Next we compare 33 and 35. We find that both are in already sorted positions.



Then we move to the next two values, 35 and 10.



We know then that 10 is smaller 35. Hence they are not sorted.



We swap these values. We find that we have reached the end of the array. After one iteration, the array should look like this –



To be precise, we are now showing how an array should look like after each iteration. After the second iteration, it should look like this –



Notice that after each iteration, at least one value moves at the end.



And when there's no swap required, bubble sorts learns that an array is completely sorted.

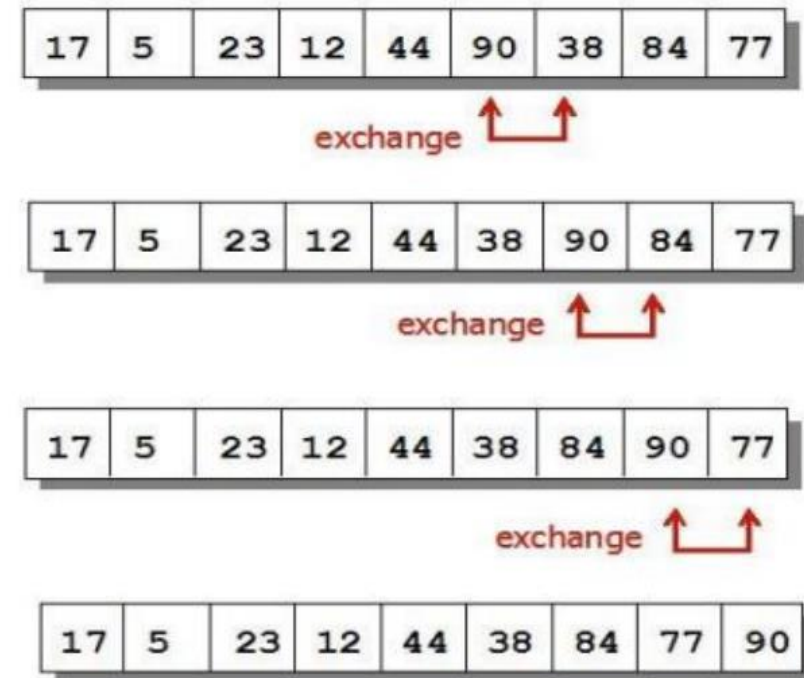
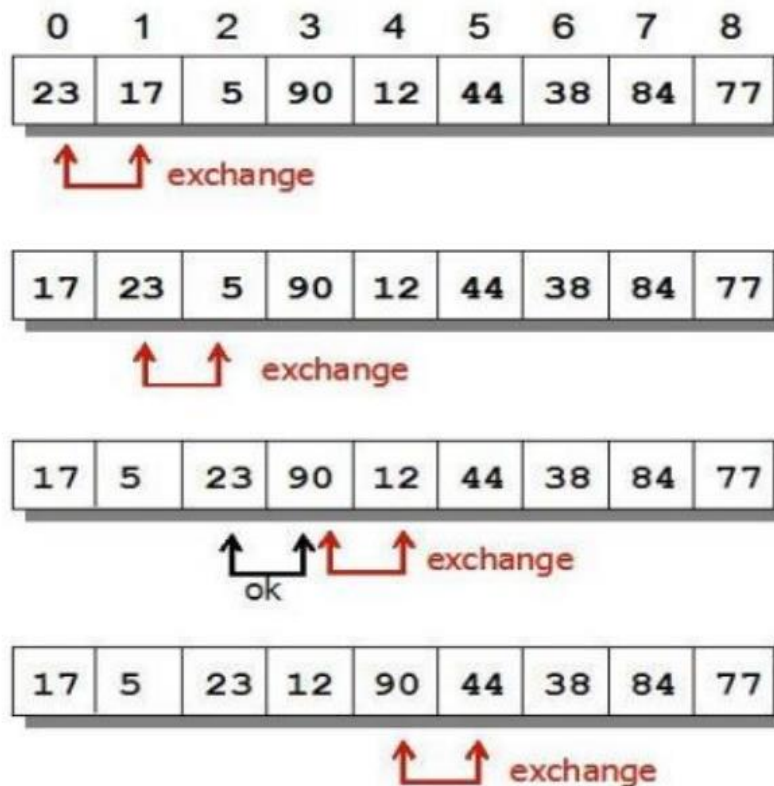


Now we should look into some practical aspects of bubble sort.

7.4 - Understand standard methods of solution

2- Bubble Sort:

How Bubble Sort Works example 2?



The largest value 90 is at the end of the list.

7.4 - Understand **standard** methods of solution

2- Bubble Sort:

Click the link below to watch the video for the Bubble sort

<https://www.youtube.com/watch?v=IVBYtXxuZiM&list=PLCiOXwirraUCKOH2duohyLRk9O75bgUdI&index=9>

7 Algorithm design and problem-solving

Bubble sort

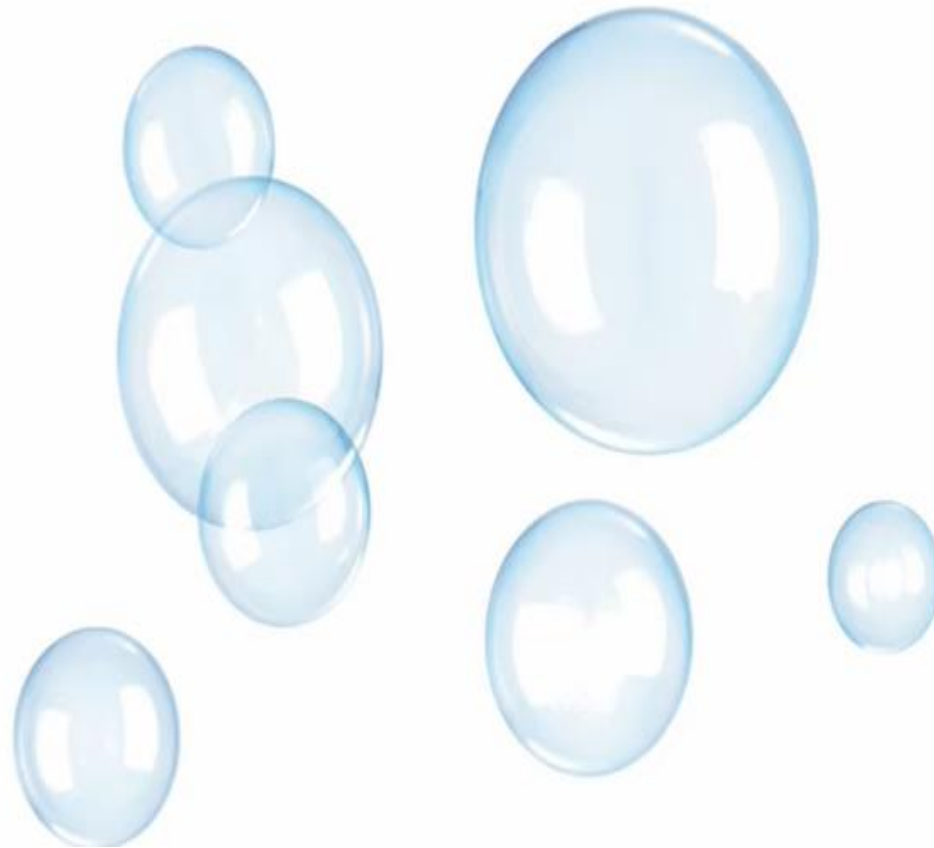
7.4 - Understand **standard** methods of solution

2- Bubble Sort:



Bubble sort

- Sorts an unordered list of items.
- Compares each item with the next and swaps them if they are out of order – in effect, *bubbling* the largest (or smallest) item up to the end of the list.
- Finishes when no more swaps need to be made.
- The most inefficient of the sorting algorithms but very easy to implement.
- A popular choice for very small data sets.



7.4 - Understand **standard** methods of solution

2- Bubble Sort:

Bubble sort

```
FUNCTION bubbleSort(items)
  DECLARE n : INTEGER
  DECLARE swapped : BOOLEAN

  n ← items.Length
  swapped ← True

  WHILE n > 0 AND swapped DO
    swapped ← False
    n ← n - 1
    FOR index ← 0 TO n - 1
      IF items[index] > items[index+1]
        THEN
          Swap(items[index], items[index+1])
          swapped ← true
        ENDF
    NEXT index
  ENDWHILE
  RETURN items
ENDFUNCTION
```

We have declared a function called *bubbleSort* that takes in one parameter – an array of data items we want to sort, called *items*.

7.4 - Understand standard methods of solution

2- Bubble Sort:

Bubble sort

```
FUNCTION bubbleSort(items)
  DECLARE n : INTEGER
  DECLARE swapped : BOOLEAN

  n ← items.Length
  swapped ← True

  WHILE n > 0 AND swapped DO
    swapped ← False
    n ← n - 1
    FOR index ← 0 TO n - 1
      IF items[index] > items[index+1]
        THEN
          Swap(items[index], items[index+1])
          swapped ← true
        ENDIF
    NEXT index
  ENDWHILE
  RETURN items
ENDFUNCTION
```

We declare two additional local variables to use during the *bubbleSort* function.

n: An integer used to keep track of how far through the *items* array we will need to travel on each pass through the WHILE loop.

swapped: A Boolean flag we set to *true* if we have just performed a swap during the sort function.

7.4 - Understand **standard** methods of solution

2- Bubble Sort:

Bubble sort

```
FUNCTION bubbleSort(items)
  DECLARE n : INTEGER
  DECLARE swapped : BOOLEAN

  n ← items.Length
  swapped ← True

  WHILE n > 0 AND swapped DO
    swapped ← False
    n ← n - 1
    FOR index ← 0 TO n - 1
      IF items[index] > items[index+1]
        THEN
          Swap(items[index], items[index+1])
          swapped ← true
        ENDIF
    NEXT index
  ENDWHILE
  RETURN items
ENDFUNCTION
```

We now set n to the length of the *items* array and initialise the *swapped* flag to *true*.

7.4 - Understand standard methods of solution

2- Bubble Sort:

Bubble sort

```
FUNCTION bubbleSort(items)
  DECLARE n : INTEGER
  DECLARE swapped : BOOLEAN

  n ← items.Length
  swapped ← True

  WHILE n > 0 AND swapped DO
    swapped ← False
    n ← n - 1
    FOR index ← 0 TO n - 1
      IF items[index] > items[index+1]
        THEN
          Swap(items[index], items[index+1])
          swapped ← true
        ENDIF
    NEXT index
  ENDWHILE
  RETURN items
ENDFUNCTION
```

Now we enter a WHILE loop that will run as long as n is greater than 0 and swaps are taking place.

7.4 - Understand **standard** methods of solution

2- Bubble Sort:

Bubble sort

```
FUNCTION bubbleSort(items)
  DECLARE n : INTEGER
  DECLARE swapped : BOOLEAN

  n ← items.Length
  swapped ← True

  WHILE n > 0 AND swapped DO
    swapped ← False
    n ← n - 1
    FOR index ← 0 TO n - 1
      IF items[index] > items[index+1]
        THEN
          Swap(items[index], items[index+1])
          swapped ← true
        ENDIF
    NEXT index
  ENDWHILE
  RETURN items
ENDFUNCTION
```

Every time we start a new pass through the WHILE loop, we set the *swapped* flag to *false*.

In other words, each time through the loop, we assume a swap doesn't need to be made until we discover otherwise.

7.4 - Understand standard methods of solution

2- Bubble Sort:

Bubble sort

```
FUNCTION bubbleSort(items)
  DECLARE n : INTEGER
  DECLARE swapped : BOOLEAN

  n ← items.Length
  swapped ← True

  WHILE n > 0 AND swapped DO
    swapped ← False
    n ← n - 1
    FOR index ← 0 TO n - 1
      IF items[index] > items[index+1]
        THEN
          Swap(items[index], items[index+1])
          swapped ← true
        ENDIF
    NEXT index
  ENDWHILE
  RETURN items
ENDFUNCTION
```

The array is zero-indexed – the first item starts at position 0, not position 1 – so we decrement the number of items to check by 1.

7.4 - Understand standard methods of solution

2- Bubble Sort:

Bubble sort

```
FUNCTION bubbleSort(items)
  DECLARE n : INTEGER
  DECLARE swapped : BOOLEAN

  n ← items.Length
  swapped ← True

  WHILE n > 0 AND swapped DO
    swapped ← False
    n ← n - 1
    FOR index ← 0 TO n - 1
      IF items[index] > items[index+1]
        THEN
          Swap(items[index], items[index+1])
          swapped ← true
        ENDIF
      NEXT index
    ENDWHILE
  RETURN items
ENDFUNCTION
```

Next we enter a FOR loop that will run n number of times – in other words, check every item.

7.4 - Understand **standard** methods of solution

2- Bubble Sort:

Bubble sort

```
FUNCTION bubbleSort(items)
  DECLARE n : INTEGER
  DECLARE swapped : BOOLEAN

  n ← items.Length
  swapped ← True

  WHILE n > 0 AND swapped DO
    swapped ← False
    n ← n - 1
    FOR index ← 0 TO n - 1
      IF items[index] > items[index+1]
        THEN
          Swap(items[index], items[index+1])
          swapped ← true
        ENDIF
    NEXT index
  ENDWHILE
  RETURN items
ENDFUNCTION
```

Now we use an IF statement to check an item in the array against the item next to it.

7.4 - Understand standard methods of solution

2- Bubble Sort:

Bubble sort

```
FUNCTION bubbleSort(items)
  DECLARE n : INTEGER
  DECLARE swapped : BOOLEAN

  n ← items.Length
  swapped ← True

  WHILE n > 0 AND swapped DO
    swapped ← False
    n ← n - 1
    FOR index ← 0 TO n - 1
      IF items[index] > items[index+1]
        THEN
          Swap(items[index], items[index+1])
          swapped ← true
        ENDIF
      NEXT index
    ENDWHILE
  RETURN items
ENDFUNCTION
```

If the item is greater than the one we are comparing it to, it needs to be swapped.

Here we have used a made-up function that will perform the swap for us. Instead, we could use a temporary variable to perform the swap:

```
temp ← items[index]
items[index] ← items[index + 1]
items[index + 1] ← temp
```

We also set the *swapped* flag to *true* to indicate that a swap has just taken place.

7.4 - Understand **standard** methods of solution

2- Bubble Sort:

Bubble sort

```
FUNCTION bubbleSort(items)
  DECLARE n : INTEGER
  DECLARE swapped : BOOLEAN

  n ← items.Length
  swapped ← True

  WHILE n > 0 AND swapped DO
    swapped ← False
    n ← n - 1
    FOR index ← 0 TO n - 1
      IF items[index] > items[index+1]
        THEN
          Swap(items[index], items[index+1])
          swapped ← true
        ENDIF
      NEXT index
    ENDWHILE
    RETURN items
  ENDFUNCTION
```

When the WHILE loop is exited, we know all the data in the *items* array must be sorted. So, the only thing left to do is return the sorted array and exit the *bubbleSort* function.

7.4 - Understand standard methods of solution

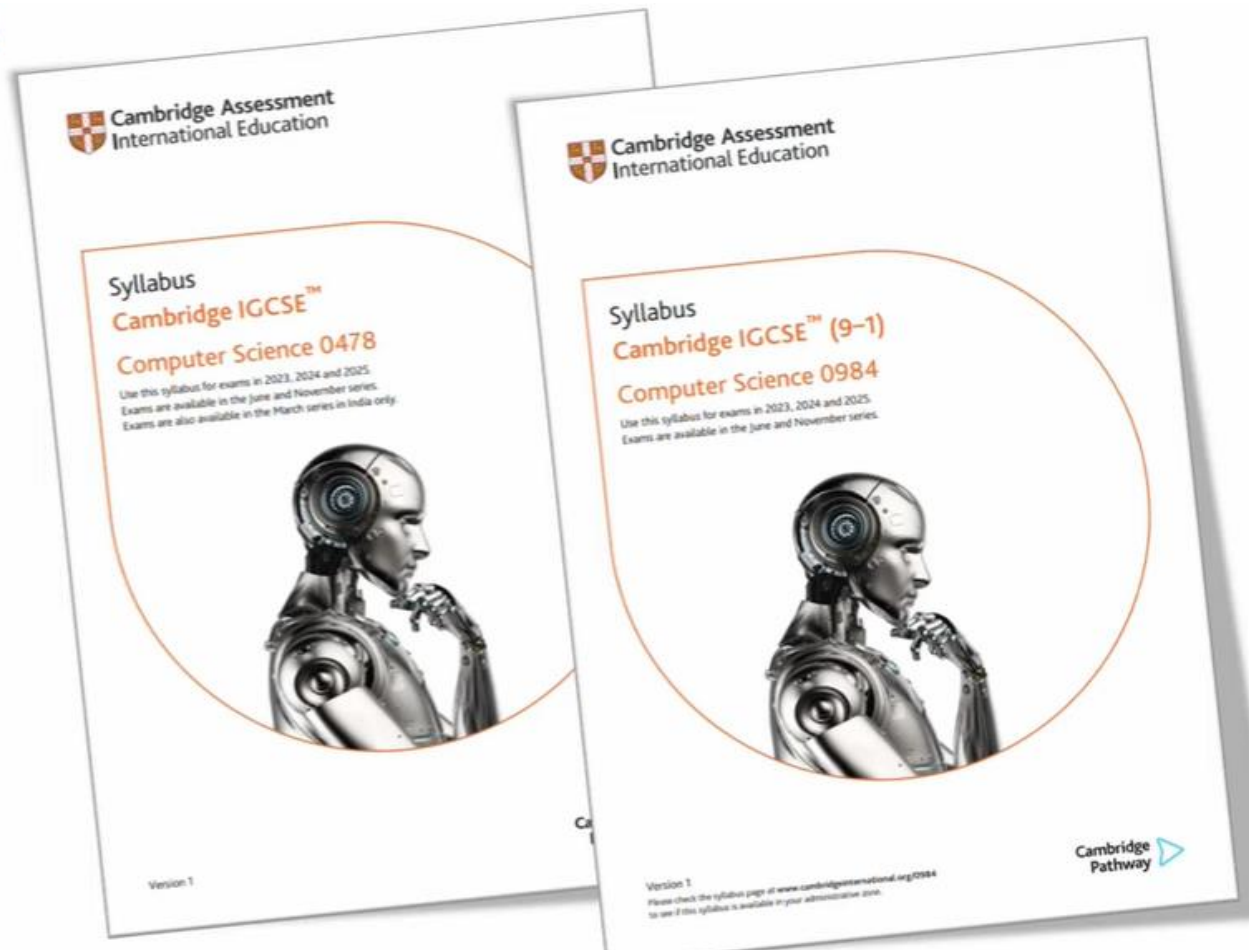
2- Bubble Sort:

Cambridge IGCSE pseudocode

The pseudocode in these videos follows the exact style that will be used in the exams.

The full set of pseudocode is defined in the syllabus:

- Cambridge IGCSE (9-1) Computer Science – 0984
- Cambridge IGCSE (A*-G) Computer Science – 0478



7.4 - Understand standard methods of solution

2. Bubble Sort:

Open Python IDLE and copy the code below for more practical activities

Example 1:

area.py - C:/Users/v90001010/AppData/Local/Programs/Python/Python37-32/area.py

File Edit Format Run Options Window Help

```
import time, random

l = [5, 7, 1, 4, 6, 2, 3, 5, 44]

def bubble_sort(l):
    while True:
        swapped = False
        for index in range(len(l)-1):
            if l[index] > l[index+1]:
                l[index], l[index+1] = l[index+1], l[index]
                swapped = True

        if swapped == False:
            return

bubble_sort(l)

print(l)
```

```
RESTART: C:/Users/v90001010/A
[1, 2, 3, 4, 5, 5, 6, 7, 44]
... |
```

7.4 - Understand standard methods of solution

2-Bubble Sort:

Example 2:

```
main.py x
1 usage
2 def bubble_sort(arr):
3     n = len(arr)
4     for i in range(n):
5         for j in range(0, n - i - 1):
6             if arr[j] > arr[j + 1]:
7                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
8     return arr
9
10 # Example usage
11 arr = [10, 2, 9, 1, 10, 7, 6]
12 sorted_arr = bubble_sort(arr.copy())
13 print("Bubble Sort Result:", sorted_arr)
```

```
C:\Users\n.tadjerout\PycharmProjects\pythonProject\venv>python main.py
Bubble Sort Result: [1, 2, 6, 7, 9, 10, 10]
```

Pseudocode Example 2

FUNCTION BubbleSort(arr)

 n = LENGTH(arr)

FOR i **FROM** 0 **TO** n - 1

FOR j **FROM** 0 **TO** n - i - 2

IF arr[j] > arr[j + 1] **THEN**

SWAP arr[j] **AND** arr[j + 1]

END IF

ENDFOR

ENDFOR

RETURN arr

END FUNCTION

7.4 - Understand **standard** methods of solution

2- Bubble Sort:Example 3:

```
main.py ×  
1 # Given list of names  
2 NAMES = ["Nuriya", "Artur", "Danial", "Inaaya", "Harsh", "Ivan", "Artur", "Ivan", "Beiza", "Nicholas", "Darius", "Samina", "Emir"]  
3  
4 # Bubble sort implementation for sorting names  
5 usage  
6 def bubbleSort(names):  
7     n = len(names)  
8     for i in range(n):  
9         swapped = False  
10        for j in range(0, n-i-1):  
11            if names[j] > names[j+1]:  
12                names[j], names[j+1] = names[j+1], names[j]  
13                swapped = True  
14        # If no two elements were swapped in the inner loop, the list is already sorted  
15        if not swapped:  
16            break  
17  
18 # Sort the NAMES list  
19 bubbleSort(NAMES)  
20  
21 # Output the sorted names  
22 print("Sorted names:")  
23 for name in NAMES:  
24     print(name)
```

```
C:\Users\n.tadjerout\Py  
Sorted names:  
Artur  
Artur  
Beiza  
Danial  
Darius  
Emir  
Harsh  
Inaaya  
Ivan  
Ivan  
Nicholas  
Nuriya  
Samina
```

7.4 - Understand **standard** methods of solution

2- Bubble Sort: Pseudocode for Example 3:

CONSTANT SIZE $\leftarrow$ 13 // Size of the NAMES array

DECLARE NAMES: ARRAY[1:SIZE] OF STRING

NAMES $\leftarrow$ ["Nuriya", "Artur", "Danial", "Inaaya", "Harsh", "Ivan", "Artur", "Ivan", "Beiza", "Nicholas", "Darius", "Samina", "Emir"]

PROCEDURE BubbleSort(NAMES: ARRAY[1:SIZE] OF STRING)  parameters

// Steps or actions to be performed

DECLARE SWAPPED: BOOLEAN

DECLARE TEMP: STRING

DECLARE INDEX: INTEGER

REPEAT

 SWAPPED $\leftarrow$ FALSE

 FOR INDEX $\leftarrow$ 1 TO SIZE - 1

7.4 - Understand **standard** methods of solution

2- Bubble Sort: Pseudocode for Example 3:

```
IF NAMES[INDEX] > NAMES[INDEX + 1]
```

```
    // Swap elements if they are in the wrong order
```

```
    TEMP ← NAMES[INDEX]
```

```
    NAMES[INDEX] ← NAMES[INDEX + 1]
```

```
    NAMES[INDEX + 1] ← TEMP
```

```
    SWAPPED ← TRUE
```

```
END IF
```

```
NEXT INDEX
```

```
UNTIL NOT SWAPPED
```

```
END PROCEDURE
```

7.4 - Understand **standard** methods of solution

2- **Bubble Sort**: Pseudocode for Example 3:

```
// Call the BubbleSort procedure to sort NAMES
```

```
CALL BubbleSort(NAMES)
```

```
// Output the sorted names
```

```
FOR INDEX ← 1 TO SIZE
```

```
    OUTPUT NAMES[INDEX]
```

```
NEXT INDEX
```

In pseudocode PROCEDURE: is a keyword used to define a named sequence of steps or actions that can be executed as a single unit. Think of a procedure as a named block of code that performs a specific task or a set of tasks. It helps in organizing the code into manageable and reusable sections.

```
def PrintMessage(message):  
    print(message)  
  
# Call the procedure with a specific message  
PrintMessage("Hello, IGCSE CS Year 11 Students ")
```

```
PROCEDURE ProcedureName(parameters)  
    // Steps or actions to be performed  
ENDPROCEDURE
```

```
C:\Users\n.tadjerout\PycharmProje  
Hello, IGCSE CS Year 11 Students
```

7.4 - Understand **standard** methods of solution

Other standard methods of a solution: totalling – counting – finding maximum, minimum and average values:

Click the link below to watch the video for other standard methods of a solution:

<https://www.youtube.com/watch?v=rbRgc9y7Pw0&list=PLCiOXwirraUCKOH2duohyLRk9O75bgUdI&index=10>

7 Algorithm design and problem-solving

Other standard methods
of a solution

4:23



7.4 - Understand **standard** methods of solution

3- Totalling

Totalling refers to maintaining a total that values are added to.

There are many situations where you might want to use totalling in your programs – for example, keeping a running total of marks awarded to a student during a test.

```
Total ← 0
```

It is good practice to always initialise variables, so here we start by setting total to zero.

```
FOR Counter ← 1 TO MaxNoOfQuestionsInTest
```

```
    Total ← Total + StudentsTest[Counter]
```

```
NEXT counter
```

This line is performing totalling by adding together the marks held in an array called StudentsTest.

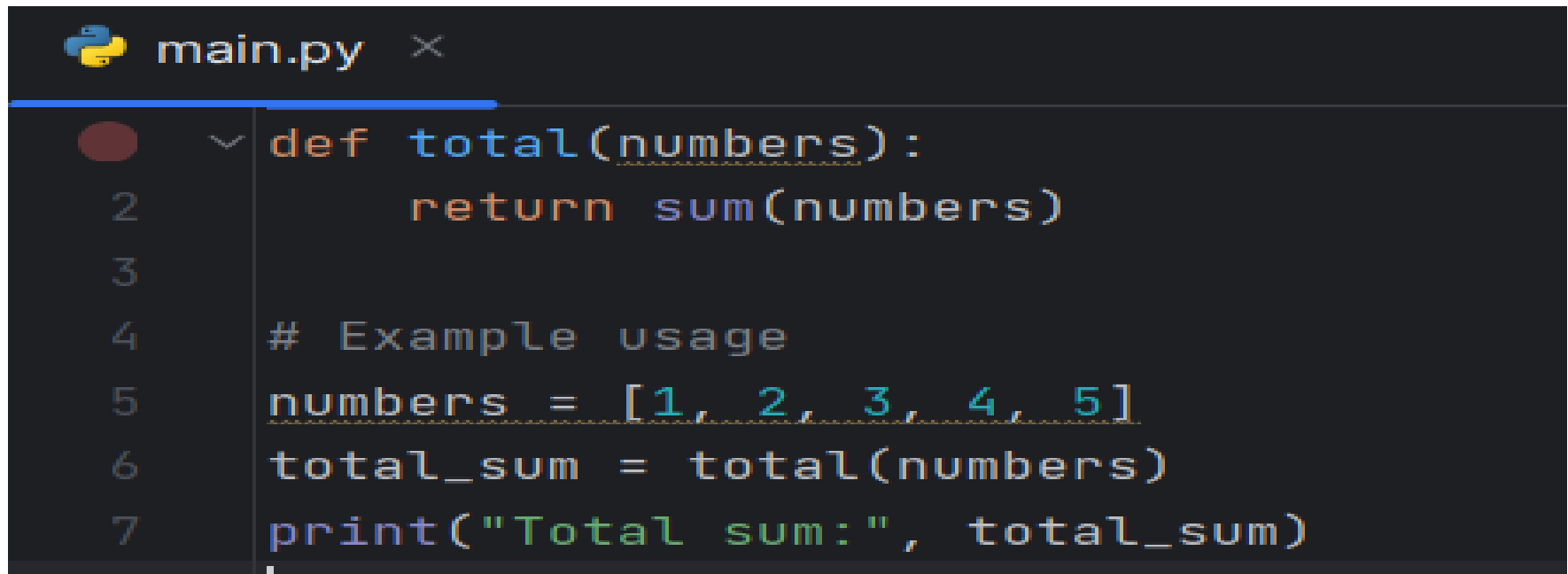


7.4 - Understand **standard** methods of solution

3- Totalling

Totalling involves summing up all the elements in a list.

Example 1:



```
main.py x
1 def total(numbers):
2     return sum(numbers)
3
4 # Example usage
5 numbers = [1, 2, 3, 4, 5]
6 total_sum = total(numbers)
7 print("Total sum:", total_sum)
```

```
C:\Users\n.tadjerout\PycharmProjects
Total sum: 15
```

7.4 - Understand **standard** methods of solution

3- Totalling: Pseudocode for the example 1

// Declare a function named '**total**' that takes an array of integers as input and returns an integer

FUNCTION total(numbers: ARRAY) RETURNS INTEGER

 // Calculate the sum of the elements in the 'numbers' array

 totalSum ← 0

 FOR EACH number IN numbers

 totalSum ← totalSum + number

 ENDFOR

 // Return the total sum

 RETURN totalSum

ENDFUNCTION

// Declare an array of integers

DECLARE numbers: ARRAY

// Initialize the 'numbers' array with values (for example)

numbers ← [1, 2, 3, 4, 5]

// Call the '**total**' function with the 'numbers' array as input

totalSum ← total(numbers)

// Output the total sum

OUTPUT "Total sum:", totalSum

In this **pseudocode**, a function named **total** is defined, which takes an array of integers as input and calculates the sum of its elements. The function iterates through the array, adds each element to the **totalSum** variable, and returns the result. The numbers array is declared and initialized with values. The **total** function is then called with the **numbers** array as input, and the total sum is output to the console.

7.4 - Understand **standard** methods of solution

4- Counting

Keeping count of the number of times a particular action is performed (i.e., the number of times you iterate through a loop) is another very common action performed in programs – for example, counting up the number of students that have been awarded a pass mark in a test.

```
PassCount ← 0
```

It is good practice to always initialise variables, so here we start by setting PassCount to zero.

```
FOR Counter ← 1 TO ClassSize
```

```
    INPUT StudentMark
```

```
    IF StudentMark > 50
```

```
        THEN
```

```
            PassCount ← PassCount + 1
```

```
        ENDIF
```

```
    NEXT Counter
```

7.4 - Understand **standard** methods of solution

4- Counting

Keeping count of the number of times a particular action is performed (i.e., the number of times you iterate through a loop) is another very common action performed in programs – for example, counting up the number of students that have been awarded a pass mark in a test.

```
PassCount ← 0
FOR Counter ← 1 TO ClassSize
    INPUT StudentMark
    IF StudentMark > 50
        THEN
            PassCount ← PassCount + 1
        ENDIF
NEXT Counter
```



If the StudentMark input is greater than 50, we need to update the count.



7.4 - Understand **standard** methods of solution

4- Counting

We can count down as well as up, and there are many reasons why you might want to do this in a program – for example, a retailer keeping track of how much stock they have left so they know when to put in an order.

...

```
StockAvailable ← StockAvailable - 1
```

```
IF StockAvailable < 10
```

```
    THEN
```

```
        CALL OrderNewStock()
```

...

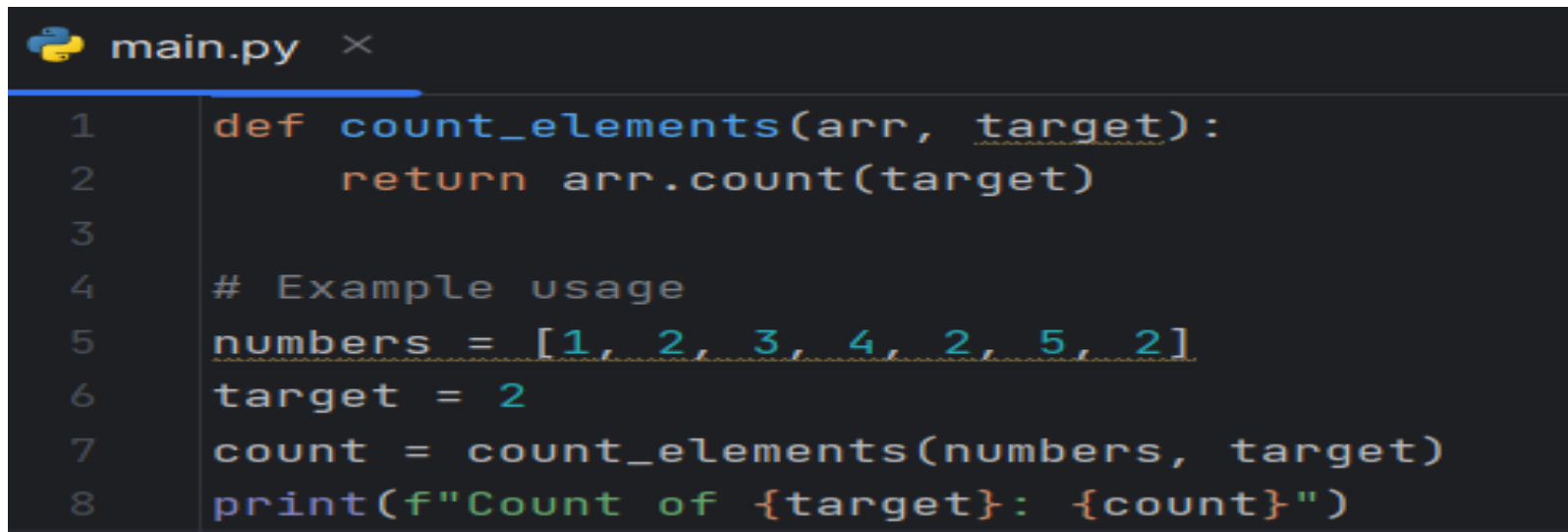
In this line, we are counting down by decreasing the value held in the StockAvailable variables.

7.4 - Understand **standard** methods of solution

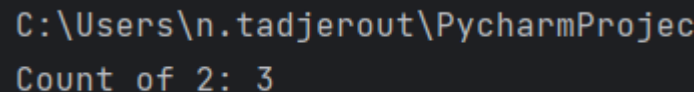
4- Counting:

Counting involves counting the occurrences of a specific element in a list.

Example 1:



```
main.py x
1 def count_elements(arr, target):
2     return arr.count(target)
3
4 # Example usage
5 numbers = [1, 2, 3, 4, 2, 5, 2]
6 target = 2
7 count = count_elements(numbers, target)
8 print(f"Count of {target}: {count}")
```



```
C:\Users\n.tadgerout\PycharmProjec
Count of 2: 3
```

7.4 - Understand **standard** methods of solution

4- Counting: Pseudocode for Example 1:

```
// Declare a function named 'count_elements' that takes an array 'arr' and an integer 'target' as input and returns an integer
FUNCTION count_elements(arr: ARRAY, target: INTEGER) RETURNS INTEGER
    // Initialize a variable 'count' to store the count of occurrences of 'target'
    count ← 0
    // Iterate through the elements of the 'arr' array
    FOR EACH element IN arr
        // Check if the current 'element' is equal to the 'target'
        IF element EQUALS target
            // Increment the 'count' variable if the element matches the target
            count ← count + 1
        ENDIF
    ENDFOR
    // Return the final count
    RETURN count
ENDFUNCTION
```

7.4 - Understand **standard** methods of solution

4- Counting: Pseudocode for Example 1:

```
// Declare an array of integers
DECLARE numbers: ARRAY
// Declare an integer variable 'target'
DECLARE target: INTEGER
// Declare an integer variable 'count' to store the count of occurrences
DECLARE count: INTEGER

// Initialize the 'numbers' array with values (for example)
numbers ← [1, 2, 3, 4, 2, 5, 2]
// Assign a value to the 'target' variable (for example)
target ← 2

// Call the 'count_elements' function with the 'numbers' array and 'target' as inputs
count ← count_elements(numbers, target)

// Output the count of occurrences of the 'target' element
OUTPUT "Count of ", target, ": ", count
```

In this **pseudocode**, a function named **count_elements** is defined, which takes an array **arr** and an integer **target** as input and calculates the count of occurrences of the **target** element in the **arr** array. The function iterates through the array, compares each element with the **target**, increments the **count** variable if there's a match, and returns the final count. The **numbers** array, **target**, and **count** variables are declared and initialized. The **count_elements** function is then called with the **numbers** array and target as inputs, and the **count** of occurrences is output to the console

7.4 - Understand **standard** methods of solution

5- Finding Maximum, Minimum and Average values:

Being able to find the largest and smallest values in a data set is another common technique used in algorithms – for example, finding the highest and lowest marks awarded to students in a class.

```
MaximumMark ← 0
```

```
MinimumMark ← 100
```

```
FOR Counter ← 1 TO ClassSize
```

```
  IF StudentMark[Counter] > MaximumMark
```

```
    THEN
```

```
      MaximumMark ← StudentMark[Counter]
```

```
    ENDIF
```

```
  IF StudentMark[Counter] < MinimumMark
```

```
    THEN
```

```
      MinimumMark ← StudentMark[Counter]
```

```
    ENDIF
```

```
NEXT Counter
```

Initialise the maximum to the lowest possible mark and the minimum to the highest possible mark.

7.4 - Understand **standard** methods of solution

5- Finding Maximum, Minimum and Average values:

Being able to find the largest and smallest values in a data set is another common technique used in algorithms – for example, finding the highest and lowest marks awarded to students in a class.

```
MaximumMark ← 0
```

```
MinimumMark ← 100
```

```
FOR Counter ← 1 TO ClassSize
```

```
  IF StudentMark[Counter] > MaximumMark
```

```
    THEN
```

```
      MaximumMark ← StudentMark[Counter]
```

```
    ENDIF
```

```
  IF StudentMark[Counter] < MinimumMark
```

```
    THEN
```

```
      MinimumMark ← StudentMark[Counter]
```

```
    ENDIF
```

```
NEXT Counter
```

Check the current value in the StudentMark array and see if it is higher than the value currently stored in MaximumMark or lower than the value currently stored in MinimumMark.

7.4 - Understand **standard** methods of solution

5- Finding Maximum, Minimum and Average values:

Following on from the totalling method we looked at earlier, it is also very common to calculate the average (mean) of a set of values – for example, the average number of marks achieved in a test.

```
Total ← 0
```

```
FOR Counter ← 1 TO MaxNoOfQuestionsInTest
```

```
    Total ← Total + StudentsTest[Counter]
```

```
NEXT counter
```

```
Average ← Total / MaxNoOfQuestionsInTest
```

Here we calculate the average from the total after the loop has completed.

7.4 - Understand standard methods of solution

5- Finding Maximum, Minimum and Average values: Finding the maximum, minimum, and average values involves iterating through the list and comparing elements or summing up the elements and dividing by the count to find the average.

Example 1:

```
main.py x
1 usage
2 def find_maximum(arr):
3     return max(arr)
4
5 usage
6 def find_minimum(arr):
7     return min(arr)
8
9 usage
10 def find_average(arr):
11     return sum(arr) / len(arr)
12
13 # Example usage
14 numbers = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]
15 max_value = find_maximum(numbers)
16 min_value = find_minimum(numbers)
17 average_value = find_average(numbers)
18
19 print("Maximum value:", max_value)
20 print("Minimum value:", min_value)
21 print("Average value:", average_value)
```

```
C:\Users\n.tadjerout\PycharmP
Maximum value: 9
Minimum value: 1
Average value: 4.0
```

7.4 - Understand **standard** methods of solution

5- Finding Maximum, Minimum and Average values:

Pseudocode for Example 1:

// Declare a function named '**find_maximum**' that takes an array '**arr**' as input and returns an integer

FUNCTION find_maximum(arr: ARRAY) RETURNS INTEGER

 // Return the maximum value in the array using the **MAX** function

 RETURN MAX(arr)

ENDFUNCTION

// Declare a function named '**find_minimum**' that takes an array '**arr**' as input and returns an integer

FUNCTION find_minimum(arr: ARRAY) RETURNS INTEGER

 // Return the minimum value in the array using the **MIN** function

 RETURN MIN(arr)

ENDFUNCTION

// Declare a function named '**find_average**' that takes an array '**arr**' as input and returns a real number

FUNCTION find_average(arr: ARRAY)

 RETURNS REAL

 // Calculate the sum of all elements in the array

 total $\leftarrow$ SUM(arr)

7.4 - Understand **standard** methods of solution

5- Finding Maximum, Minimum and Average values:

Pseudocode for Example 1:

```
// Calculate the average by dividing the total by the number of elements in the array
average ← total / LENGTH(arr)
// Return the average value
RETURN average
ENDFUNCTION

// Declare an array of integers
DECLARE numbers: ARRAY
// Declare integer variables to store maximum and minimum values
DECLARE max_value: INTEGER
DECLARE min_value: INTEGER
// Declare a real variable to store the average value
DECLARE average_value: REAL

// Initialize the 'numbers' array with values
numbers ← [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]
```

7.4 - Understand **standard** methods of solution

5- Finding Maximum, Minimum and Average values:

Pseudocode for Example 1:

```
// Call the 'find_maximum' function with the 'numbers' array and store the result in 'max_value'
```

```
max_value ← find_maximum(numbers)
```

```
// Call the 'find_minimum' function with the 'numbers' array and store the result in 'min_value'
```

```
min_value ← find_minimum(numbers)
```

```
// Call the 'find_average' function with the 'numbers' array and store the result in 'average_value'
```

```
average_value ← find_average(numbers)
```

```
// Output the maximum, minimum, and average values
```

```
OUTPUT "Maximum value:", max_value
```

```
OUTPUT "Minimum value:", min_value
```

```
OUTPUT "Average value:", average_value
```

7.4 - Understand **standard** methods of solution

5- Finding Maximum, Minimum and Average values:

Pseudocode for Example 1:

```
// Call the 'find_maximum' function with the 'numbers' array and store the result in 'max_value'
max_value ← find_maximum(numbers)
// Call the 'find_minimum' function with the 'numbers' array and store the result in 'min_value'
min_value ← find_minimum(numbers)
// Call the 'find_average' function with the 'numbers' array and store the result in 'average_value'
average_value ← find_average(numbers)
// Output the maximum, minimum, and average values
OUTPUT "Maximum value:", max_value
OUTPUT "Minimum value:", min_value
OUTPUT "Average value:", average_value
```

In this pseudocode, three functions (**find_maximum**, **find_minimum**, and **find_average**) are defined, each taking an array as input and returning the maximum, minimum, and average values, respectively. The **numbers** array and variables (**max_value**, **min_value**, and **average_value**) are declared and initialized. The functions are called with the **numbers** array, and the calculated maximum, minimum, and average values are output to the console.

7.4 - Understand **standard** methods of solution

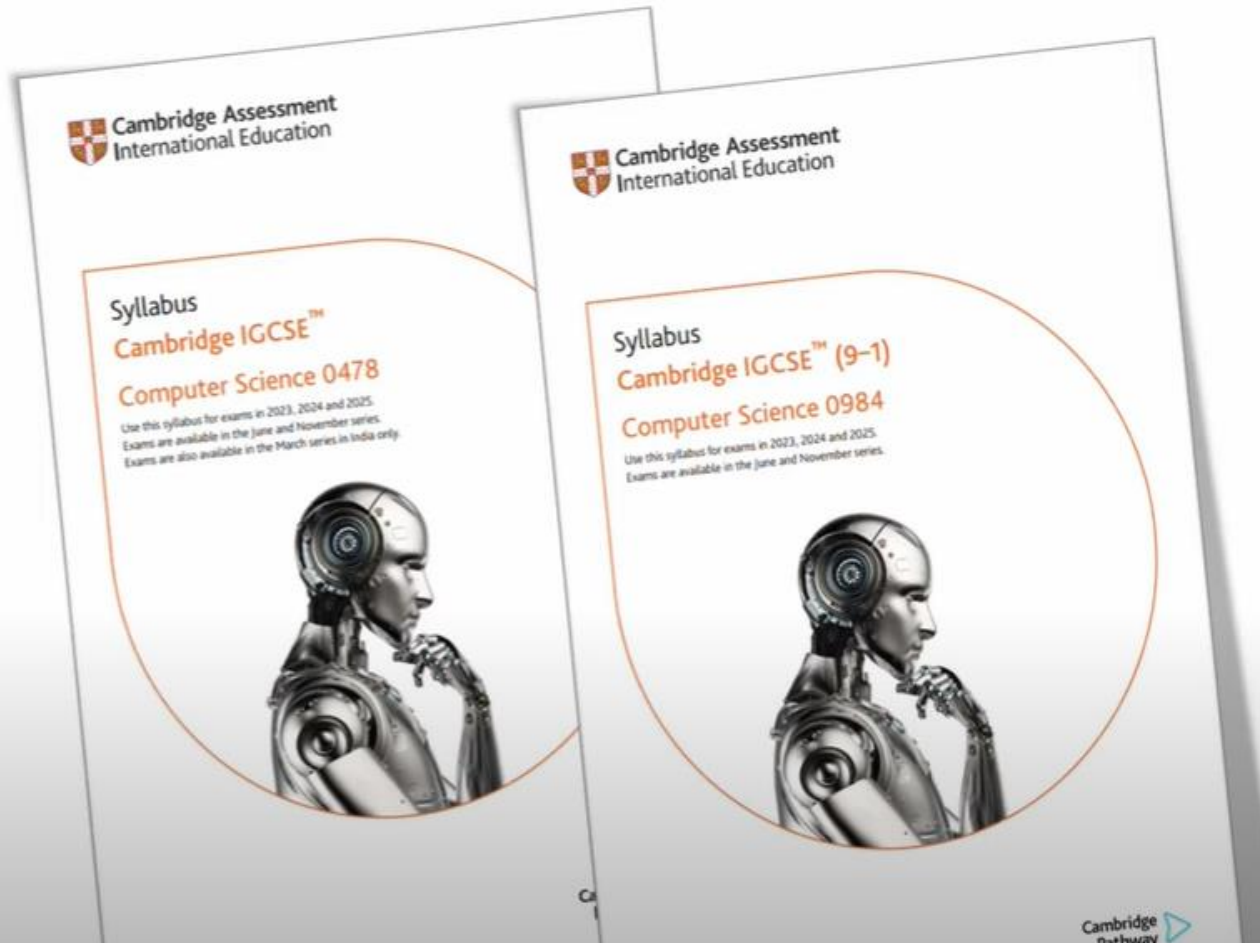
5- Finding maximum, minimum and average values:

Cambridge IGCSE pseudocode

The pseudocode in these videos follows the exact style that will be used in the exams.

The full set of pseudocode is defined in the syllabus:

- Cambridge IGCSE (9-1) Computer Science – 0984
- Cambridge IGCSE (A*-G) Computer Science – 0478



7.5 - (a) Understand the need for **validation checks** to be made on input data and the different types of **validation check**

Validation and verification

Validation and **Verification** are both used to ensure that data entered a computer system is correct and reasonable. The use of both Validation and verification is vital for the smooth running of computer systems.

1- VALIDATION: is performed automatically by the computer system to ensure that only data entered is accepted.

2. VERIFICATION: is used to check that the data does not change as it is being entered

Summary

Validation	Verification
range check length check type check character check format check presence check check digits	double entry (data entry) screen/visual check (data entry) parity check (data transmission) checksum (data transmission)

7.5 - (a) Understand the need for **validation checks** to be made on input data and the different types of **validation check**

Validation and verification

Click the link below to watch the video for Validation vs Verification

https://www.youtube.com/watch?v=AsZkd_6MNHY



Bad Data

- Why validation is important...

Hi, I'm Bawan Thring from Sandton in South Africa.

Please fill in your details below:

Name	Bwl
Surname	3ng
Date of Birth	20/13/1878
City	Sandton Doll
Country	SA

Save

/ 8:38

7.5 - (a) Understand the need for validation checks to be made on input data and the different types of validation check

Click the link below to watch the video for Validation

<https://www.youtube.com/watch?v=nElFAO8Ou34&list=PLCiOXwirraUCKOH2duohyLRk9O75bgUdl&index=11>

7 Algorithm design and problem-solving

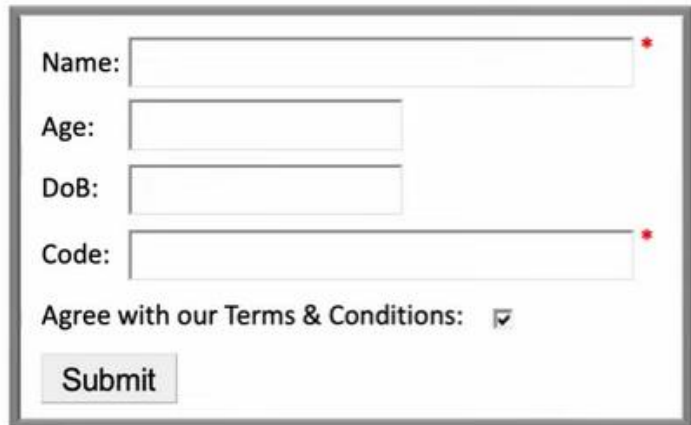
Validation

4:56

7.5 - (a) Understand the need for **validation checks** to be made on input data and the different types of **validation check**

Input validation

Input validation means checking that data input by the user meets specific criteria/rules before processing.



Name: *

Age:

DoB:

Code: *

Agree with our Terms & Conditions: ☒



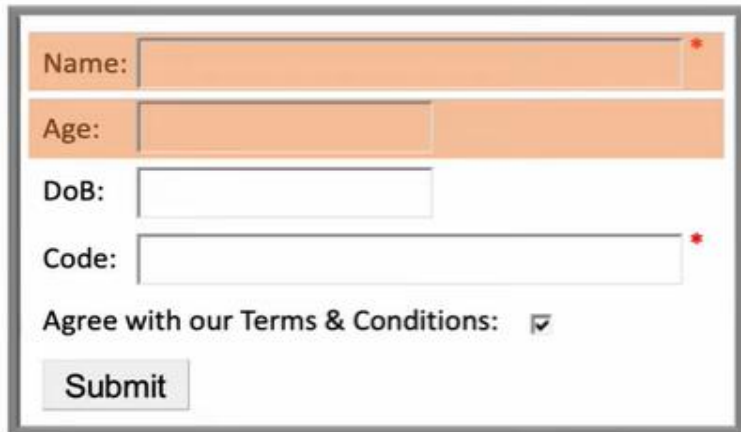
Input
validation
techniques

7.5 - (a) Understand the need for **validation checks** to be made on input data and the different types of **validation check**

Input validation

Is the input of the correct data type? For example:

- Integer
- Real
- String



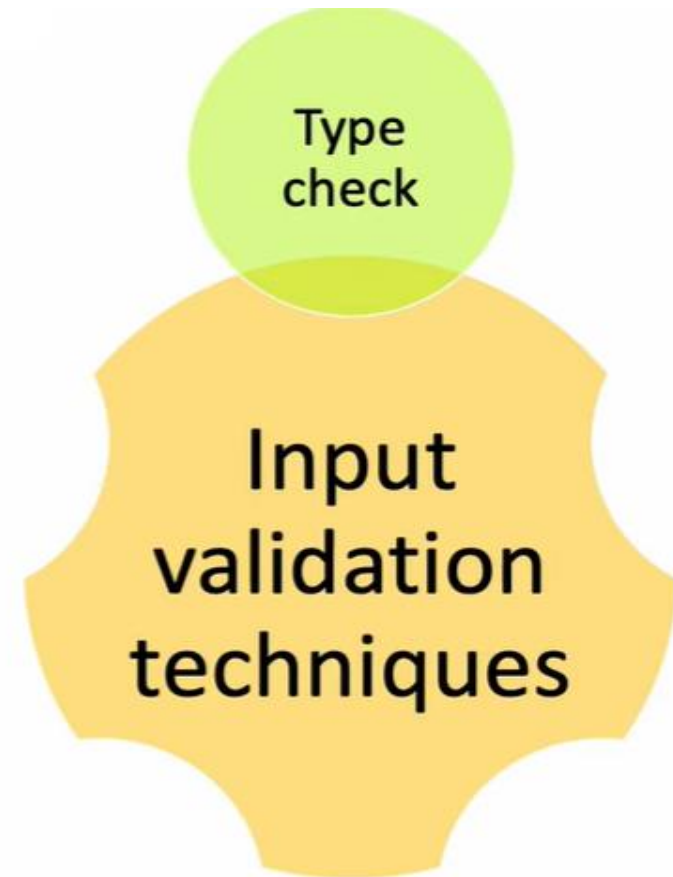
Name: *

Age:

DoB:

Code: *

Agree with our Terms & Conditions: ☒



7.5 - (a) Understand the need for **validation checks** to be made on input data and the different types of **validation check**

Input validation

Is the input of the correct data type? For example:

- Integer
- Real
- String

Name: *

Age:

DoB:

Code: *

Agree with our Terms & Conditions: ☒

Pseudocode example

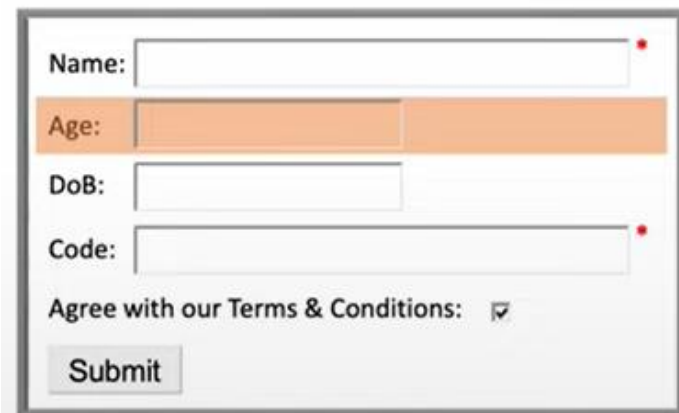
```
OUTPUT "Please enter your age"
REPEAT
    INPUT Age
    IF Age <> DIV(Age, 1)
        THEN
            OUTPUT "Age must be a whole number, try again."
        ENDIF
UNTIL Age = DIV(Age, 1)
```

7.5 - (a) Understand the need for **validation checks** to be made on input data and the different types of **validation check**

Input validation

Is the input within the correct range? For example:

- Between 1 and 10
- Between A and Z



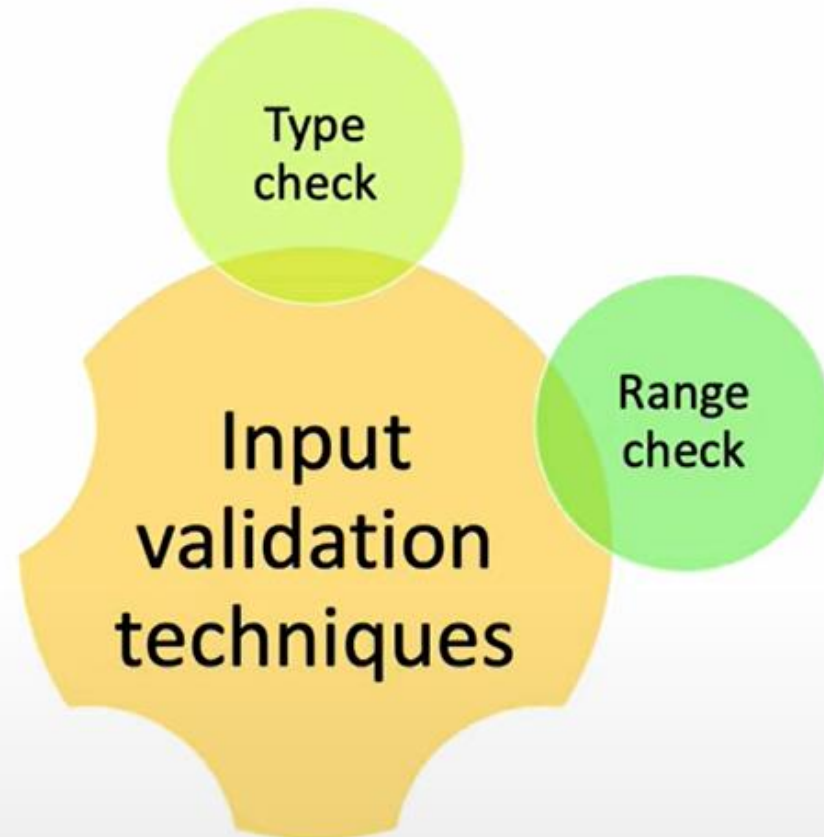
Name:

Age:

DoB:

Code:

Agree with our Terms & Conditions: ☒



7.5 - (a) Understand the need for **validation checks** to be made on input data and the different types of **validation check**

Input validation

Is the input within the correct range? For example:

- Between 1 and 10
- Between A and Z

Name: *

Age:

DoB:

Code: *

Agree with our Terms & Conditions: ☒

Pseudocode example

```
OUTPUT "Please enter your age"
REPEAT
    INPUT Age
    IF (Age < 0) OR (Age > 100)
        THEN
            OUTPUT "Age must be between 0-100, try again."
        ENDIF
UNTIL (Age => 0) AND (Age <= 100)
```

7.5 - (a) Understand the need for **validation checks** to be made on input data and the different types of **validation check**

Input validation

Has all required data been entered? For example:

- Reject blank inputs

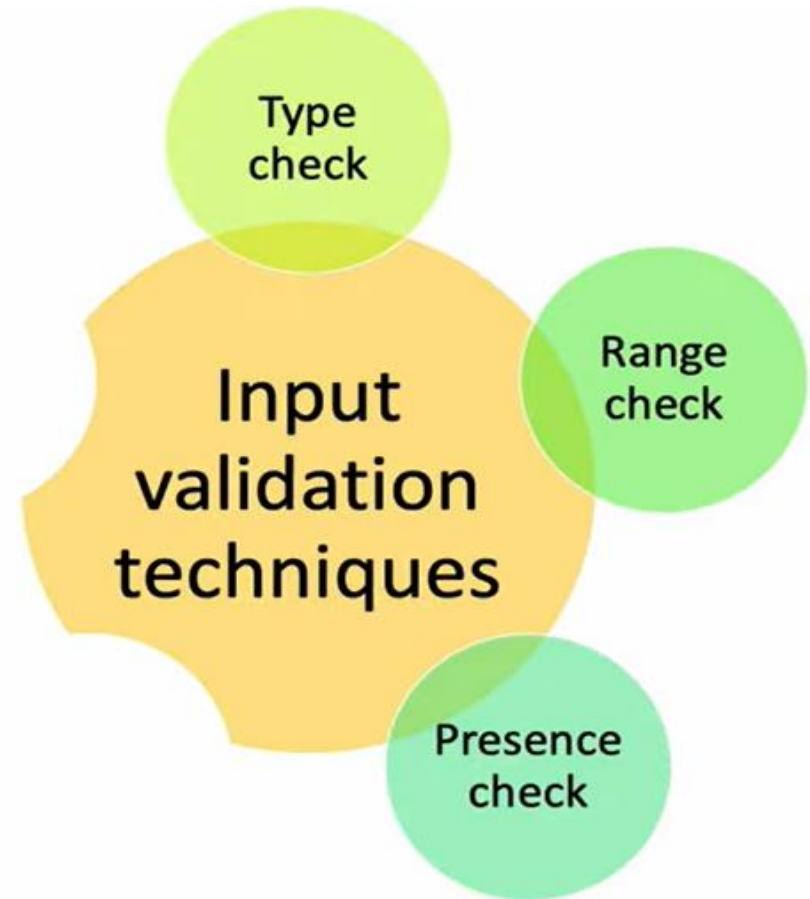
Name: ●

Age:

DoB:

Code: ●

Agree with our Terms & Conditions: ☒

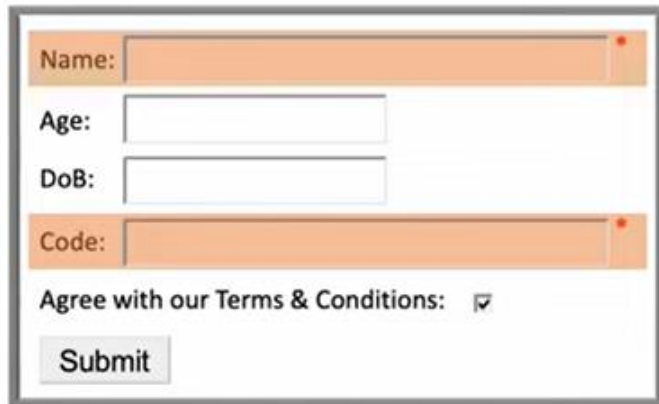


7.5 - (a) Understand the need for **validation checks** to be made on input data and the different types of **validation check**

Input validation

Has all required data been entered? For example:

- Reject blank inputs



Name: *

Age:

DoB:

Code: *

Agree with our Terms & Conditions: ☒

Pseudocode example

```
OUTPUT "Enter your name"
REPEAT
    INPUT Name
    IF Name = ""
        THEN
            OUTPUT "You must enter your name to continue."
        ENDIF
UNTIL Name <> ""
```

7.5 - (a) Understand the need for **validation checks** to be made on input data and the different types of **validation check**

Input validation

Is the input in the correct format? For example:

- Date
- Postcode

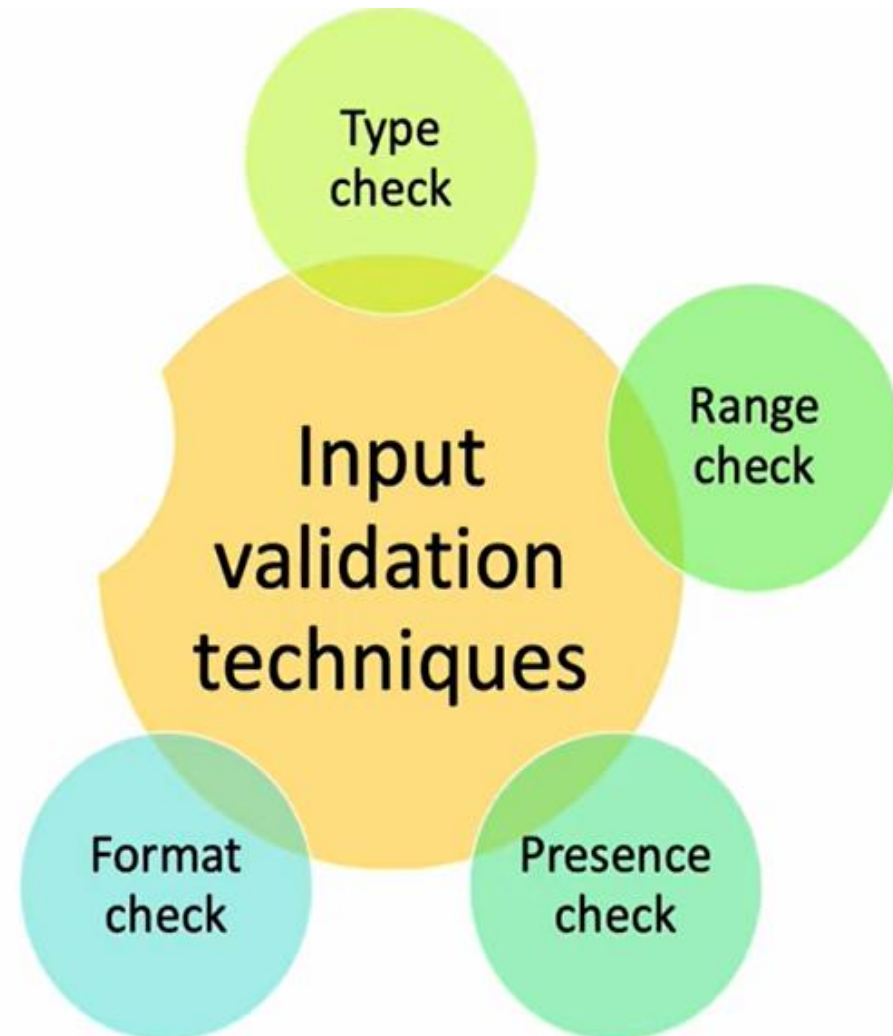
Name:

Age:

DoB:

Code:

Agree with our Terms & Conditions: ☒



Covered in more detail in the following video:

- **8.1 The use of basic string handling**

7.5 - (a) Understand the need for **validation checks** to be made on input data and the different types of **validation check**

Input validation

Does the input include the correct/minimum/maximum number of characters? For example:

- Password
- Car licence plate
- National Insurance number

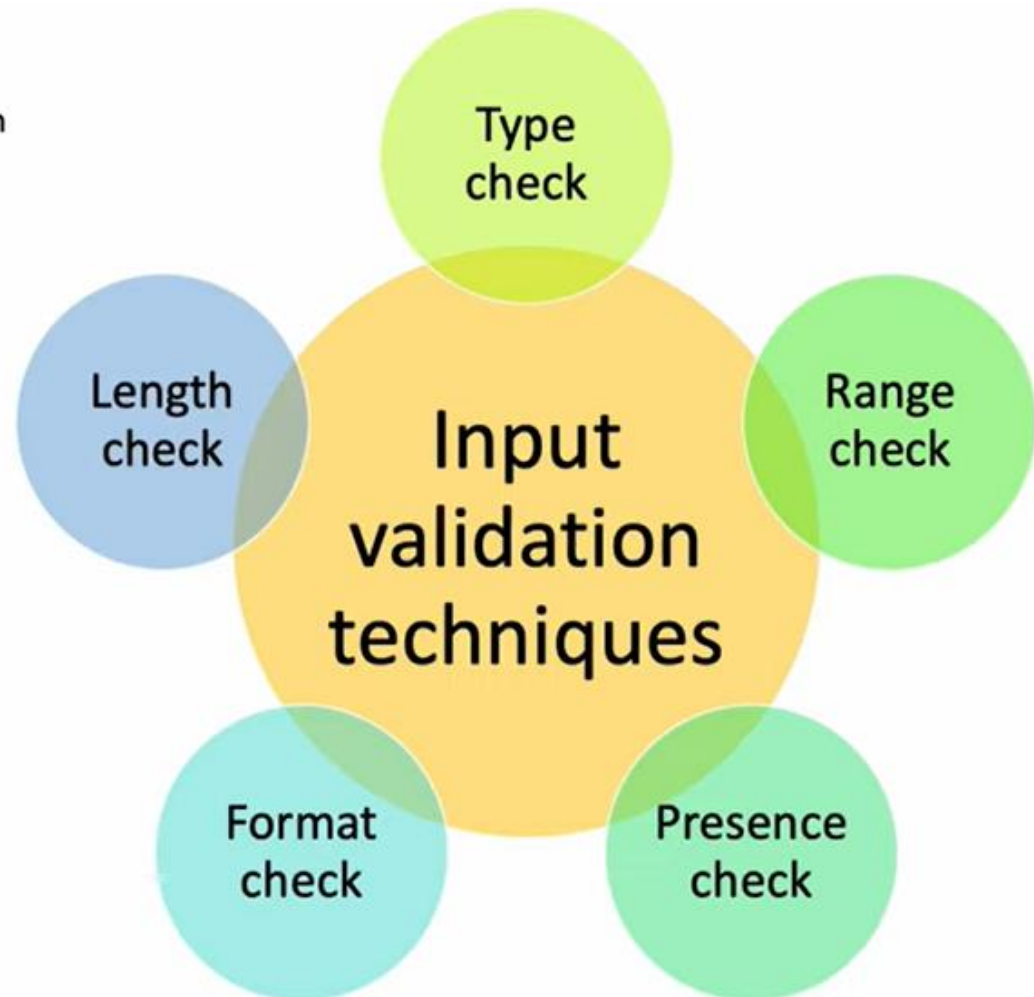
Name:

Age:

DoB:

Code:

Agree with our Terms & Conditions: ☒

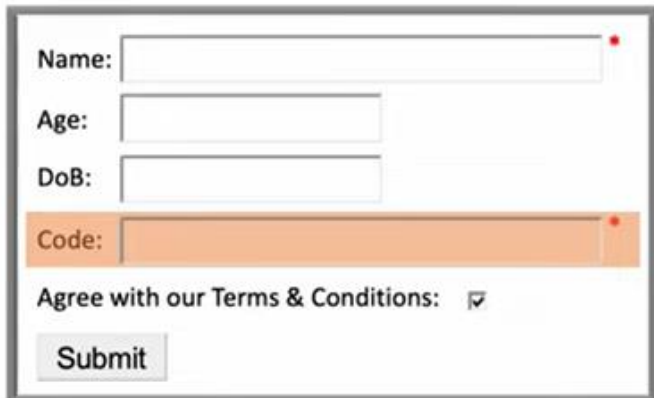


7.5 - (a) Understand the need for **validation checks** to be made on input data and the different types of **validation check**

Input validation

Does the input include the correct/minimum/maximum number of characters? For example:

- Password
- Car licence plate
- National Insurance number



Name:

Age:

DoB:

Code:

Agree with our Terms & Conditions: ☒

Pseudocode example

```
OUTPUT "Enter your code"
REPEAT
    INPUT Code
    IF LENGTH(Code) <> 4
        THEN
            OUTPUT "You must supply a 4 letter code."
        ENDIF
UNTIL LENGTH(Code) = 4
```

7.5 - (a) Understand the need for **validation checks** to be made on input data and the different types of **validation check**

Check digits

A **check digit** is the last digit included in a code or other identification number. It is calculated from all the other digits in the code.

Check digits are commonly used with:

- Barcodes
- ISBN numbers
- Vehicle Identification Numbers (VIN)



Covered in more detail in the following video:

- 2.2 Check digits



7.5 - (a) Understand the need for **validation checks** to be made on input data and the different types of **validation check**



Input validation means checking that data input by the user meets specific criteria/rules before processing:

- Type check: The input is of the correct data type – e.g., integer, real, string.
- Range check: The input is within a predetermined range – e.g., between 1 and 2.
- Presence check: All required data has been entered – e.g., reject blank inputs.
- Format check: The input is in the correct format – e.g., DD/MM/YYYY.
- Length check: The input includes the correct/minimum/maximum number of characters – e.g., password.

By using input validation techniques, a programmer can:

- Make their program more robust and user-friendly
- Prevent further errors occurring later in the algorithm

7.5 - (a) Understand the need for **validation checks** to be made on input data and the different types of **validation check**

Validation checks are essential in computer programming to ensure that input data is accurate, reliable, and within the expected parameters. They are different types of validation checks.

Including:

1– Range Check: A range check ensures that the input data falls within a specified range.

2– Length Check: A length check ensures that the length of the input data is within a specific range.

3– Type Check: A type check ensures that the input data is of the correct data type.

4– Presence Check: A presence check ensures that the input data is not empty.

5– Format Check: A format check ensures that the input data follows a specific format or pattern.

6– Check Digit: A check digit is used to verify the integrity of a code or number.

7.5 - (a) Understand the need for validation checks to be made on input data and the different types of validation check

Simple example of a student registration system that incorporates various types of validation checks:

Student Registration System: In this example, students need to register for a school event using a registration system. The system requires different types of validation checks to ensure the accuracy of the entered information.

1- Range Check: checks that only numbers within a specified range are accepted. For example, percentage marks between 0 and 100 inclusive.

Example - If a program asks for some bodies age, then you might limit entries to be between 13 and 100. This would potentially prevent people under the age of 13 from signing up and it would also prevent people giving an unrealistic age e.g., 250 years old.

Purpose: Ensures the entered age is within an acceptable range (e.g., 10 to 18 years old).

Algorithm: Compare the input value with the minimum and maximum allowed values.

7.5 - (a) Understand the need for **validation checks** to be made on input data and the different types of **validation check**

1- Range Check

FUNCTION ValidateAge(age):

IF age >= 10 **AND** age <= 18 **THEN**
 RETURN TRUE

ELSE
 RETURN FALSE

ENDIF

ENDFUNCTION

```
main.py x
1  def validate_range(value, minimum, maximum):
2      return minimum <= value <= maximum
3
4  # Example Usage
5  age = 20
6  if validate_range(age, minimum: 18, maximum: 25):
7      print("Age is within the valid range.")
8  else:
9      print("Age is outside the valid range.")
```

```
C:\Users\n.tadjerout\PycharmProj
Age is within the valid range.
```

7.5 - (a) Understand the need for **validation checks** to be made on input data and the different types of **validation check**

2– Length Check:

Purpose: Ensures the student's name is between **3** and **50** characters long.

Algorithm: Measure the length of the input and compare it with the minimum and maximum allowed lengths.

FUNCTION ValidateName(name):

IF LENGTH(name) >= 3 **AND** LENGTH(name) <= 50 **THEN**

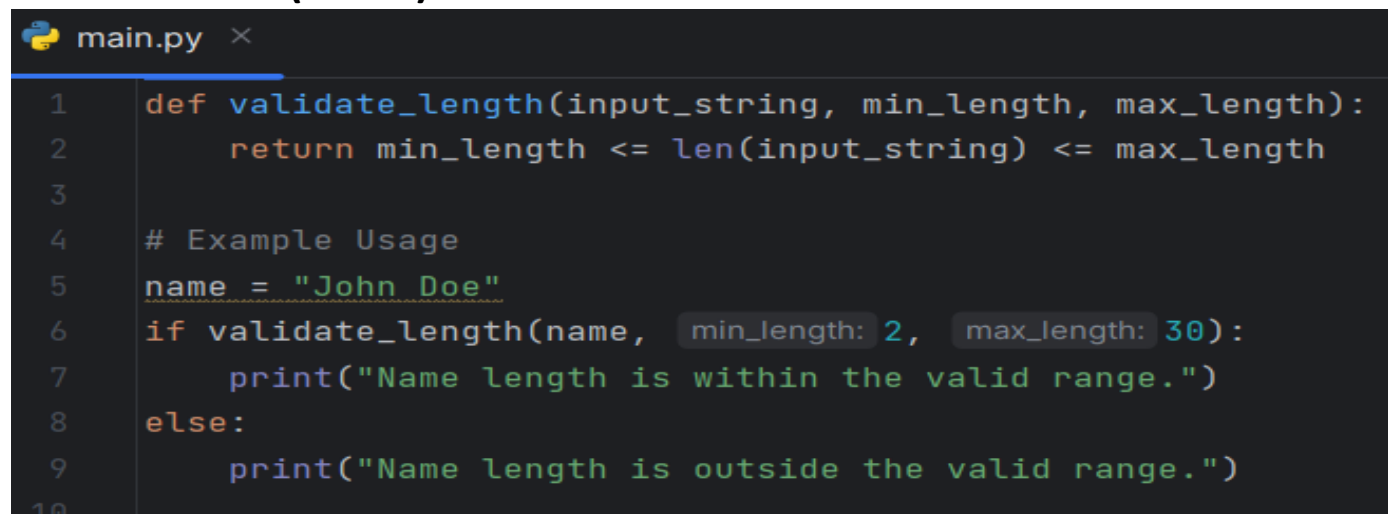
RETURN TRUE

ELSE

RETURN FALSE

ENDIF

ENDFUNCTION



```
main.py x
1 def validate_length(input_string, min_length, max_length):
2     return min_length <= len(input_string) <= max_length
3
4 # Example Usage
5 name = "John Doe"
6 if validate_length(name, min_length: 2, max_length: 30):
7     print("Name length is within the valid range.")
8 else:
9     print("Name length is outside the valid range.")
10
```

```
C:\Users\n.tadjerout\PycharmProjects\pytho
Name length is within the valid range.
```

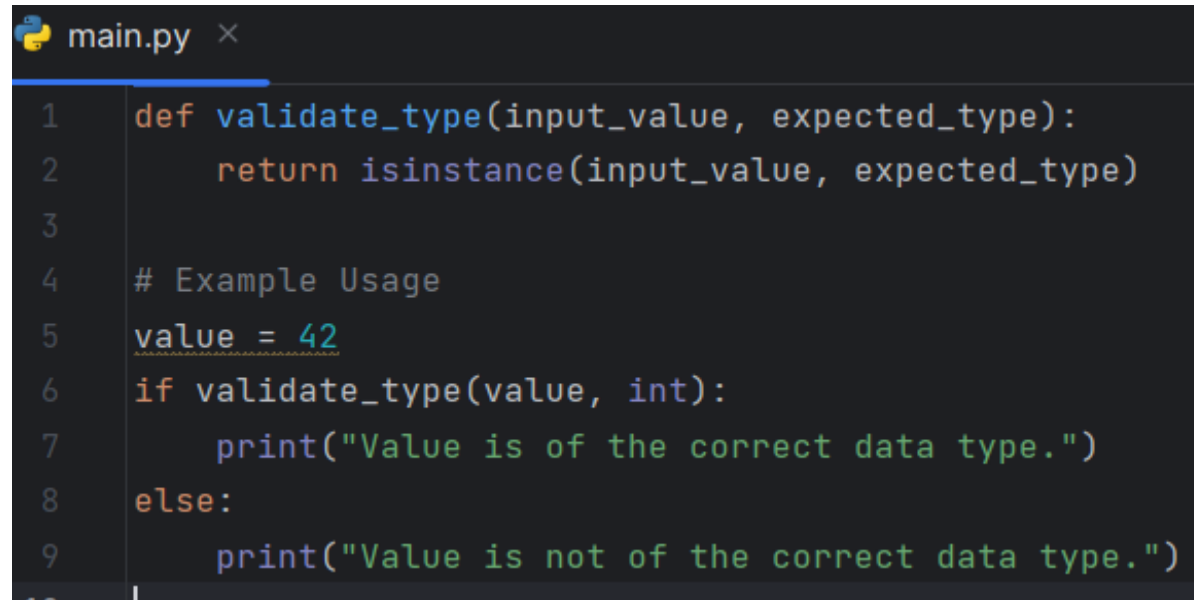
7.5 - (a) Understand the need for **validation checks** to be made on input data and the different types of **validation check**

3– Type Check:

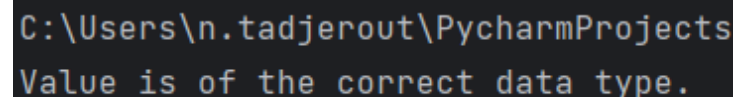
Purpose: Ensures that the input is of the correct data type (e.g., integer, string).

Algorithm: Use language-specific functions to determine the data type of the input.

```
FUNCTION ValidateType(input):  
    IF IS_INTEGER(input) THEN  
        RETURN TRUE  
    ELSE  
        RETURN FALSE  
    ENDIF  
ENDFUNCTION
```



```
main.py x  
1  def validate_type(input_value, expected_type):  
2      return isinstance(input_value, expected_type)  
3  
4  # Example Usage  
5  value = 42  
6  if validate_type(value, int):  
7      print("Value is of the correct data type.")  
8  else:  
9      print("Value is not of the correct data type.")
```



```
C:\Users\n.tadjerout\PycharmProjects  
Value is of the correct data type.
```

7.5 - (a) Understand the need for **validation checks** to be made on input data and the different types of **validation check**

4– Presence Check:

To ensure that some data has been entered and the value has not been left blank, for example an email address must be given for an online transaction and a user cannot sign up to a service if they have left the e-mail field empty.



The screenshot shows a web interface with a header containing 'Register' and 'Login' links. Below the header, there is a section titled 'Customer Information'. Within this section, there is a label 'Email*' in red text, followed by an empty text input field. Above the input field, the text '(*) = Required' is displayed in red, indicating that the email field is mandatory. This visualizes a presence check where the system ensures that the email field is not left empty.

7.5 - (a) Understand the need for **validation checks** to be made on input data and the different types of **validation check**

4– Presence Check:

Purpose: Ensures that the student's address is provided.

Algorithm: Check if the input has a non-zero length or if it's not null/undefined.

FUNCTION ValidateAddress(address):

IF LENGTH(address) > 0 THEN

RETURN TRUE

ELSE

RETURN FALSE

ENDIF

ENDFUNCTION

```
main.py x
1 def validate_presence(input_value):
2     return input_value is not None and len(str(input_value).strip()) > 0
3
4 # Example Usage
5 email = ""
6 if validate_presence(email):
7     print("Email is provided.")
8 else:
9     print("Email is missing or empty.")
```

```
C:\Users\n.tadjerout\PycharmF
Email is missing or empty.
```

7.5 - (a) Understand the need for **validation checks** to be made on input data and the different types of **validation check**

5– Format Check:

A format check is used to ensure that data entered matches a desired pattern. e.g. if a user is required to enter an email address, then it should follow a set pattern of "XXXXX" + @ + "XXXXX" + "." + "XXX" This would allow an email address such as "**example@gmail.com**" to be accepted

for example in Figure below the cub number must be in the form CUB9999 or M or F .

The screenshot shows a Microsoft Access form titled "Cub". The form has the following fields and values:

Field	Value
Cub Number	CUB0006
Name	Roy San
Date of Birth	05-Nov-05
Address	77 Marsh Lane
Gender	r
School	
Telephone Number	
Date Joined	

A validation error message box is displayed over the Gender field. The message reads: "Please Enter M for male or F for female". The box includes an "OK" button, a "Help" button, and a link that says "Was this information helpful?".

7.5 - (a) Understand the need for **validation checks** to be made on input data and the different types of **validation check**

5– Format Check:

Purpose: Ensures the entered email follows a valid format.

Algorithm: Use regular expressions to match the input against the desired pattern.

FUNCTION ValidateEmail(email):

IF MATCHES_PATTERN(email, "[a-zA-Z0-9]+@[a-zA-Z0-9]+\.[a-zA-Z]{2,}\$") THEN

RETURN TRUE

ELSE

RETURN FALSE

ENDIF

ENDFUNCTION

```
main.py x
1 usage
3  def validate_email_format(email):
4      pattern = r'^[a-zA-Z0-9_+-.]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-]+\.$'
5      return re.match(pattern, email) is not None
6
7  # Example Usage
8  email = "example@email.com"
9  if validate_email_format(email):
10     print("Email format is valid.")
11 else:
12     print("Email format is invalid.")
```

7.5 - (a) Understand the need for **validation checks** to be made on input data and the different types of **validation check**

6– Check Digit:

Purpose: Check digits are used to validate the integrity of data, particularly in identification numbers or barcodes.

Algorithm: Apply a specific formula to the input data, generate a check digit, and compare it with the provided check digit.

Check Digit Calculation:

Let's say the ID without the check digit is **12345**. To calculate the check digit, you can sum up the digits, multiply the sum by a factor (e.g., **7**), and then take the modulus **10** of the result.

For the ID 12345:

Sum of digits: $1 + 2 + 3 + 4 + 5 = 15$

Multiply by 7: $15 * 7 = 105$

Check digit: $105 \% 10 = 5$

So, the complete ID with the check digit would be **123455**.

7.5 - (a) Understand the need for **validation checks** to be made on input data and the different types of **validation check**

6– Check Digit:

Check Digit Validation:

To validate the ID, you can follow the same steps. For example, for the ID 123455:

Remove the last digit (check digit): 12345

Sum of digits: $1 + 2 + 3 + 4 + 5 = 15$

Multiply by 7: $15 * 7 = 105$

Calculated check digit: $105 \% 10 = 5$

Since the calculated check digit matches the provided check digit, the ID is considered valid.

This is a simple example of a check digit system where the check digit is calculated based on the sum of the digits. In real-world applications, more complex algorithms are used for generating and validating check digits based on specific requirements and standards.

7.5 - (b) Understand the need for verification checks to be made on input data and the different types of verification check.

Click the link below to watch the video for verification

<https://www.youtube.com/watch?v=PUNd-s2sYc0&list=PLCiOXwirraUCKOH2duohyLRk9O75bgUdI&index=12>

7 Algorithm design and problem-solving

Verification



2:11

7.5 - (b) Understand the **need** for **verification checks** to be made on input data and the different types of **verification check**.

Validation vs verification

Both **validation** and **verification** are methods of ensuring that only reasonable and accurate data input to system are accepted. They cover a variety of techniques that attempt to minimise the number of errors caused by inaccurate data.

Validation

When entering data, **validation** ensures that only reasonable data is accepted.

Verification

Verification ensures that data is not changed, either accidentally or on purpose, as it is entered.

7.5 - (b) Understand the **need** for **verification checks** to be made on input data and the different types of **verification check**.

Verification: Double entry

Double entry verification is a process whereby important data has to be entered twice, sometimes by different people. The system then compares both entries to check they are the same before accepting the data.



Choose your new password

Password:

Confirm password:

```
OUTPUT "Please enter your new password"
INPUT Password
OUPUT "Please confirm password"
INPUT ConfirmPassword
IF Password = ConfirmPassword
    THEN
        OUTPUT "New password accepted"
    ELSE
        OUTPUT "Invalid - Passwords don't match"
ENDIF
```

7.5 - (b) Understand the **need** for **verification checks** to be made on input data and the different types of **verification check**.

Verification: Screen/visual check

A **screen check** or **visual check** is simply a manual check performed by the user who is entering the data.

Once data entry is complete, all the data is displayed back to the user, usually with a tick-box or button asking them to verify the data before submission.



7.5 - (b) Understand the **need** for **verification checks** to be made on input data and the different types of **verification check**.



Verification

Verification ensures that data is not changed, either accidentally or on purpose, as it is being entered.

Double entry verification is a process whereby important data has to be entered twice, sometimes by different people. The system then compares both entries to check they are the same before accepting the data.

A **screen check** or **visual check** is a manual check performed by the user who is entering the data.

The purpose of these verification checks is to enhance the quality and reliability of input data, reducing the likelihood of errors, inaccuracies, and inconsistencies. Visual checks catch obvious mistakes, while double entry checks provide an additional layer of validation by comparing independent entries, ensuring data integrity and reliability in critical applications.

7.6 - Suggest and apply suitable **test data**

Click the link below to watch the video for Suitable test data

<https://www.youtube.com/watch?v=UUSH4K5dBEg&list=PLCiOXwirraUCKOH2duohyLRk9O75bgUdI&index=13>

7 Algorithm design and problem-solving

CAMBRIDGE
IGCSE
(0478/0984)

Suitable test data

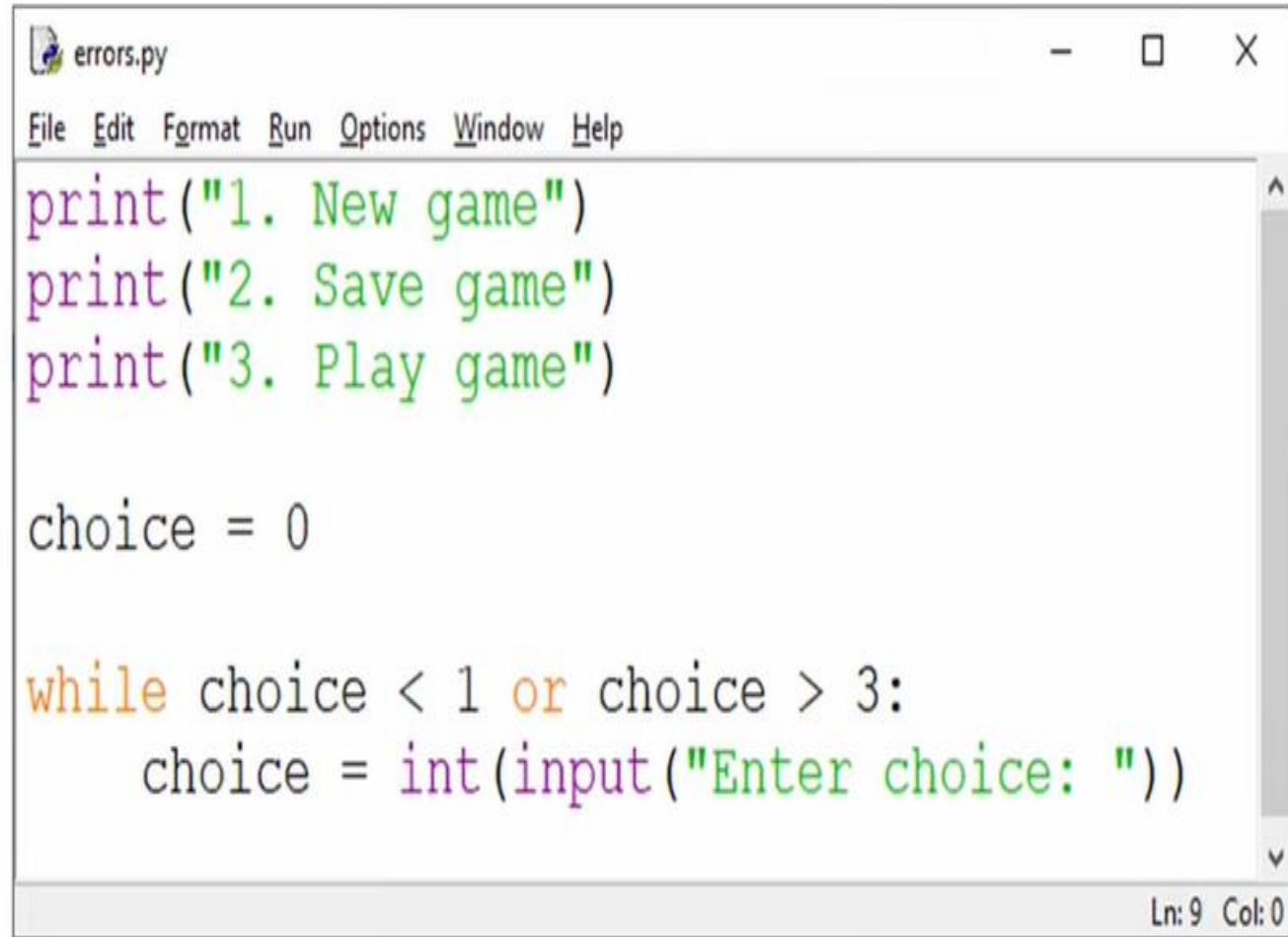


4:11

7.6 - Suggest and apply suitable test data

Suitable test data

Let's look at range of suitable test data for this simple program.



```
errors.py
File Edit Format Run Options Window Help
print("1. New game")
print("2. Save game")
print("3. Play game")

choice = 0

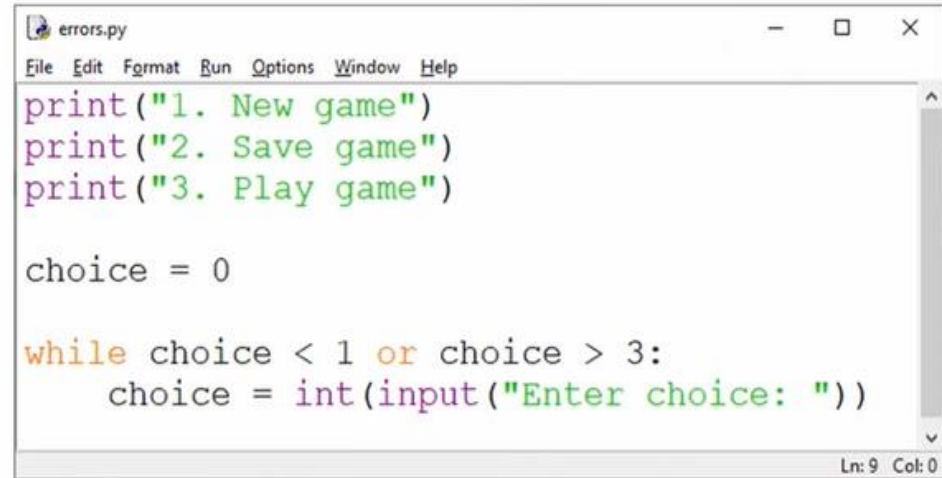
while choice < 1 or choice > 3:
    choice = int(input("Enter choice: "))

Ln: 9 Col: 0
```

7.6 - Suggest and apply suitable test data

Suitable test data

No.	Type of test	Input	Expected output
1	No data		Reject input
2	Normal data	2	Accept input
3	Abnormal data	j	Reject input
4	Abnormal data	#	Reject input
5	Abnormal data	-6	Reject input
6	Extreme data Boundary data	1	Accept input
7	Extreme data Boundary data	3	Accept input
8	Boundary data	0	Reject input
9	Boundary data	4	Reject input



```
errors.py
File Edit Format Run Options Window Help

print("1. New game")
print("2. Save game")
print("3. Play game")

choice = 0

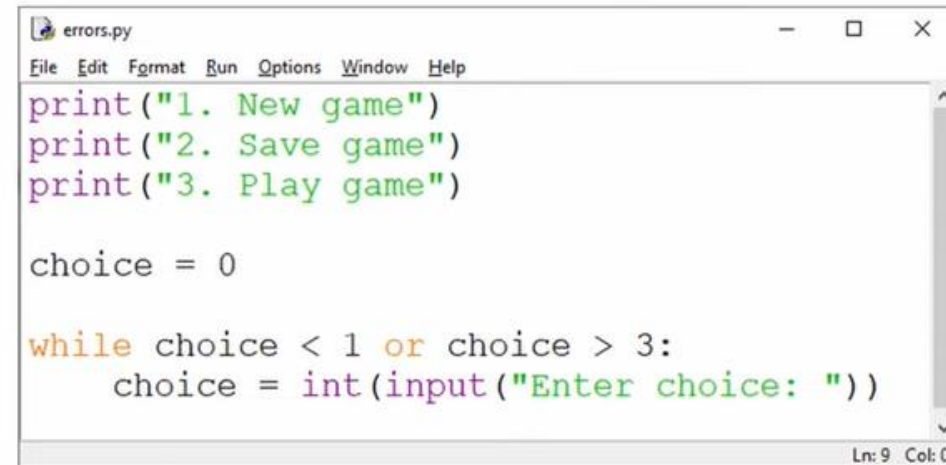
while choice < 1 or choice > 3:
    choice = int(input("Enter choice: "))

Ln: 9 Col: 0
```

7.6 - Suggest and apply suitable test data

Suitable test data

No.	Type of test	Input	Expected output
1	No data		Reject input
2	Normal data	2	Accept input
3	Abnormal data	j	Reject input
4	Abnormal data	#	Reject input
5	Abnormal data	-6	Reject input
6	Extreme data Boundary data	1	Accept input
7	Extreme data Boundary data	3	Accept input
8	Boundary data	0	Reject input
9	Boundary data	4	Reject input



```
errors.py
File Edit Format Run Options Window Help

print("1. New game")
print("2. Save game")
print("3. Play game")

choice = 0

while choice < 1 or choice > 3:
    choice = int(input("Enter choice: "))

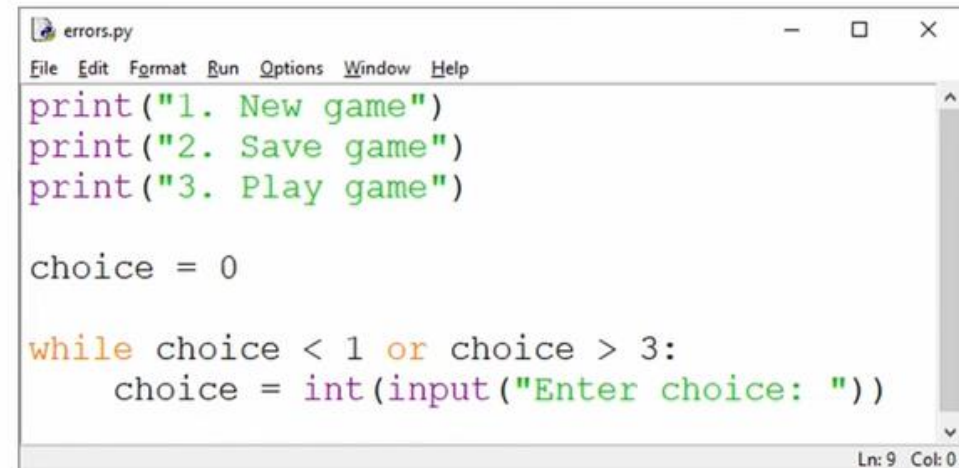
Ln: 9 Col: 0
```

No data

You should always check the special case of no data being entered – this isn't strictly invalid or abnormal data, but it certainly isn't what you want the user to do.

Suitable test data

No.	Type of test	Input	Expected output
1	No data		Reject input
2	Normal data	2	Accept input
3	Abnormal data	j	Reject input
4	Abnormal data	#	Reject input
5	Abnormal data	-6	Reject input
6	Extreme data Boundary data	1	Accept input
7	Extreme data Boundary data	3	Accept input
8	Boundary data	0	Reject input
9	Boundary data	4	Reject input



```
errors.py
File Edit Format Run Options Window Help

print("1. New game")
print("2. Save game")
print("3. Play game")

choice = 0

while choice < 1 or choice > 3:
    choice = int(input("Enter choice: "))

Ln: 9 Col: 0
```

Abnormal data

Data that should be rejected by the program.

7.6 - Suggest and apply suitable test data

Suitable test data

No.	Type of test	Input	Expected output
1	No data		Reject input
2	Normal data	2	Accept input
3	Abnormal data	j	Reject input
4	Abnormal data	#	Reject input
5	Abnormal data	-6	Reject input
6	Extreme data Boundary data	1	Accept input
7	Extreme data Boundary data	3	Accept input
8	Boundary data	0	Reject input
9	Boundary data	4	Reject input



```
errors.py
File Edit Format Run Options Window Help
print("1. New game")
print("2. Save game")
print("3. Play game")

choice = 0

while choice < 1 or choice > 3:
    choice = int(input("Enter choice: "))
```

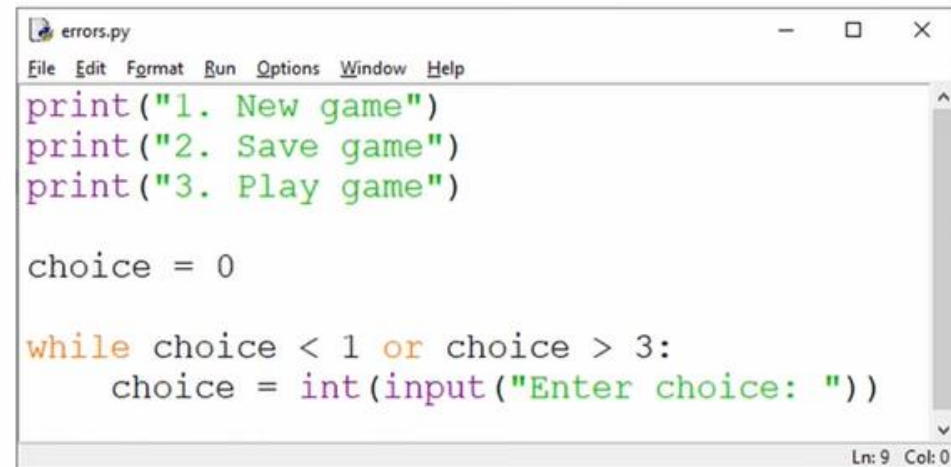
Extreme data

Data that is the largest/smallest acceptable value.

7.6 - Suggest and apply suitable test data

Suitable test data

No.	Type of test	Input	Expected output
1	No data		Reject input
2	Normal data	2	Accept input
3	Abnormal data	j	Reject input
4	Abnormal data	#	Reject input
5	Abnormal data	-6	Reject input
6	Extreme data Boundary data	1	Accept input
7	Extreme data Boundary data	3	Accept input
8	Boundary data	0	Reject input
9	Boundary data	4	Reject input



```
errors.py
File Edit Format Run Options Window Help

print("1. New game")
print("2. Save game")
print("3. Play game")

choice = 0

while choice < 1 or choice > 3:
    choice = int(input("Enter choice: "))

Ln: 9 Col: 0
```

Boundary data

Data that is the largest/smallest acceptable value as well as the corresponding smallest/largest rejected value.

7.6 - Suggest and apply suitable test data

Suitable test data

What data could be used for this car park ticket machine software?



7.6 - Suggest and apply suitable test data

Suitable test data

What data could be used for this car park ticket machine software?

- ½ hour parking
- 1 hour parking
- 1 ½ hour parking
- Sunday parking
- Evening parking past 7pm
- Parking before 9:30am
- Parking after 9:30am
- Parking during invalid opening hours



7.6 - Suggest and apply suitable **test data**



Suitable test data

Test data needs to include a range of:

- **Normal** inputs: Data that should be accepted without causing errors.
- **Abnormal** inputs: Data that should be rejected by the program – includes no input when one is expected.
- **Boundary** inputs: Data that is the largest/smallest acceptable value as well as the corresponding smallest/largest rejected value.
- **Extreme data**: Data that is the largest/smallest acceptable value.

7.7 - Complete a **trace table** to document a dry-run of an algorithm

Click the link below to watch the video for Trace tables

https://www.youtube.com/watch?v=7_kyY88Y9Y&list=PLCiOXwirraUCKOH2duohyLRk9O75bgUdI&index=14

7 Algorithm design and problem-solving

Trace tables

CAMBRIDGE
IGCSE
(0478/0984)



6:18

7.7 - Complete a **trace table** to document a dry-run of an algorithm



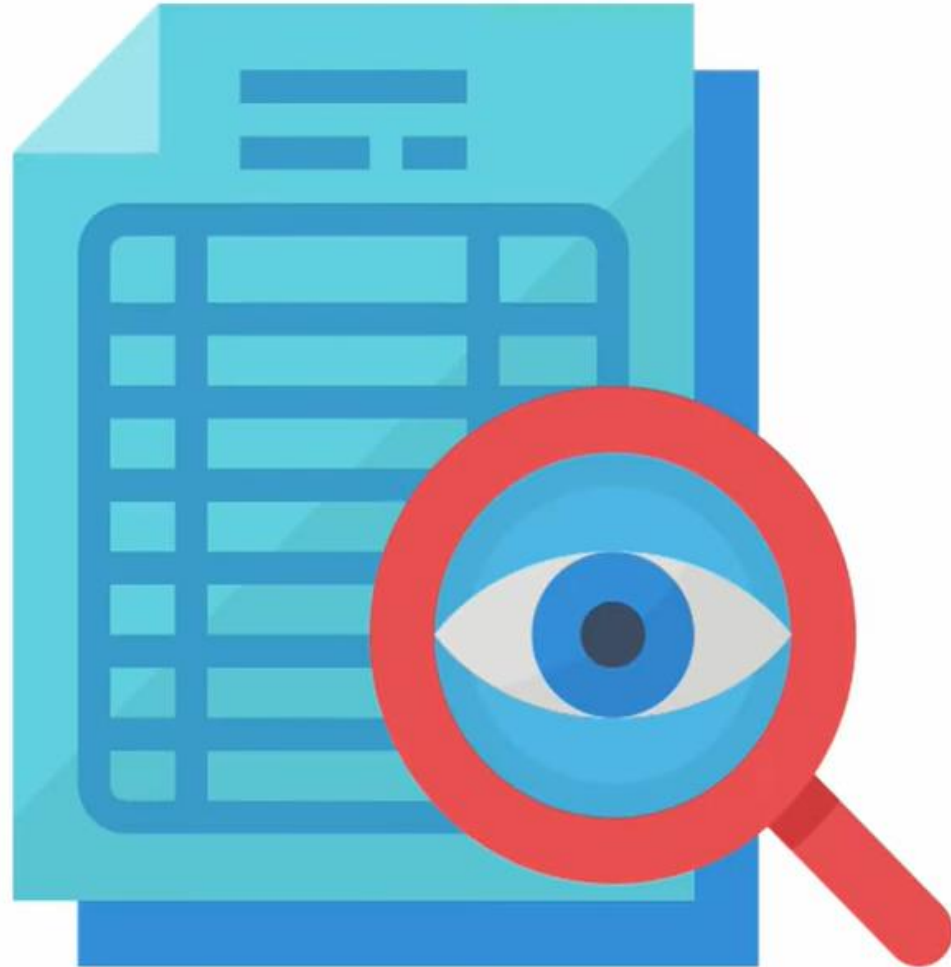
Trace tables

Tracing execution: A vital skill for understanding program flow and testing the accuracy of an algorithm for logic.

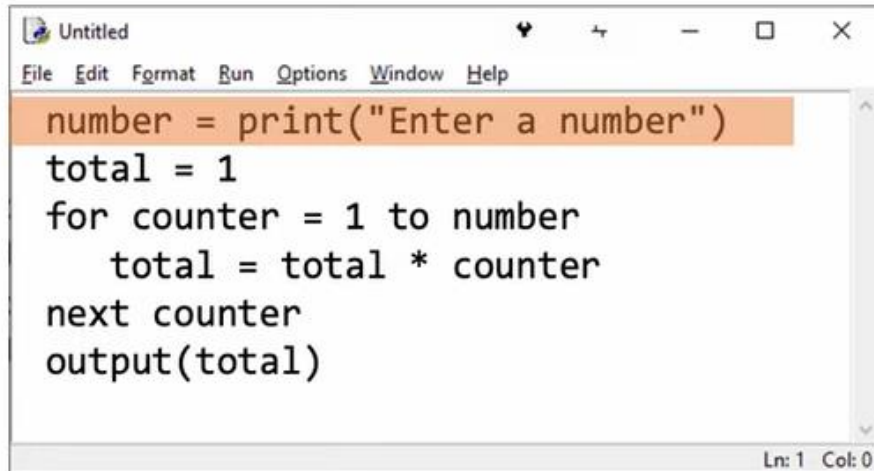
It involves examining a printed extract of program code and running thorough the program:

- Take each line at a time and write the current state of each variable in a trace table.
- Note down any output the program produces.
- Each variable present in the program should have its own column in the trace table.
- A new row should be added under any column if the state of a variable changes.

Trace tables are an excellent way to track down logic errors in a problem.



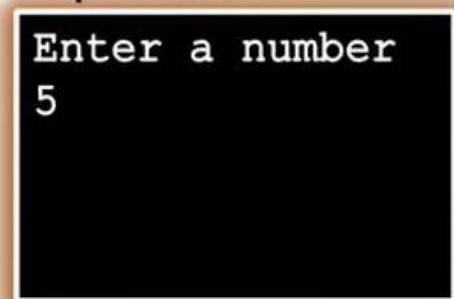
7.7 - Complete a **trace table** to document a dry-run of an algorithm



```
number = print("Enter a number")
total = 1
for counter = 1 to number
    total = total * counter
next counter
output(total)
```

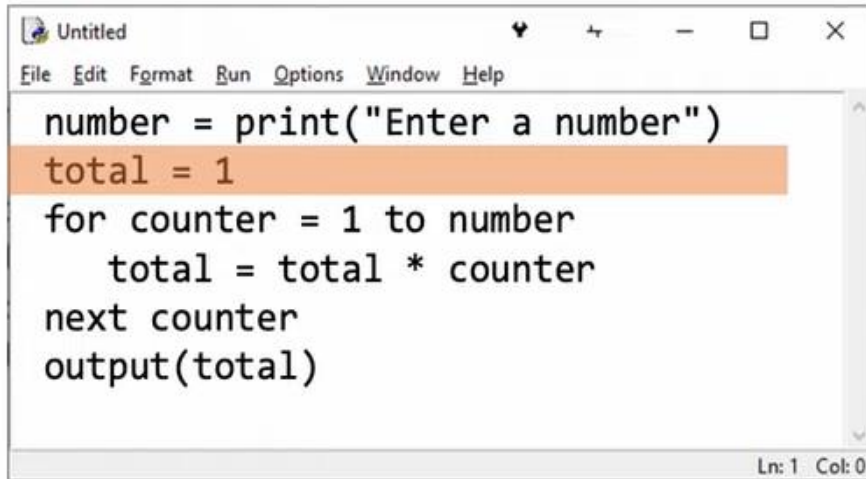
number	counter	total
5		

Output window:



```
Enter a number
5
```

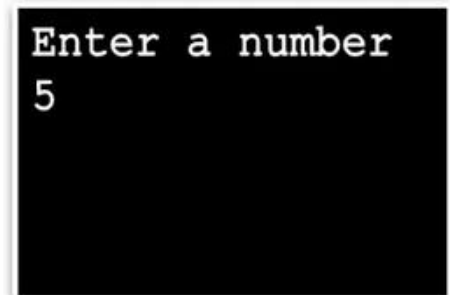
7.7 - Complete a **trace table** to document a dry-run of an algorithm



```
number = print("Enter a number")
total = 1
for counter = 1 to number
    total = total * counter
next counter
output(total)
```

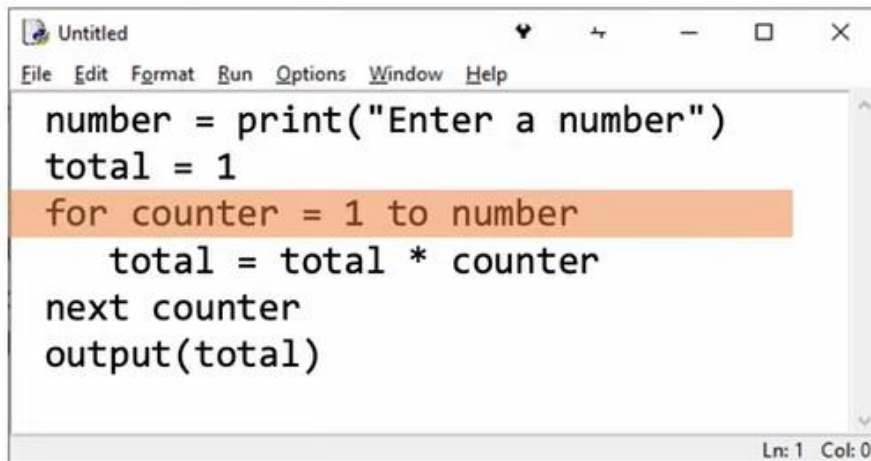
number	counter	total
5		1

Output window:



```
Enter a number
5
```

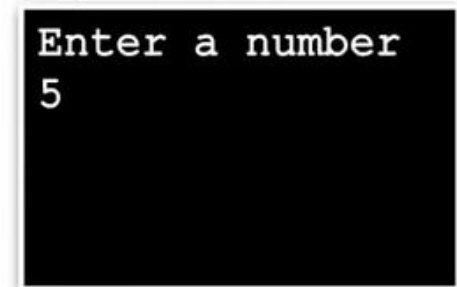
7.7 - Complete a **trace table** to document a dry-run of an algorithm



```
number = print("Enter a number")
total = 1
for counter = 1 to number
    total = total * counter
next counter
output(total)
```

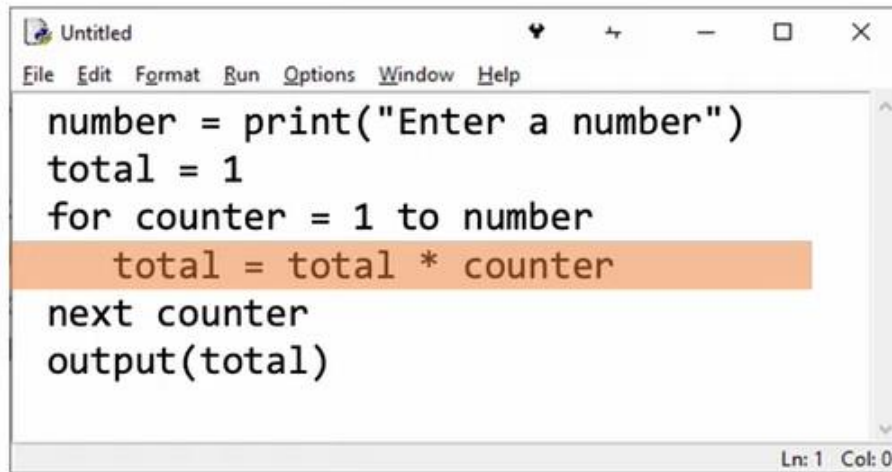
number	counter	total
5	1	1

Output window:



```
Enter a number
5
```

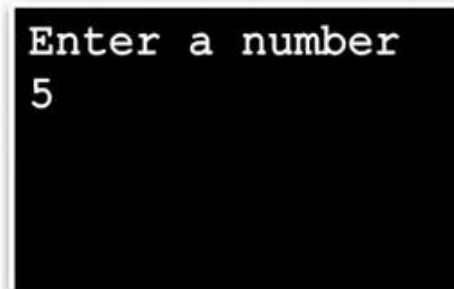
7.7 - Complete a trace table to document a dry-run of an algorithm



```
number = print("Enter a number")
total = 1
for counter = 1 to number
    total = total * counter
next counter
output(total)
```

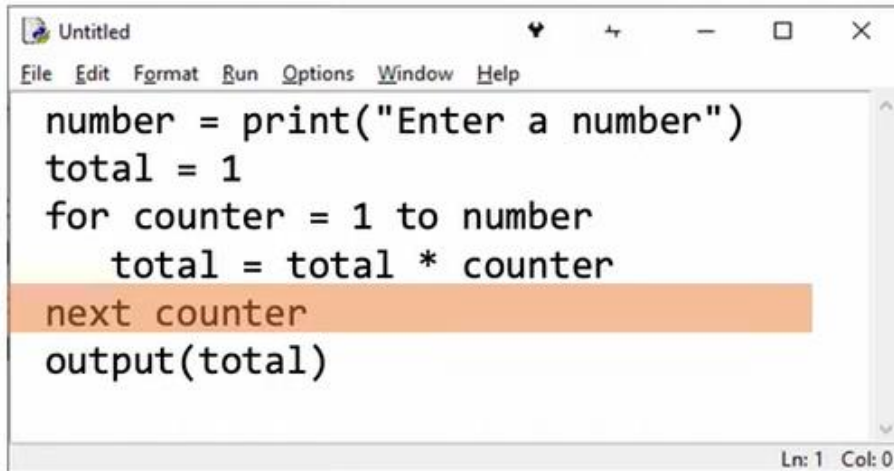
number	counter	total
5	1	1
		1

Output window:



```
Enter a number
5
```

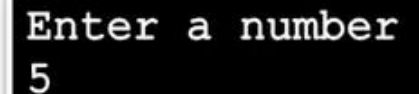
7.7 - Complete a trace table to document a dry-run of an algorithm



```
number = print("Enter a number")
total = 1
for counter = 1 to number
    total = total * counter
next counter
output(total)
```

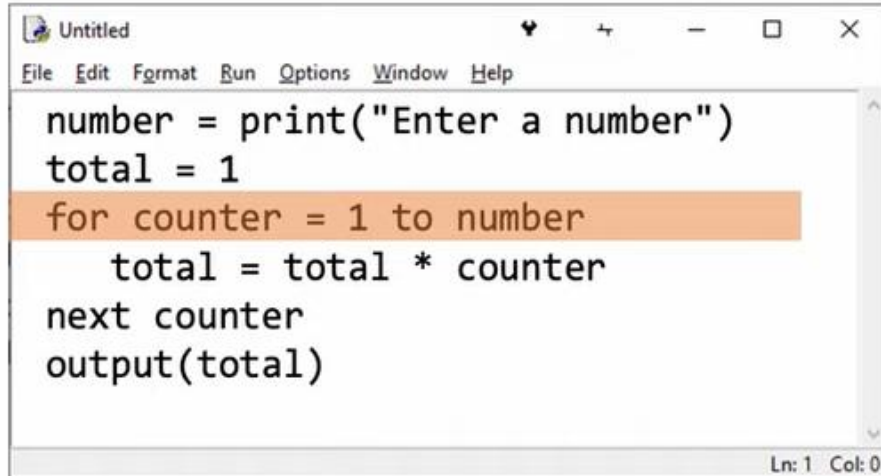
number	counter	total
5	1	1
	2	1

Output window:



```
Enter a number
5
```

7.7 - Complete a **trace table** to document a dry-run of an algorithm



```
number = print("Enter a number")
total = 1
for counter = 1 to number
    total = total * counter
next counter
output(total)
```

Ln: 1 Col: 0

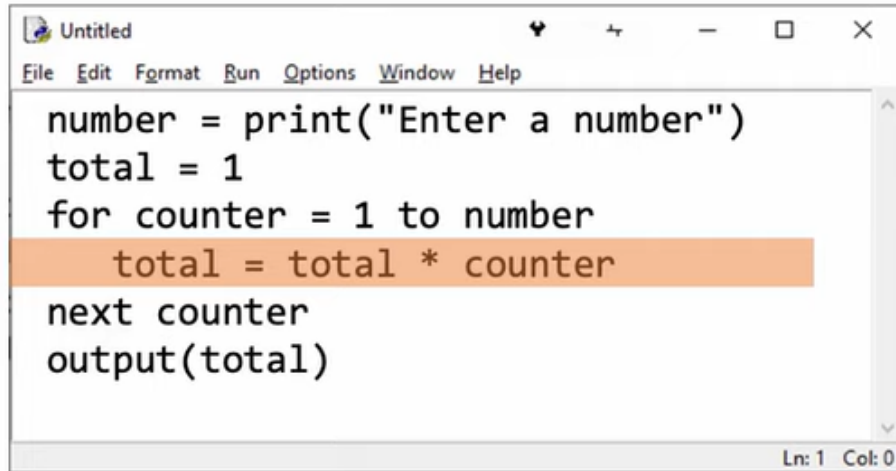
number	counter	total
5	1	1
	2	1

Output window:



```
Enter a number
5
```

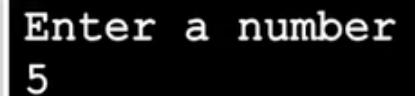
7.7 - Complete a **trace table** to document a dry-run of an algorithm



```
number = print("Enter a number")
total = 1
for counter = 1 to number
    total = total * counter
next counter
output(total)
```

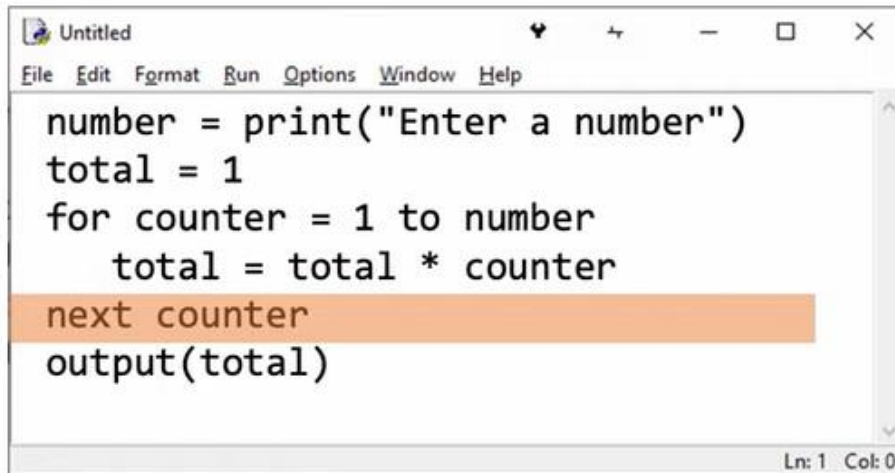
number	counter	total
5	1	1
	2	1
		2

Output window:



```
Enter a number
5
```

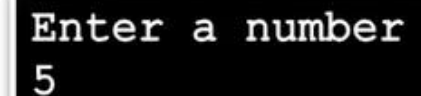
7.7 - Complete a **trace table** to document a dry-run of an algorithm



```
number = print("Enter a number")
total = 1
for counter = 1 to number
    total = total * counter
next counter
output(total)
```

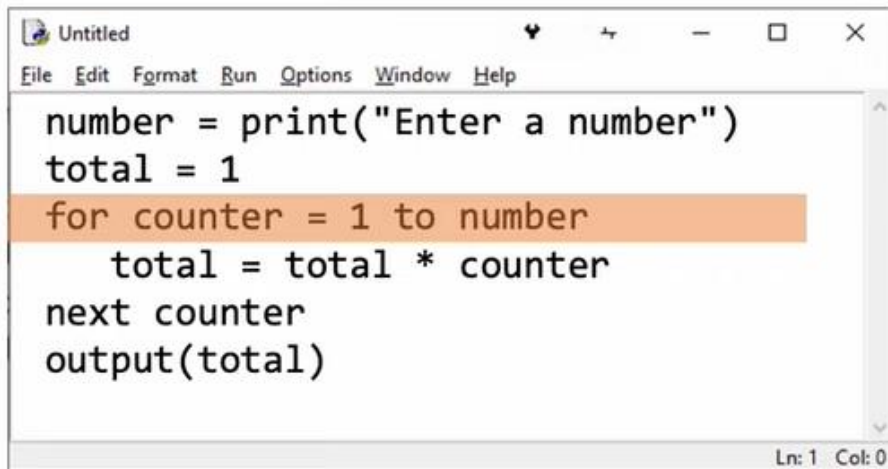
number	counter	total
5	1	1
	2	1
	3	2

Output window:



```
Enter a number
5
```

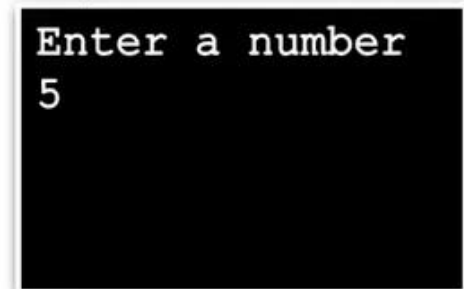
7.7 - Complete a trace table to document a dry-run of an algorithm



```
number = print("Enter a number")
total = 1
for counter = 1 to number
    total = total * counter
next counter
output(total)
```

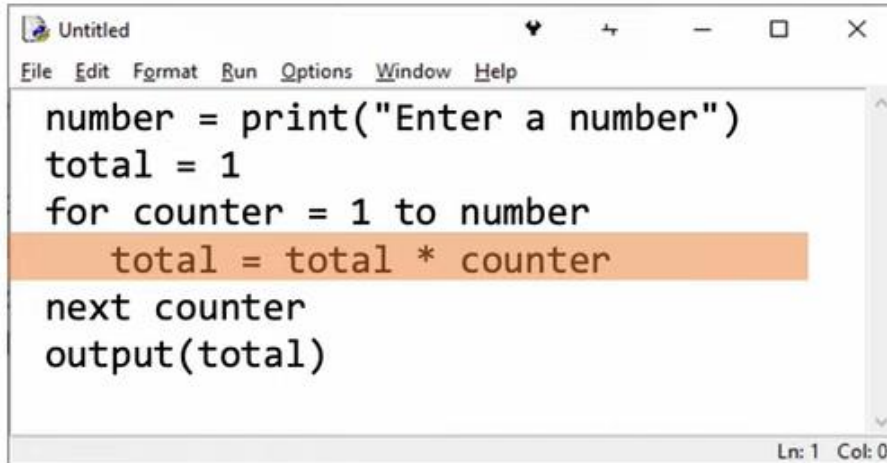
number	counter	total
5	1	1
	2	1
	3	2

Output window:



```
Enter a number
5
```

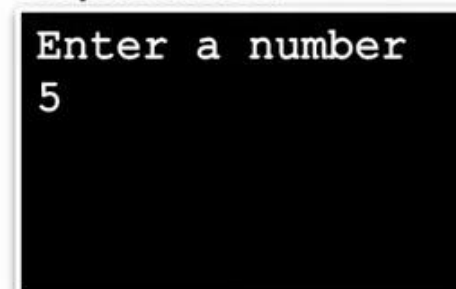
7.7 - Complete a **trace table** to document a dry-run of an algorithm



```
number = print("Enter a number")
total = 1
for counter = 1 to number
    total = total * counter
next counter
output(total)
```

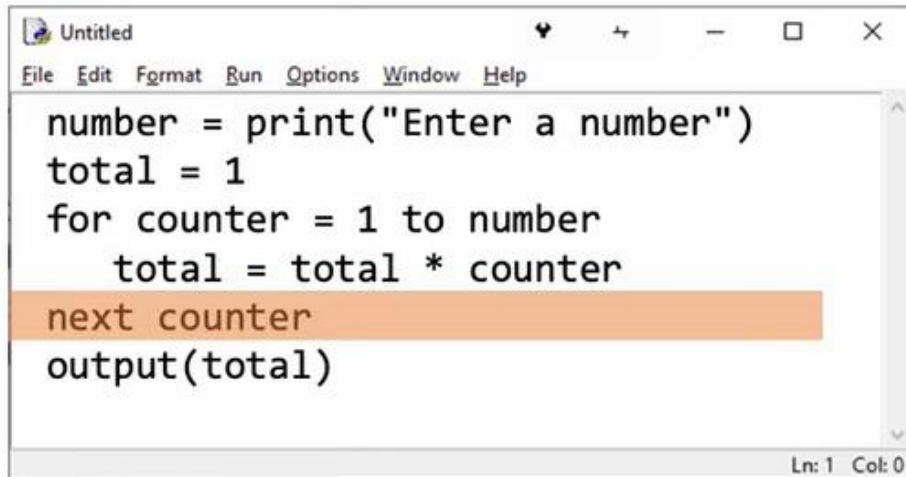
number	counter	total
5	1	1
	2	1
	3	2
		6

Output window:



```
Enter a number
5
```

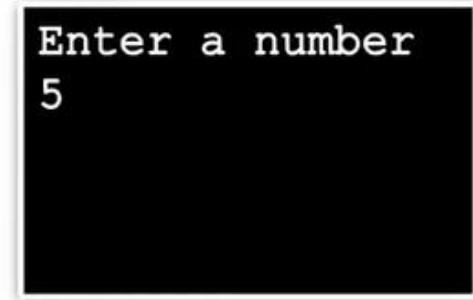
7.7 - Complete a trace table to document a dry-run of an algorithm



```
number = print("Enter a number")
total = 1
for counter = 1 to number
    total = total * counter
next counter
output(total)
```

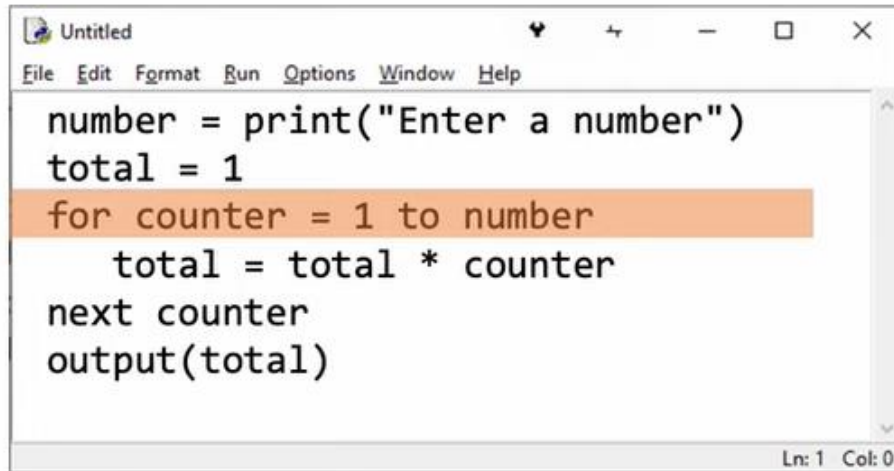
number	counter	total
5	1	1
	2	1
	3	2
	4	6

Output window:



```
Enter a number
5
```

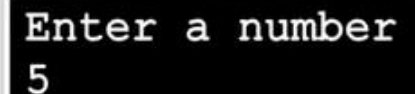
7.7 - Complete a trace table to document a dry-run of an algorithm



```
number = print("Enter a number")
total = 1
for counter = 1 to number
    total = total * counter
next counter
output(total)
```

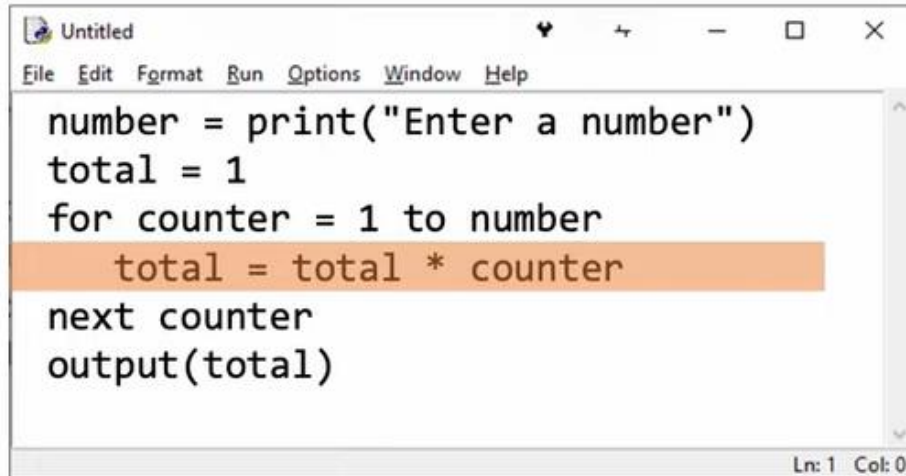
number	counter	total
5	1	1
	2	1
	3	2
	4	6

Output window:



```
Enter a number
5
```

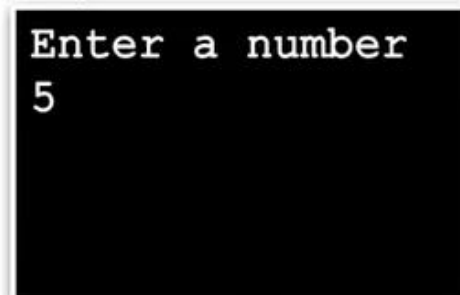
7.7 - Complete a trace table to document a dry-run of an algorithm



```
number = print("Enter a number")
total = 1
for counter = 1 to number
    total = total * counter
next counter
output(total)
```

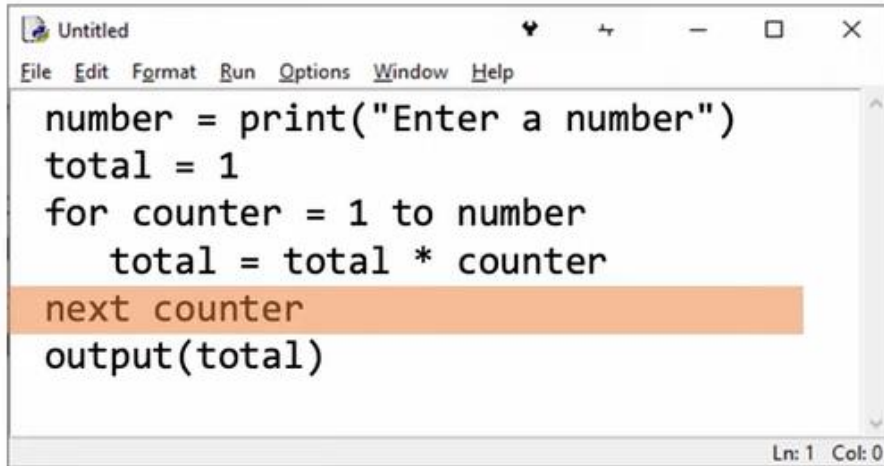
number	counter	total
5	1	1
	2	1
	3	2
	4	6
		24

Output window:



```
Enter a number
5
```

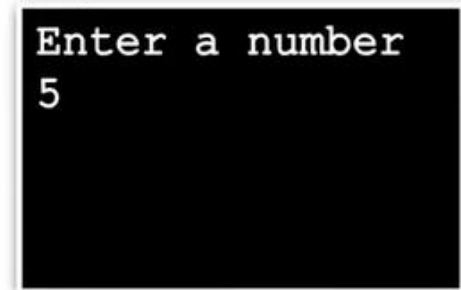
7.7 - Complete a **trace table** to document a dry-run of an algorithm



```
number = print("Enter a number")
total = 1
for counter = 1 to number
    total = total * counter
next counter
output(total)
```

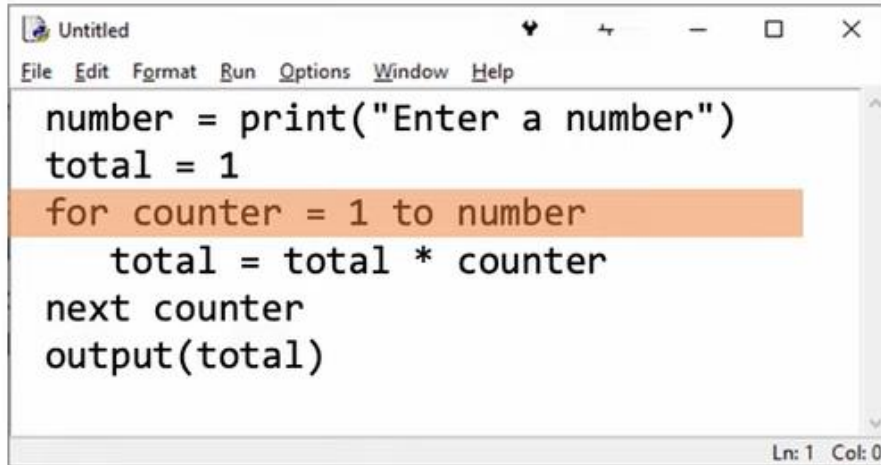
number	counter	total
5	1	1
	2	1
	3	2
	4	6
	5	24

Output window:



```
Enter a number
5
```

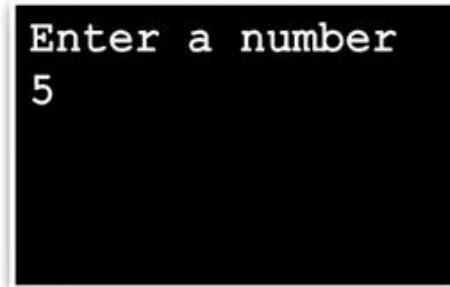
7.7 - Complete a **trace table** to document a dry-run of an algorithm



```
number = print("Enter a number")
total = 1
for counter = 1 to number
    total = total * counter
next counter
output(total)
```

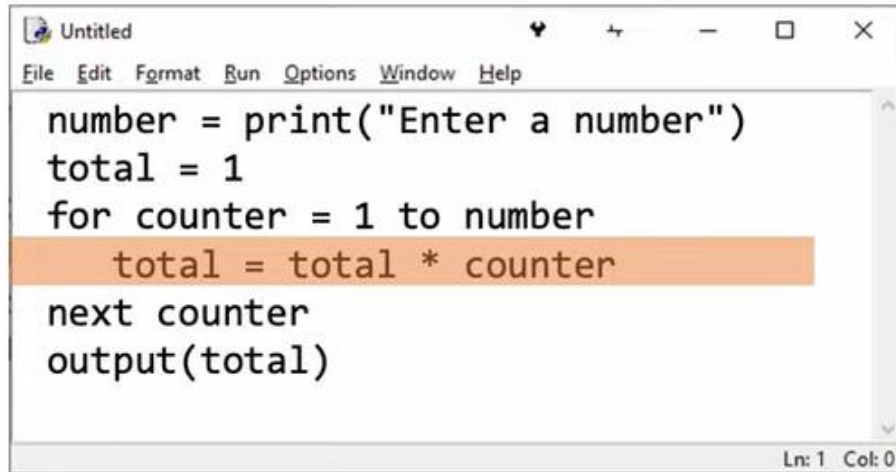
number	counter	total
5	1	1
	2	1
	3	2
	4	6
	5	24

Output window:



```
Enter a number
5
```

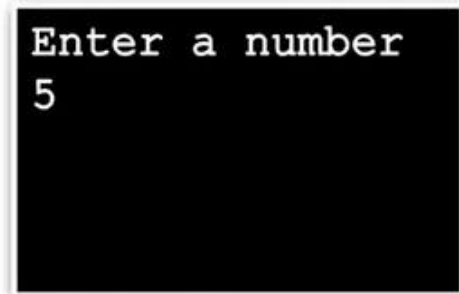
7.7 - Complete a trace table to document a dry-run of an algorithm



```
number = print("Enter a number")
total = 1
for counter = 1 to number
    total = total * counter
next counter
output(total)
```

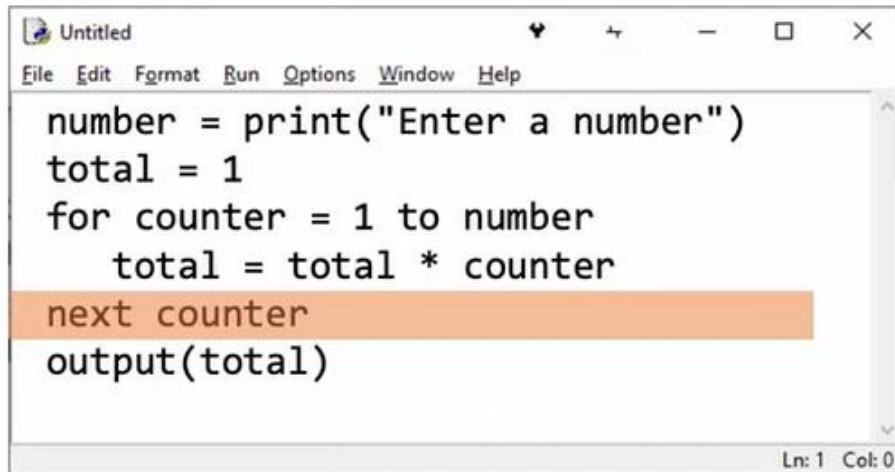
number	counter	total
5	1	1
	2	1
	3	2
	4	6
	5	24
		120

Output window:



```
Enter a number
5
```

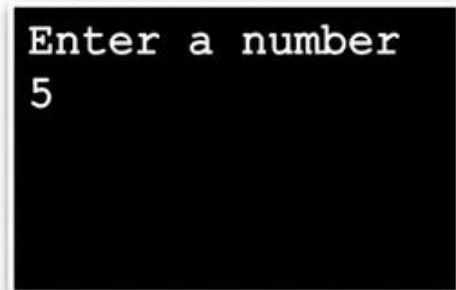
7.7 - Complete a trace table to document a dry-run of an algorithm



```
number = print("Enter a number")
total = 1
for counter = 1 to number
    total = total * counter
next counter
output(total)
```

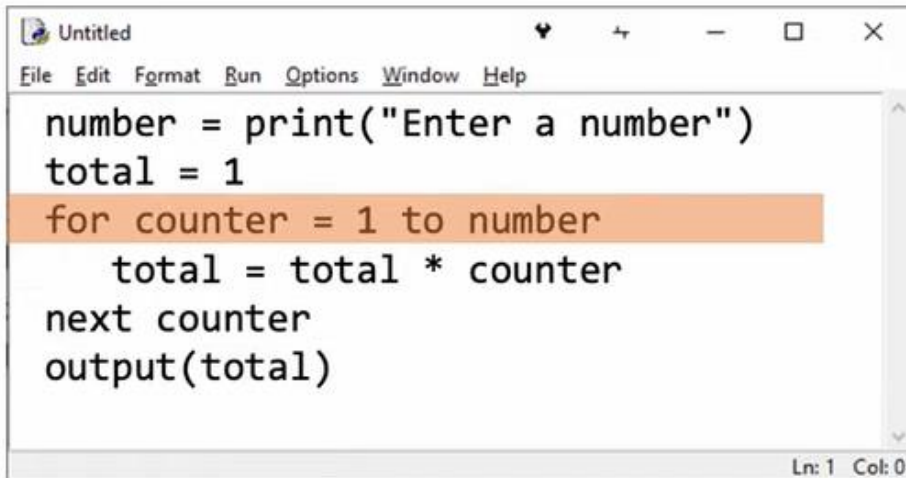
number	counter	total
5	1	1
	2	1
	3	2
	4	6
	5	24
	6	120

Output window:



```
Enter a number
5
```

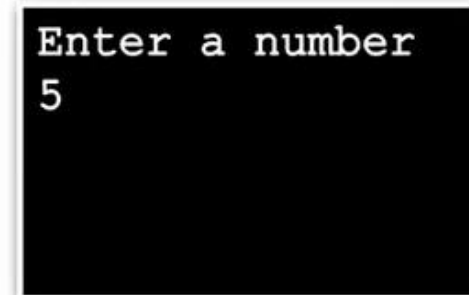
7.7 - Complete a trace table to document a dry-run of an algorithm



```
number = print("Enter a number")
total = 1
for counter = 1 to number
    total = total * counter
next counter
output(total)
```

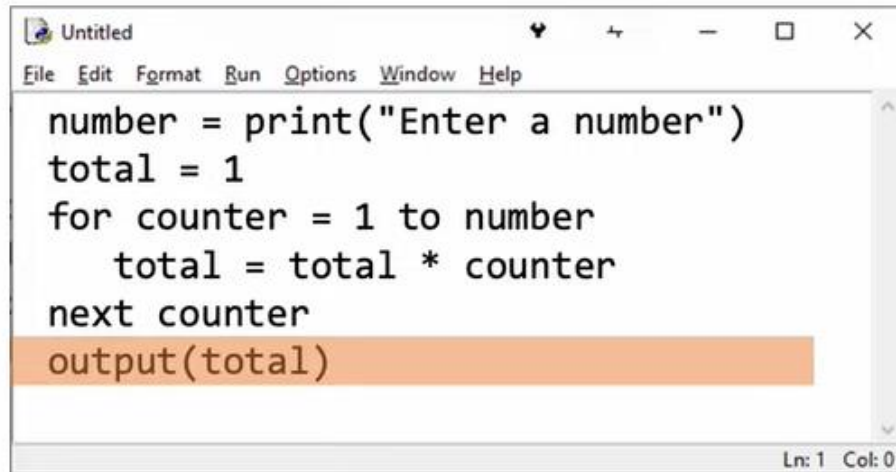
number	counter	total
5	1	1
	2	1
	3	2
	4	6
	5	24
	6	120

Output window:



```
Enter a number
5
```

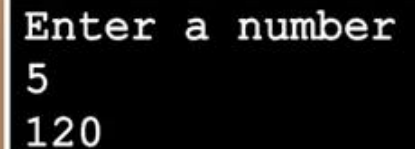
7.7 - Complete a trace table to document a dry-run of an algorithm



```
number = print("Enter a number")
total = 1
for counter = 1 to number
    total = total * counter
next counter
output(total)
```

number	counter	total
5	1	1
	2	1
	3	2
	4	6
	5	24
	6	120

Output window:



```
Enter a number
5
120
```

7.7 - Complete a trace table to document a dry-run of an algorithm

```
Untitled
File Edit Format Run Options Window Help
number = print("Enter a number")
total = 1
for counter = 1 to number
    total = total * counter
next counter
output(total)
```

What is this program doing?

It is outputting the factorial of the number entered.

*Factorial 5 = $1*2*3*4*5 = 120$*

number	counter	total
5	1	1
	2	1
	3	2
	4	6
	5	24
	6	120

Output window:

```
Enter a number
5
120
```

7.8 - Identify errors in given algorithms and suggest ways of correcting these errors.

Click the link below to watch the video Identifying errors and suggesting fixes

https://www.youtube.com/watch?v=F_BiYH9b8OQ&list=PLCiOXwirraUCKOH2duohyLRk9O75bgUdI&index=15

7 Algorithm design and problem-solving

Identifying errors and
suggesting fixes

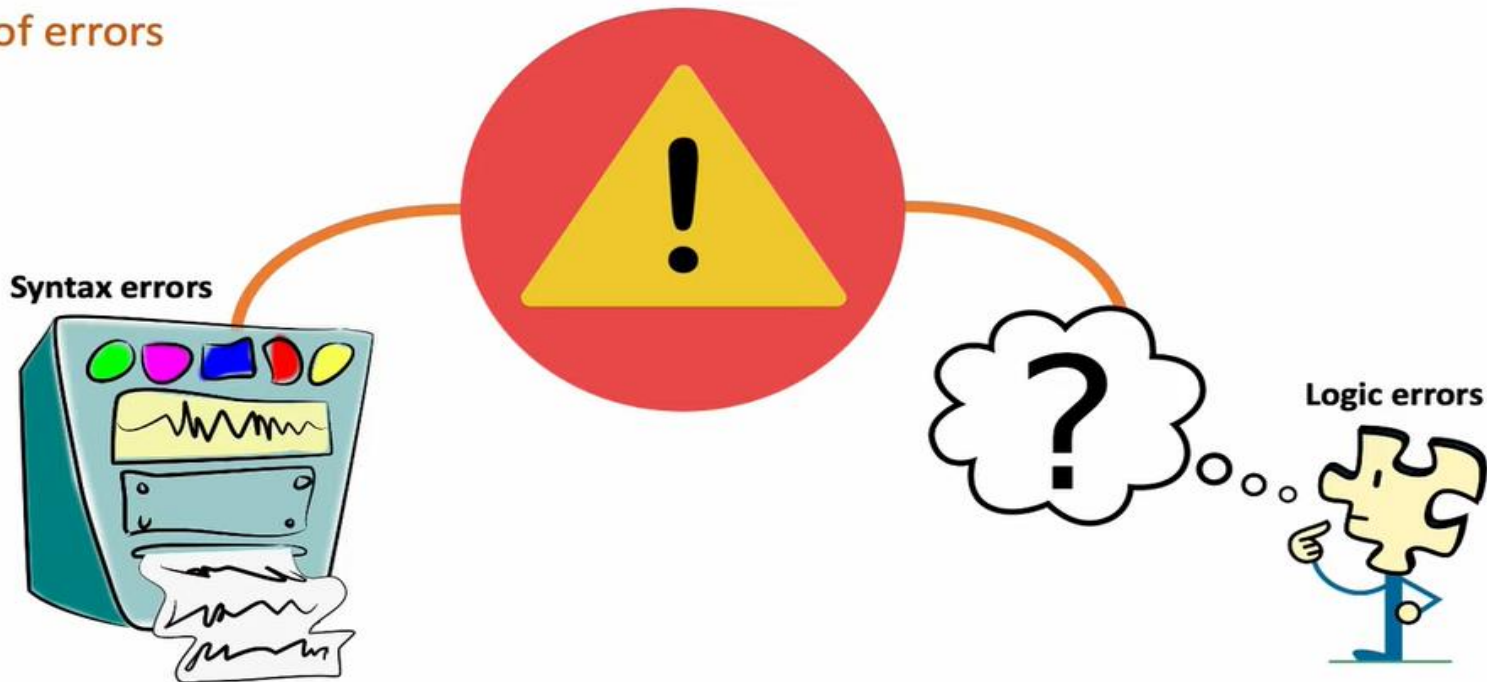
CAMBRIDGE
IGCSE
(0478/0984)



/ 3:05 •

7.8 - Identify **errors** in given algorithms and **suggest** ways of correcting these errors.

Types of errors



Syntax errors are errors that break the grammatical rules of the programming language and prevent it from being run or translated.

Logic errors are errors that produce unexpected output but won't stop the program running.

7.8 - Identify **errors** in given algorithms and **suggest** ways of correcting these errors.

Types of errors

See if you can spot the errors in this program and suggest how they can be fixed.

```
//program to work out if you are old enough to vote

CONSTANT VotingAge ← 18
OUTPUT "How hold are you?"
INPUT YourAge

IF (YourAge <= VotingAge)

    OUTPUT "You are old enough to vote"
ELSE
    OUTPUT "You are not old enough to vote"
ENDIF
```

7.8 - Identify **errors** in given algorithms and **suggest** ways of correcting these errors.

Syntax errors

Here we have missed out an essential key word – this is a **syntax** error, which will prevent the program from running.

```
//program to work out if you are old enough to vote
```

```
CONSTANT VotingAge ← 18
```

```
OUTPUT "How hold are you?"
```

```
INPUT YourAge
```

```
IF (YourAge <= VotingAge)
```



We have missed the key word THEN, which goes with the IF statement

```
    OUTPUT "You are old enough to vote"
```

```
ELSE
```

```
    OUTPUT "You are not old enough to vote"
```

```
ENDIF
```

7.8 - Identify **errors** in given algorithms and **suggest** ways of correcting these errors.

Syntax errors

Here we have missed out an essential key word – this is a **syntax** error, which will prevent the program from running.

```
//program to work out if you are old enough to vote

CONSTANT VotingAge ← 18
OUTPUT "How hold are you?"
INPUT YourAge

IF (YourAge <= VotingAge)
    THEN
        OUTPUT "You are old enough to vote"
    ELSE
        OUTPUT "You are not old enough to vote"
ENDIF
```

7.8 - Identify **errors** in given algorithms and **suggest** ways of correcting these errors.

Syntax errors

We are missing the double quotes on the end of the output line – this is a **syntax** error, which will prevent the program from running.

```
//program to work out if you are old enough to vote

CONSTANT VotingAge ← 18
OUTPUT "How hold are you?"
INPUT YourAge

IF (YourAge <= VotingAge)
    THEN
        OUTPUT "You are old enough to vote"
    ELSE
        OUTPUT "You are not old enough to vote"
    ENENDIF
```

 **Missing double quotes**

7.8 - Identify **errors** in given algorithms and **suggest** ways of correcting these errors.

Syntax errors

All the **syntax** errors have now be fixed, so the program will run. However, it won't behave as expected because a **logical** error remains – can you spot it?

```
//program to work out if you are old enough to vote

CONSTANT VotingAge ← 18
OUTPUT "How hold are you?"
INPUT YourAge

IF (YourAge <= VotingAge)
  THEN
    OUTPUT "You are old enough to vote"
  ELSE
    OUTPUT "You are not old enough to vote"
ENDIF
```

7.8 - Identify **errors** in given algorithms and **suggest** ways of correcting these errors.

Syntax errors

All the **syntax** errors have now be fixed, so the program will run. However, it won't behave as expected because a **logical** error remains – can you spot it?

```
//program to work out if you are old enough to vote

CONSTANT VotingAge ← 18
OUTPUT "How hold are you?"
INPUT YourAge

IF (YourAge <= VotingAge)
    THEN
        OUTPUT "You are old enough to vote"
    ELSE
        OUTPUT "You are not old enough to vote"
ENDIF
```

7.8 - Identify **errors** in given algorithms and **suggest** ways of correcting these errors.

Logical errors

We used the wrong comparison operator – this is a **logical** error. The program would run, but it would not behave as expected, allowing only people *under* the age of 18 to vote.

```
//program to work out if you are old enough to vote

CONSTANT VotingAge ← 18
OUTPUT "How hold are you?"
INPUT YourAge
                                <= voting_age will only permit users aged 18 or under to vote
IF (YourAge <= VotingAge)
    THEN
        OUTPUT "You are old enough to vote"
    ELSE
        OUTPUT "You are not old enough to vote"
ENDIF
```

7.8 - Identify **errors** in given algorithms and **suggest** ways of correcting these errors.

Logical errors

We used the wrong comparison operator – this is a **logical** error. The program would run, but it would not behave as expected, allowing only people *under* the age of 18 to vote.

```
//program to work out if you are old enough to vote

CONSTANT VotingAge ← 18
OUTPUT "How old are you?"
INPUT YourAge

IF (YourAge >= VotingAge)
    THEN
        OUTPUT "You are old enough to vote"
    ELSE
        OUTPUT "You are not old enough to vote"
ENDIF
```

**7.8 - Write and amend algorithms for given problems or scenarios, using:
pseudocode, program code and flowcharts**

Click the link below to watch the video Algorithmic thinking

<https://www.youtube.com/watch?v=AYhBXR-fVTI&list=PLCiOXwirraUCKOH2duohyLRk9O75bgUdI&index=16>

7 Algorithm design and problem-solving

Algorithmic thinking



CAMBRIDGE
IGCSE
(0478/0984)

11:29
2 | 30

7.8 - Write and amend algorithms for given problems or scenarios, using: pseudocode, program code and flowcharts



Algorithmic thinking

Algorithmic thinking is a way of getting to a solution by identifying the individual steps required.

By creating a set of rules, an algorithm – when followed precisely – will lead to an answer. For example, in school, we all learn algorithms for doing multiplication.

If a human (or a computer) follows these rules precisely, the answer to any multiplication problem can be found.



7.8 - Write and amend algorithms for given problems or scenarios, using: pseudocode, program code and flowcharts

Algorithmic thinking

The approach we have taken here could be used to tackle a range of problems.

- Break the problem down.
- Think about what data is important/not important.
- Consider the relevant data structures.
- Tackle each part of the problem step by step.

Algorithmic thinking takes practice.

The more you do it, the better you will become.



7.8 - Write and amend algorithms for given problems or scenarios, using: pseudocode, program code and flowcharts

Easy Example: Calculating Total Price:

Scenario: A customer wants to buy items from a store. Each item has a price. The customer wants to know the **total price** of the items they are buying.

// Initialize variables

total_price $\leftarrow$ 0

// Input prices of items

price1 $\leftarrow$ INPUT("Enter price of item 1: ")

price2 $\leftarrow$ INPUT("Enter price of item 2: ")

price3 $\leftarrow$ INPUT("Enter price of item 3: ")

// Calculate total price

total_price $\leftarrow$ price1 + price2 + price3

// Output total price

OUTPUT("Total price: £" + total_price)

```
main.py x
1  # Input prices of items
2  price1 = float(input("Enter price of item 1: "))
3  price2 = float(input("Enter price of item 2: "))
4  price3 = float(input("Enter price of item 3: "))
5
6  # Calculate total price
7  total_price = price1 + price2 + price3
8
9  # Output total price
10 print("Total price: £" + str(total_price))
```

```
C:\Users\n.tadgerout\PycharmPro
Enter price of item 1: 100
Enter price of item 2: 50
Enter price of item 3: 25
Total price: £175.0
```

7.8 - Write and amend algorithms for given problems or scenarios, using: pseudocode, program code and flowcharts

Medium Example: Calculating Factorial:

Scenario: A user wants to find the **factorial** of a number. The factorial of a **non-negative integer** **n** is the product of all **positive integers less than or equal to n**.

// Initialize variables

factorial $\leftarrow$ 1

// Input a number

number $\leftarrow$ INPUT("Enter a number: ")

// Calculate factorial

FOR i FROM 1 TO number

 factorial $\leftarrow$ factorial * i

END FOR

// Output factorial

OUTPUT("Factorial of " + number + " is " + factorial)

```
main.py x
1  # Input a number
2  number = int(input("Enter a number: "))
3
4  # Calculate factorial
5  factorial = 1
6  for i in range(1, number + 1):
7      factorial *= i
8
9  # Output factorial
10 print("Factorial of", number, "is", factorial)
11
```

```
C:\Users\n.tadjerout\Pycharm
Enter a number: 5
Factorial of 5 is 120
```

7.8 - Write and amend algorithms for given problems or scenarios, using: pseudocode, program code and flowcharts

Easy Example 1: Array of Numbers:

Scenario: Declare the array and variables

// Initialize variables (i,j, temp)

DECLARE numbers : ARRAY[1:10] OF INTEGER

DECLARE i, j, temp : INTEGER

// Input numbers into the array

FOR i FROM 1 TO 10

 numbers[i] ← INPUT("Enter number " + i + ": ")

END FOR

temp: (is a variable used to temporarily store a value)

 Declare the array and variables.py - C:/Program Files/Python312/Declare the array and varia...

File Edit Format Run Options Window Help

```
# Declare the array and variables
```

```
numbers = [0] * 10
```

```
i, j, temp = 0, 0, 0
```

```
# Input numbers into the array
```

```
for i in range(1, 11):
```

```
    numbers[i - 1] = int(input("Enter number " + str(i) + ": "))
```

```
==== RESTART: C:/Progr
```

```
Enter number 1: 2
```

```
Enter number 2: 3
```

```
Enter number 3: 4
```

```
Enter number 4: 5
```

```
Enter number 5: 6
```

```
Enter number 6: 7
```

```
Enter number 7: 8
```

```
Enter number 8: 9
```

```
Enter number 9: 10
```

```
Enter number 10: 11
```

7.8 - Write and amend algorithms for given problems or scenarios, using: pseudocode, program code and flowcharts

Scenario: A user has a list of numbers and wants to sort them in ascending order using the Bubble Sort algorithm

Medium Example 2: Sorting an Array of Numbers:

// Initialize variables (i,j, temp)

DECLARE numbers : ARRAY[1:10] OF INTEGER

DECLARE i, j, temp : INTEGER

// Input numbers into the array

FOR i FROM 1 TO 10

 numbers[i] ← INPUT("Enter number " + i + " : ")

END FOR

// Sort the array using Bubble Sort algorithm

FOR i FROM 1 TO 10

 FOR j FROM 1 TO 10 - i

 IF numbers[j] > numbers[j + 1]

 // Swap numbers[j] and numbers[j + 1]

 temp ← numbers[j]

 numbers[j] ← numbers[j + 1]

 numbers[j + 1] ← temp

 END IF

 END FOR

END FOR

 Sort the array using Bubble Sort algorithm.py - C:/Program Files/Python312/Sort the array usi...

File Edit Format Run Options Window Help

Initialize variables

numbers = [0] * 11 # Creating an array of size 11 (index 1 to 10)

temp = 0

Input numbers into the array

for i in range(1, 11):

 numbers[i] = int(input("Enter number " + str(i) + " : "))

Sort the array using Bubble Sort algorithm

for i in range(1, 11):

 for j in range(1, 11 - i):

 if numbers[j] > numbers[j + 1]:

 # Swap numbers[j] and numbers[j + 1]

 temp = numbers[j]

 numbers[j] = numbers[j + 1]

 numbers[j + 1] = temp

Output the sorted array

print("Sorted array:", numbers[1:])

= RESTART: C:/Program Files/Python312/Sort the
-py

Enter number 1: 2

Enter number 2: 3

Enter number 3: 4

Enter number 4: 5

Enter number 5: 6

Enter number 6: 7

Enter number 7: 8

Enter number 8: 9

Enter number 9: 10

Enter number 10: 11

Sorted array: [2, 3, 4, 5, 6, 7, 8, 9, 10, 11]

7.8 - Write and amend algorithms for given problems or scenarios, using: pseudocode, program code and flowcharts

// Initialize variables (i,j, temp)

DECLARE numbers : ARRAY[1:10] OF INTEGER

DECLARE i, j, temp : INTEGER

// Input numbers into the array

FOR i FROM 1 TO 10

numbers[i] ← INPUT("Enter number " + i + ": ")

END FOR

// Sort the array using Bubble Sort algorithm

FOR i FROM 1 TO 10

FOR j FROM 1 TO 10 - i

IF numbers[j] > numbers[j + 1]

// Swap numbers[j] and numbers[j + 1]

temp ← numbers[j]

numbers[j] ← numbers[j + 1]

numbers[j + 1] ← temp

END IF

END FOR

END FOR

// Output sorted numbers

OUTPUT("Sorted numbers: ")

FOR i FROM 1 TO 10

OUTPUT(numbers[i])

END FOR

Output sorted numbers.py - C:/Program Files/Python312/Output sorted numbers.py (3.12.0)

File Edit Format Run Options Window Help

Initialize variables (i, j, temp)

numbers = [0] * 11 # Creating an array of size 11 (1 to 10)

i = j = temp = 0

Input numbers into the array

for i in range(1, 11):

 numbers[i] = int(input("Enter number " + str(i) + ": "))

Sort the array using Bubble Sort algorithm

for i in range(1, 11):

 for j in range(1, 11 - i):

 if numbers[j] > numbers[j + 1]:

 # Swap numbers[j] and numbers[j + 1]

 temp = numbers[j]

 numbers[j] = numbers[j + 1]

 numbers[j + 1] = temp

Output sorted numbers

print("Sorted numbers:")

for i in range(1, 11):

 print(numbers[i])

===== RESTART: C:/

Enter number 1: 3

Enter number 2: 2

Enter number 3: 5

Enter number 4: 7

Enter number 5: 4

Enter number 6: 1

Enter number 7: 8

Enter number 8: 10

Enter number 9: 9

Enter number 10: 11

Sorted numbers:

1

2

3

4

5

7

8

9

10

11

7.8 - Write and amend algorithms for given problems or scenarios, using: pseudocode, program code and flowcharts

The expression **10 - i** represents subtracting the value of **i** from **10** to illustrate **10 - i**. Suppose we have a **loop** with **i ranging** from **1** to **5**:

 for loop 1.py - C:/Program Files/Python312/for loop 1.py (3.12.0)
File Edit Format Run Options Window Help

```
for i in range(1, 6):  
    limit = 10 - i  
    print(f"For i = {i}, the limit is {limit}")
```

```
===== RESTART: C:/Progr  
For i = 1, the limit is 9  
For i = 2, the limit is 8  
For i = 3, the limit is 7  
For i = 4, the limit is 6  
For i = 5, the limit is 5  
|
```

In this example, $10 - i$ is calculating the limit for the inner loop. As i increases, the limit decreases, ensuring that the inner loop doesn't unnecessarily iterate over the entire range from 1 to 10. This is a common technique used in algorithms like the bubble sort algorithm to optimize the number of iterations.

Past paper 2 Exam Questions and Answers:

Question 1:

3 (a) Describe what is meant by data validation.

.....

.....

.....

..... [2]

(b) A validation check is used to make sure that any value that is input is an integer between 30 and 200 inclusive.

Give **one** example of each type of test data to check that the validation check is working as intended. Each example of test data must be different.

Give a reason for each of your choices of test data.

Normal test data

Reason

.....

Abnormal test data

Reason

.....

Extreme test data

Reason

.....

Past paper 2 Exam Questions and Answers:

Answer 1:

Question	Answer	Marks
3(a)	One mark per mark point, max two • Validation is an automated check carried out by a computer • ... to make sure the data entered is sensible/acceptable/reasonable	2
3(b)	One mark for each appropriate test data, max three One mark for each correct accompanying reason, max three For example: Normal – 75 Reason – the data lies within the required range and should be accepted Abnormal – Sixty Reason – this is the wrong data type and should be rejected Extreme – 200 Reason – the highest value in the required range that should be accepted	6

Past paper 2 Exam Questions and Answers:

Question 2:

- 5 An algorithm has been written in pseudocode to allow some numbers to be input. All the positive numbers that are input are totalled and this total is output at the end. An input of 0 stops the algorithm.

```
01 Exit ← 1
02 WHILE Exit <> 0 DO
03     INPUT Number
04     IF Number < 0
05         THEN
06             Total ← Total + Number
07         ELSE
08             IF Number = 0
09                 THEN
10                     Exit ← 1
11             ENDIF
12         ENDIF
13 ENDIF
14 OUTPUT "The total value of your numbers is ", Number
```

Past paper 2 Exam Questions and Answers:

Question 2:

(a) Identify the **four** errors in the pseudocode and suggest a correction for each error.

Error 1

Correction

.....

Error 2

Correction

.....

Error 3

Correction

.....

Error 4

Correction

.....

[4]

Past paper 2 Exam Questions and Answers:

Question 2:

(b) Describe how you could change the corrected algorithm to record and output how many positive numbers have been included in the final total.

You do **not** need to rewrite the algorithm.

.....

.....

.....

.....

.....

.....

.....

..... [4]

Past paper 2 Exam Questions and Answers:

Answer 2:

Question	Answer	Marks
5(a)	One mark per mark point, max four - Line 04 / IF Number < 0 should be IF Number > 0 - Line 10 / Exit ← 1 // Line 01/ Exit ← 1 and Line 02 / WHILE Exit <> 0 should be Exit ← 0 // should be Exit ← 0 and WHILE Exit = 0 - Line 13 / ENDIF should be ENDWHILE - Line 14 / OUTPUT "The total value of your numbers is ", Number should be OUTPUT "The total value of your numbers is ", Total Correct algorithm: <pre> 01 Exit ← 1 02 WHILE Exit <> 0 DO 03 INPUT Number 04 IF Number > 0 05 THEN 06 Total ← Total + Number 07 ELSE 08 IF Number = 0 09 THEN 10 Exit ← 0 11 ENDIF 12 ENDIF 13 ENDWHILE 14 OUTPUT "The total value of your numbers is ", Total </pre>	4

Past paper 2 Exam Questions and Answers:

Answer 2:

 t2loop.py - C:/Program Files/Python312/t2loop.py (3.12.0)

File Edit Format Run Options Window Help

```
# Initialize Exit variable
Exit = 1
Total = 0

# Continue looping until Exit becomes 0
while Exit != 0:
    # Input a number
    Number = int(input("Enter a number: "))

    # Check if the number is greater than 0
    if Number > 0:
        # Add the number to the total
        Total += Number
    else:
        # Check if the number is equal to 0
        if Number == 0:
            # Set Exit to 0 to exit the loop
            Exit = 0

# Output the total value of the numbers
print("The total value of your numbers is", Total)
```

 IDLE Shell 3.12.0

File Edit Shell Debug Options Window Help

```
Python 3.12.0 (tags/v3.12.0:0fb18b0, 0
AMD64)] on win32
Type "help", "copyright", "credits" or
>>>
= RESTART: C:/Program Files/Python312/
Enter a number: 1
Enter a number: 2
Enter a number: 3
Enter a number: 0
The total value of your numbers is 6
```

Past paper 2 Exam Questions and Answers:

Answer 2:

Question	Answer	Marks
5(b)	One mark per mark point, max four • Initialise a new (counting) variable • ... Count $\leftarrow$ 0 // to count the acceptable numbers • Insert a counting statement between lines 05 and 07 • ... Count $\leftarrow$ Count + 1 • Add a new output after the loop/after line 13 / at the end (of the program) • ... OUTPUT Count	4

 t3loop.py - C:/Program Files/Python312/t3loop.py (3.12.0)

File Edit Format Run Options Window Help

```
# Initialize Exit variable, Total, and Count
Exit = 1
Total = 0
Count = 0

# Continue looping until Exit becomes 0
while Exit != 0:
    # Input a number
    Number = int(input("Enter a number: "))

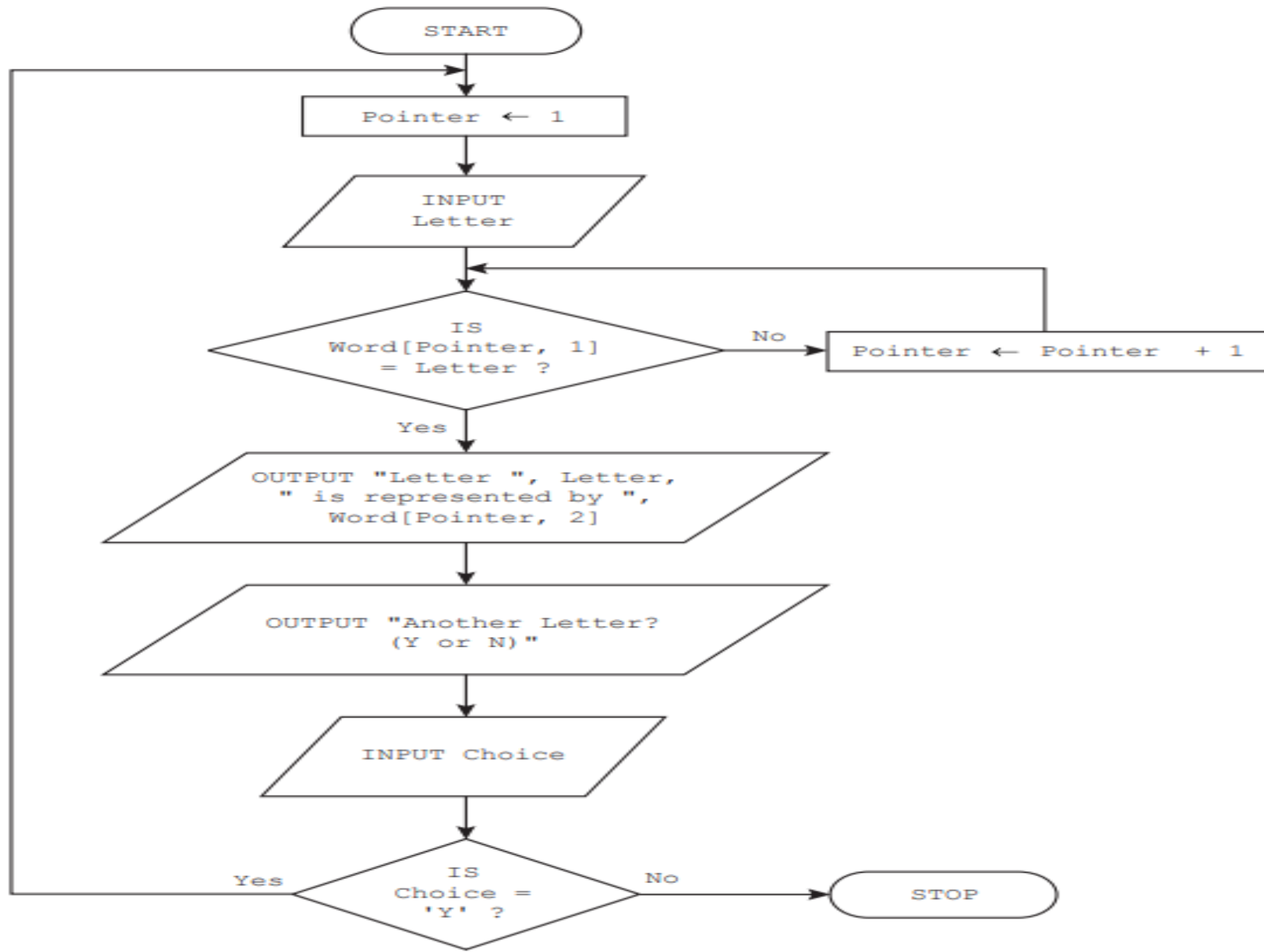
    # Increment the count for each acceptable number (r
    Count = Count + 1

    # Check if the number is greater than 0
    if Number > 0:
        # Add the number to the total
        Total += Number
    else:
        # Check if the number is equal to 0
        if Number == 0:
            # Set Exit to 0 to exit the loop
            Exit = 0

# Output the total value of the numbers
print("The total value of your numbers is", Total)

# Output the count of acceptable numbers
print("Number of acceptable numbers entered:", Count)
```

```
===== RESTART: C:/Program F
Enter a number: 1
Enter a number: 2
Enter a number: 3
Enter a number: 0
The total value of your numbers is 6
Number of acceptable numbers entered: 4
|
```



Past paper 2 Exam Questions and Answers:

Question 3:

The table represents the two-dimensional (2D) array `Word[]` which stores the first half of the phonetic alphabet used for radio transmission. For example, `Word[10,1]` is 'J'.

Index	1	2
1	A	Alpha
2	B	Bravo
3	C	Charlie
4	D	Delta
5	E	Echo
6	F	Foxtrot
7	G	Golf
8	H	Hotel
9	I	India
10	J	Juliet
11	K	Kilo
12	L	Lima
13	M	Mike

Past paper 2 Exam Questions and Answers:

Question 3:

(a) Complete the trace table for the algorithm by using the input data: F, Y, D, N

Pointer	Letter	Choice	OUTPUT

Past paper 2 Exam Questions and Answers:

Question 3:

(b) Identify the type of algorithm used.

.....

..... [1]

(c) Describe **one** problem that could occur with this algorithm if an invalid character was input.

.....

.....

.....

..... [2]

Past paper 2 Exam Questions and Answers:

Answer 3:

Question	Answer	Marks																																																												
7(a)	One mark per correct column, max four <table><tr><th>Pointer</th><th>Letter</th><th>Choice</th><th>OUTPUT</th></tr><tr><td>1</td><td>F</td><td></td><td></td></tr><tr><td>2</td><td></td><td></td><td></td></tr><tr><td>3</td><td></td><td></td><td></td></tr><tr><td>4</td><td></td><td></td><td></td></tr><tr><td>5</td><td></td><td></td><td></td></tr><tr><td>6</td><td></td><td></td><td>Letter F is represented by Foxtrot</td></tr><tr><td></td><td></td><td></td><td>Another Letter? (Y or N)</td></tr><tr><td></td><td></td><td>Y</td><td></td></tr><tr><td>1</td><td>D</td><td></td><td></td></tr><tr><td>2</td><td></td><td></td><td></td></tr><tr><td>3</td><td></td><td></td><td></td></tr><tr><td>4</td><td></td><td></td><td>Letter D is represented by Delta</td></tr><tr><td></td><td></td><td></td><td>Another Letter? (Y or N)</td></tr><tr><td></td><td></td><td>N</td><td></td></tr></table>	Pointer	Letter	Choice	OUTPUT	1	F			2				3				4				5				6			Letter F is represented by Foxtrot				Another Letter? (Y or N)			Y		1	D			2				3				4			Letter D is represented by Delta				Another Letter? (Y or N)			N		4
Pointer	Letter	Choice	OUTPUT																																																											
1	F																																																													
2																																																														
3																																																														
4																																																														
5																																																														
6			Letter F is represented by Foxtrot																																																											
			Another Letter? (Y or N)																																																											
		Y																																																												
1	D																																																													
2																																																														
3																																																														
4			Letter D is represented by Delta																																																											
			Another Letter? (Y or N)																																																											
		N																																																												
7(b)	(Linear) search	1																																																												

Past paper 2 Exam Questions and Answers:

Answer 3:

PUBLISHED

Question	Answer	Marks
7(c)	One mark per mark point, max two • The algorithm would not stop • ... because it would not have found the item it was seeking Or • The array would run out of values after the pointer reached 13 • the algorithm will crash	2

Past paper 2 Exam Questions and Answers:

Question 4:

3 Identify **three** different ways that the design of a solution to a problem can be presented.

1

.....

2

.....

3

.....

[3]

Past paper 2 Exam Questions and Answers:

Answer 4:

Question	Answer	Marks
3	One mark for each correct answer • structure diagram / chart • flowchart • pseudocode	3

Past paper 2 Exam Questions and Answers:

Question 5:

- 4 A program needs to make sure the value input for a measurement meets the following rules:
- the value is a positive number
 - a value is always input
 - the value is less than 1000.
- (a) Describe the validation checks that the programmer would need to use.
-
-
-
-
-
- [3]
- (b) The program needs editing to include a double entry check for the value input.
- (i) State why this check needs to be included.
-
- [1]

Past paper 2 Exam Questions and Answers:

Question 5:

- (ii) The input value needs to be stored in the variable `Measurement`
Write pseudocode to perform the double entry check until a successful input is made.

.....

.....

.....

.....

.....

.....

.....

..... [3]

Past paper 2 Exam Questions and Answers:

Answer 5:

Question	Answer	Marks
4(a)	One mark for each point (max three). • range check with acceptable values is (greater than) zero and less than 1000 • presence check to ensure the program will not continue until a value has been entered • type/character check to ensure that a number is entered • length check to ensure there are no more than 3 digits entered	3
4(b)(i)	To verify the data / for verification / as a verification check // to make sure that no changes are made to the data on entry	1
4(b)(ii)	One mark for each point (max three). • use of iteration • use of two inputs • to check that the two inputs are the same / different • use of the given variable <code>Measurement</code> For example <pre>REPEAT OUTPUT "Please enter measurement " INPUT Measurement OUTPUT "Please re-enter measurement " INPUT MeasurementCheck UNTIL Measurement = MeasurementCheck</pre>	3

Question	Answer	Marks
7(a)	07 04/12 or 16/18 02/20	3
7(b)	One mark for each error identified and correction - Line 07 <code>Total ← Total + Number * Counter</code> should be <code>Total ← Total + Number[Counter] * Counter</code> - Line 08 <code>IF Number[Counter] = 0</code> should be <code>IF Number[Counter] = -1 // should be IF Number[Counter] < 0</code> - Line 16 <code>FOR Counter ← 0 TO 5</code> should be <code>FOR Counter ← 1 TO 5</code>	3

- 7** An algorithm has been written in pseudocode to calculate a check digit for a four-digit number. The algorithm then outputs the five-digit number including the check digit. The algorithm stops when -1 is input as the fourth digit.

```

01 Flag ← FALSE
02 REPEAT
03     Total ← 0
04     FOR Counter ← 1 TO 4
05         OUTPUT "Enter a digit ", Counter
06         INPUT Number[Counter]
07         Total ← Total + Number * Counter
08         IF Number[Counter] = 0
09             THEN
10                 Flag ← TRUE
11             ENDIF
12     NEXT Counter
13     IF NOT Flag
14         THEN
15             Number[5] ← MOD(Total, 10)
16             FOR Counter ← 0 TO 5
17                 OUTPUT Number[Counter]
18             NEXT
19         ENDIF
20 UNTIL Flag

```

- (a)** Give the line number(s) for the statements showing:

Totalling

Count-controlled loop

Post-condition loop

Past paper 2 Exam Questions and Answers:

Question 6:

(b) Identify the **three** errors in the pseudocode and suggest a correction for each error.

Error 1

Correction

.....

Error 2

Correction

.....

Error 3

Correction

.....

[3]

Past paper 2 Exam Questions and Answers:

Question 6:

- (c) The algorithm does **not** check that each input is a single digit.
Identify the place in the algorithm where this check should occur.
Write pseudocode for this check.
Your pseudocode must make sure that the input is a single digit and checks for –1

Place in algorithm

Pseudocode

.....

.....

.....

.....

.....

[4]

Past paper 2 Exam Questions and Answers:

Answer 6:

Question	Answer	Marks
7(a)	• 07 • 04/12 or 16/18 • 02/20	3
7(b)	One mark for each error identified and correction • Line 07 <code>Total ← Total + Number * Counter</code> should be <code>Total ← Total + Number[Counter] * Counter</code> • Line 08 <code>IF Number[Counter] = 0</code> should be <code>IF Number[Counter] = -1 // should be IF Number[Counter] < 0</code> • Line 16 <code>FOR Counter ← 0 TO 5</code> should be <code>FOR Counter ← 1 TO 5</code>	3

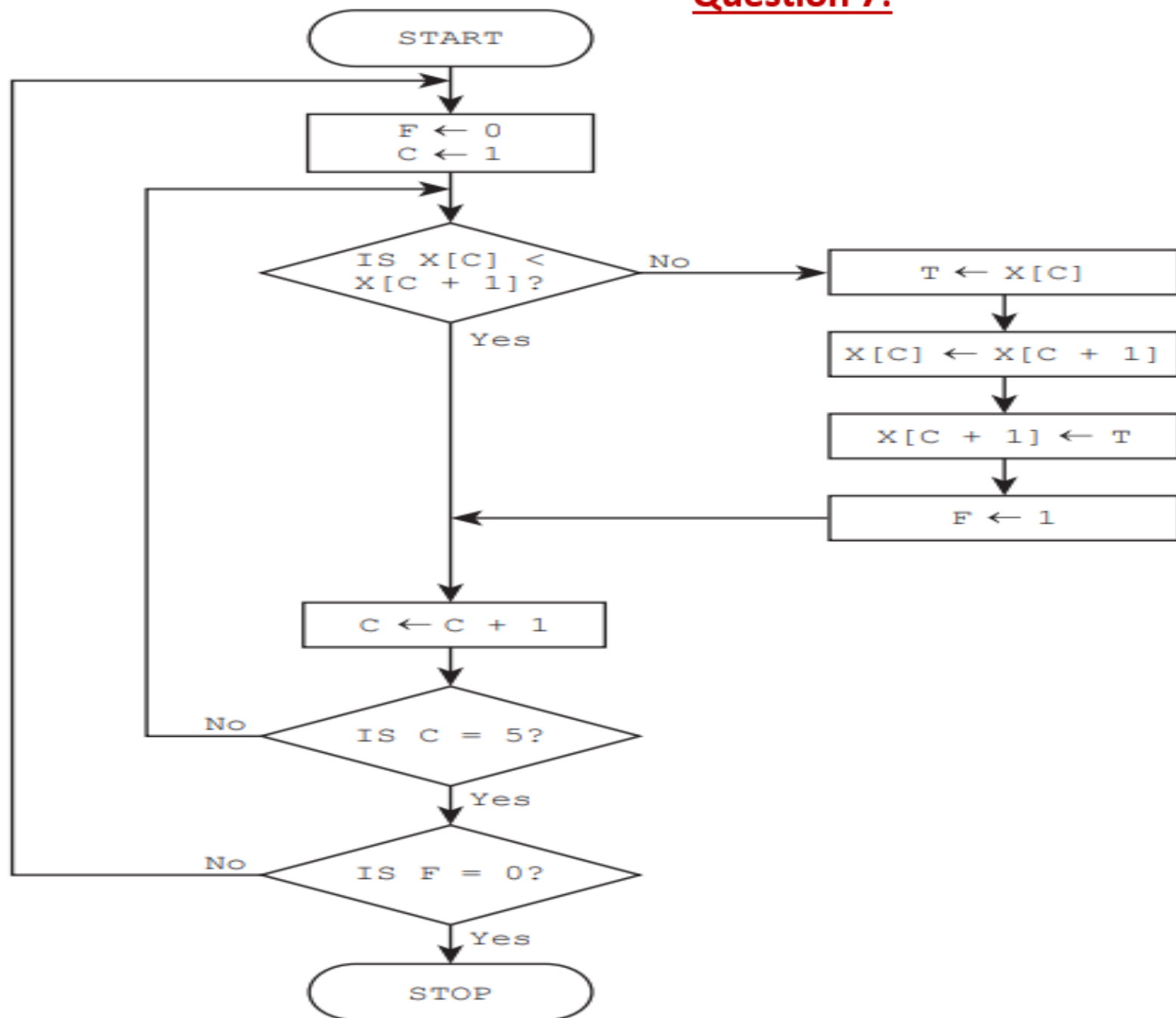
Past paper 2 Exam Questions and Answers:

Answer 6:

Question	Answer	Marks
7(c)	One mark for place in algorithm (max one) • around lines 05 and 06 • line 07 • (immediately) after the input of the number Three marks pseudocode One mark for each point (max three) • Use of REPEAT ... UNTIL // any working loop structure • check for >0 // >=0 • check for <10 // >9 • check for whole number • check for -1 • check for length of digit <> 1 Example <pre>REPEAT OUTPUT "Enter a digit " INPUT Number[Counter] UNTIL Number[Counter] = Round(Number[Counter],0) AND ((Number[Counter] = -1) OR (Number[Counter] > 0 AND Number[Counter] < 10))</pre>	4

9 This flowchart represents an algorithm.

Question 7:



(a) The array $x[1:5]$ used in the flowchart contains this data:

Question 7:

$x[1]$	$x[2]$	$x[3]$	$x[4]$	$x[5]$
10	1	5	7	11

Complete the trace table by using the data given in the array.

F	C	$x[1]$	$x[2]$	$x[3]$	$x[4]$	$x[5]$	T
		10	1	5	7	11	

[5]

(b) Describe what the algorithm represented by the flowchart is doing.

Past paper 2 Exam Questions and Answers:

Answer 7:

Question	Answer	Marks																																																																																																								
9(a)	One mark for each column F, C and T Two marks for columns X[1] to X[5] all entries correct or One mark for columns X[1] to X[5] with one error <table><tr><th>F</th><th>C</th><th>X[1]</th><th>X[2]</th><th>X[3]</th><th>X[4]</th><th>X[5]</th><th>T</th></tr><tr><td></td><td></td><td>10</td><td>1</td><td>5</td><td>7</td><td>11</td><td></td></tr><tr><td>0</td><td>1</td><td></td><td></td><td></td><td></td><td></td><td>10</td></tr><tr><td>1</td><td>2</td><td>1</td><td>10</td><td></td><td></td><td></td><td>10</td></tr><tr><td>1</td><td>3</td><td></td><td>5</td><td>10</td><td></td><td></td><td>10</td></tr><tr><td>1</td><td>4</td><td></td><td></td><td>7</td><td>10</td><td></td><td></td></tr><tr><td></td><td>5</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>0</td><td>1</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td>2</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td>3</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td>4</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td>5</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr></table>	F	C	X[1]	X[2]	X[3]	X[4]	X[5]	T			10	1	5	7	11		0	1						10	1	2	1	10				10	1	3		5	10			10	1	4			7	10				5							0	1								2								3								4								5															5
F	C	X[1]	X[2]	X[3]	X[4]	X[5]	T																																																																																																			
		10	1	5	7	11																																																																																																				
0	1						10																																																																																																			
1	2	1	10				10																																																																																																			
1	3		5	10			10																																																																																																			
1	4			7	10																																																																																																					
	5																																																																																																									
0	1																																																																																																									
	2																																																																																																									
	3																																																																																																									
	4																																																																																																									
	5																																																																																																									
9(b)	One mark for each point • (bubble) sort data in array • in ascending order	2																																																																																																								

Past paper 2 Exam Questions and Answers:

Question 8:

4 (a) Explain why verification checks are used when data is input.

.....

.....

.....

..... [2]

(b) Give **two** types of verification check and state how each one can be used.

Verification check 1

Use

.....

Verification check 2

Use

..... [4]

Past paper 2 Exam Questions and Answers:

Answer 8:

Question	Answer	Marks
4(a)	One mark per mark point, max two - To ensure that data has been accurately copied // to ensure that changes have not been made to the values originally intended when data is copied ... from one source to another	2

Question	Answer	Marks
4(b)	One mark for each appropriate verification check, max two One mark for each correct accompanying use, max two For example: Verification check 1 – Visual check Use – the user looks through the data that has been entered and confirms that no changes have been made. Verification check 2 – Double data entry Use – data is entered twice, the two entries are compared and if they do not match, a re-entry is requested.	4

Past paper 2 Exam Questions and Answers:

Question 9:

5 (a) Four descriptions of validation checks are shown.

Draw **one** line to link each description to the most appropriate check.

Not all checks will be used.

Description	Check
to check that the data entered is an integer	check digit
to check that some data has been entered	format check
to check that the data entered has an appropriate number of characters	length check
to check that an identification number contains no errors	presence check
	type check

[4]

Past paper 2 Exam Questions and Answers:

Answer 9:

Question	Answer	Marks												
5(a)	One mark for each correct line. <table><thead><tr><th>Description</th><th>Check</th></tr></thead><tbody><tr><td>to check that the data entered is an integer</td><td>check digit</td></tr><tr><td>to check that some data has been entered</td><td>format check</td></tr><tr><td>to check that the data entered has an appropriate number of characters</td><td>length check</td></tr><tr><td>to check that an identification number contains no errors</td><td>presence check</td></tr><tr><td></td><td>type check</td></tr></tbody></table>	Description	Check	to check that the data entered is an integer	check digit	to check that some data has been entered	format check	to check that the data entered has an appropriate number of characters	length check	to check that an identification number contains no errors	presence check		type check	4
Description	Check													
to check that the data entered is an integer	check digit													
to check that some data has been entered	format check													
to check that the data entered has an appropriate number of characters	length check													
to check that an identification number contains no errors	presence check													
	type check													

Past paper 2 Exam Questions and Answers:

Question 10:

- 6 An algorithm has been written in pseudocode to allow 100 positive numbers to be input. The total and the average of the numbers are output.

```
01 Counter ← 100
02 Total ← 0
03 WHILE Counter > 100 DO
04     INPUT Number
05     IF Number > 0
06         THEN
07             Total ← Total + Counter
08             Counter ← Counter + 1
09     ENDCASE
10 ENDWHILE
11 OUTPUT "The total value of your numbers is ", Total
12 OUTPUT "The average value of your numbers is ", Total / 100
```

Past paper 2 Exam Questions and Answers:

Question 10:

(a) Identify the **four** errors in the pseudocode and suggest corrections.

Error 1

Correction

Error 2

Correction

Error 3

Correction

Error 4

Correction

[4]

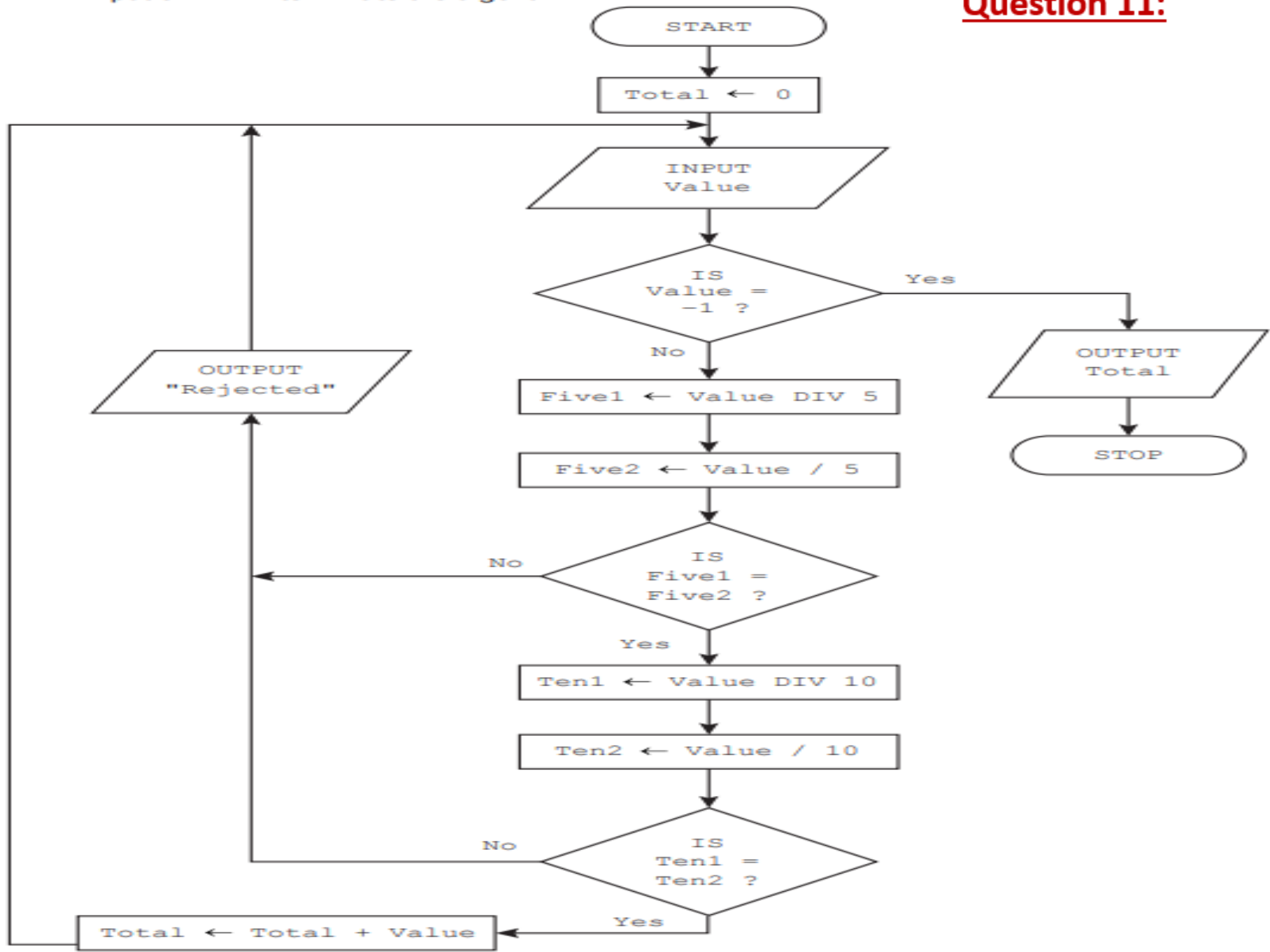
Past paper 2 Exam Questions and Answers:

Answer 10:

Question	Answer	Marks
6(a)	One mark per mark point, max four - Line 01 / Counter $\leftarrow$ 100 should be Counter $\leftarrow$ 0 - Line 03 / While Counter > 100 DO should be While Counter < 100 DO - Line 07 / Total $\leftarrow$ Total + Counter should be Total $\leftarrow$ Total + Number - Line 09 / ENDCASE should be ENDIF Correct algorithm <pre>01 Counter $\leftarrow$ 0 02 Total $\leftarrow$ 0 03 WHILE Counter < 100 DO 04 INPUT Number 05 IF Number > 0 06 THEN 07 Total $\leftarrow$ Total + Number 08 Counter $\leftarrow$ Counter + 1 09 ENDIF 10 ENDWHILE 11 OUTPUT "The total value of your numbers is ", Total 12 OUTPUT "The average value of your numbers is ", Total / 100</pre>	4

- 7 The flowchart represents an algorithm.
An input of -1 will terminate the algorithm.

Question 11:



(a) Complete the trace table for the input data:

5, 50, 52, 555, 57, 500, -1, 5500, 55

Question 11:

Total	Value	Five1	Five2	Ten1	Ten2	OUTPUT

Past paper 2 Exam Questions and Answers:

Question 11:

(b) Describe the purpose of the algorithm.

.....

.....

.....

..... [2]

Past paper 2 Exam Questions and Answers:

Answer 11:

Question	Answer	Marks																																																																													
7(a)	One mark per mark point, max six • correct <code>Total</code> column • correct <code>Value</code> column • correct <code>Five1</code> column • correct <code>Five2</code> column • correct <code>Ten1</code> and <code>Ten2</code> columns • correct <code>OUTPUT</code> column <table><tr><th>Total</th><th>Value</th><th>Five1</th><th>Five2</th><th>Ten1</th><th>Ten2</th><th>OUTPUT</th></tr><tr><td>0</td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td>5</td><td>1</td><td>1</td><td>0</td><td>0.5</td><td>Rejected</td></tr><tr><td></td><td>50</td><td>10</td><td>10</td><td>5</td><td>5</td><td></td></tr><tr><td>50</td><td>52</td><td>10</td><td>10.4</td><td></td><td></td><td>Rejected</td></tr><tr><td></td><td>555</td><td>111</td><td>111</td><td>55</td><td>55.5</td><td>Rejected</td></tr><tr><td></td><td>57</td><td>11</td><td>11.4</td><td></td><td></td><td>Rejected</td></tr><tr><td></td><td>500</td><td>100</td><td>100</td><td>50</td><td>50</td><td></td></tr><tr><td>550</td><td>−1</td><td></td><td></td><td></td><td></td><td>550</td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td><td></td></tr></table>	Total	Value	Five1	Five2	Ten1	Ten2	OUTPUT	0								5	1	1	0	0.5	Rejected		50	10	10	5	5		50	52	10	10.4			Rejected		555	111	111	55	55.5	Rejected		57	11	11.4			Rejected		500	100	100	50	50		550	−1					550															6
Total	Value	Five1	Five2	Ten1	Ten2	OUTPUT																																																																									
0																																																																															
	5	1	1	0	0.5	Rejected																																																																									
	50	10	10	5	5																																																																										
50	52	10	10.4			Rejected																																																																									
	555	111	111	55	55.5	Rejected																																																																									
	57	11	11.4			Rejected																																																																									
	500	100	100	50	50																																																																										
550	−1					550																																																																									
7(b)	One mark per mark point, max two • to find if an input is divisible by (both 5 and) 10 • ... add them together and output the total	2																																																																													

Past paper 2 Exam Questions and Answers:

Question 12:

An algorithm allows a user to input their password and checks that there are at least eight characters in the password. Then, the user is asked to re-input the password to check that both inputs are the same. The user is allowed three attempts at inputting a password of the correct length and a matching pair of passwords. The pre-defined function `LEN(X)` returns the number of characters in the string, `X`

```
01  Attempt ← 0
02  REPEAT
03      PassCheck ← TRUE
04      OUTPUT "Please enter your password "
05      INPUT Password
06      IF LEN(Password) < 8
07          THEN
08              PassCheck ← TRUE
09          ELSE
10              OUTPUT "Please re-enter your password "
11              INPUT Password2
12              IF Password <> Password2
13                  THEN
14                      PassCheck ← FALSE
15              ENDIF
16          ENDIF
17      Attempt ← Attempt + 1
18  UNTIL PassCheck OR Attempt <> 3
19  IF PassCheck
20      THEN
21          OUTPUT "Password success"
22      ELSE
23          OUTPUT "Password fail"
24  ENDIF
```

Past paper 2 Exam Questions and Answers:

Question 12:

- (a) Identify the **three** errors in the pseudocode and suggest a correction to remove each error.
- Error 1
- Correction
- Error 2
- Correction
- Error 3
- Correction [3]
- (b) The algorithm includes **two** types of check on the data input.
Identify and describe each type of check.
- Type of check 1
- Description
-
- Type of check 2
- Description
- [4]

Past paper 2 Exam Questions and Answers:

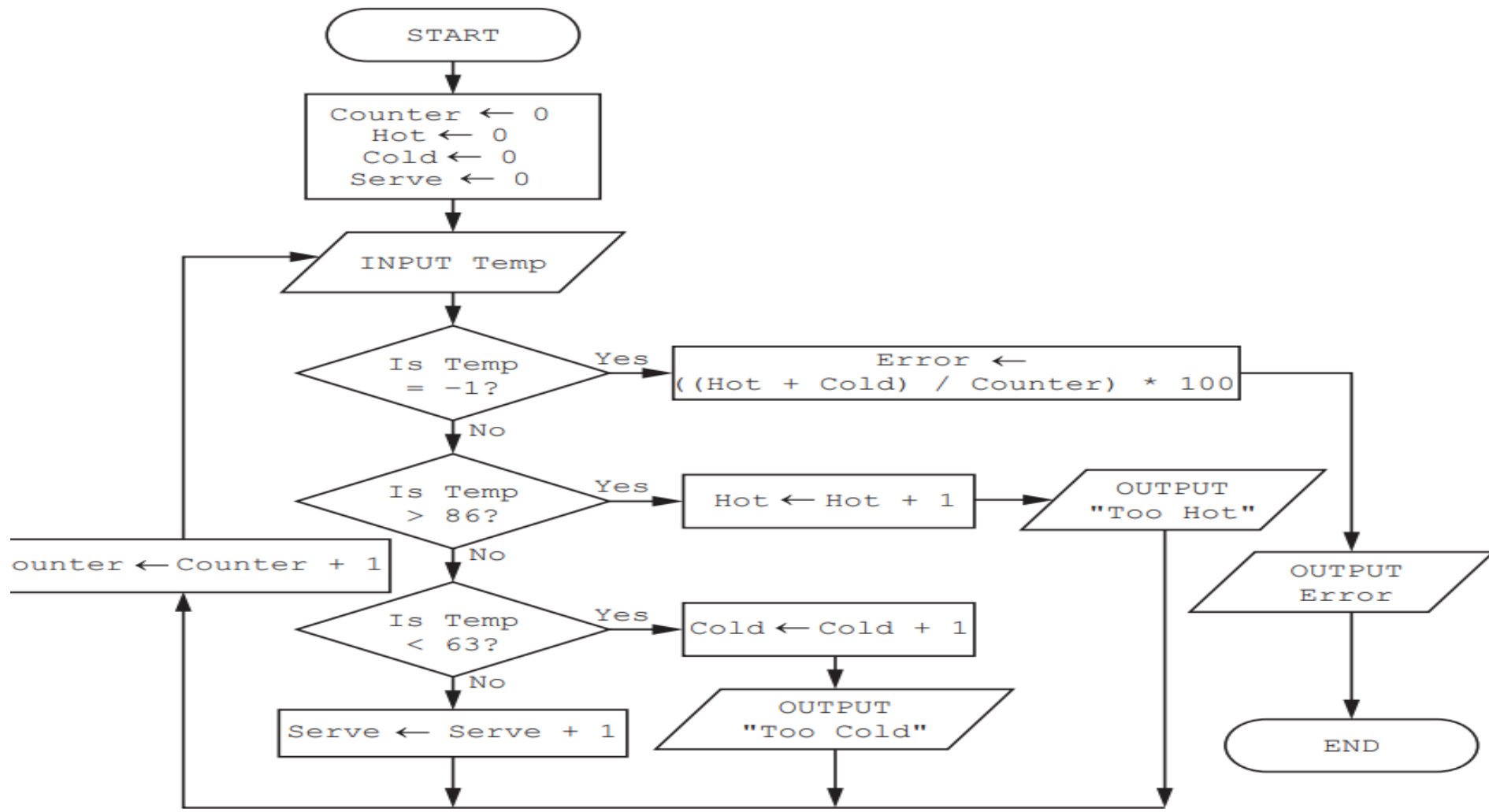
Answer 12:

Question	Answer	Marks
Section B		
2(a)	One mark per mark point, max three - line 8 / <code>PassCheck ← TRUE</code> correction <code>PassCheck ← FALSE</code> - line 12 / <code>IF Password <> Password</code> correction <code>IF Password2 <> Password // IF Password <> Password2</code> - line 18 / <code>UNTIL PassCheck OR Attempt <> 3</code> correction <code>UNTIL PassCheck OR Attempt = 3 / UNTIL PassCheck OR Attempt >= 3</code>	3
2(b)	One mark check, one mark matching description, max four Check: <code>validation</code> <code>// length check</code> Description <code>length check</code> <code>// checks number of characters in password</code> Check: <code>verification</code> <code>// double entry</code> Description <code>double entry</code> <code>// comparison that two inputs are the same</code>	4

Past paper 2 Exam Questions and Answers:

Question 13:

- 4 This algorithm checks the temperature of hot food being served to customers.



Past paper 2 Exam Questions and Answers:

Question 13:

(a) Complete the trace table for the algorithm using this input data:

75, 78, 84, 87, 91, 80, 75, 70, 65, 62, −1, 20

Counter	Hot	Cold	Serve	Temp	Error	OUTPUT

Past paper 2 Exam Questions and Answers:

Answer 13:

Question	Answer	Marks																																																																																											
4(a)	One mark for each correct column <table><tr><th>Counter</th><th>Hot</th><th>Cold</th><th>Serve</th><th>Temp</th><th>Error</th><th>OUTPUT</th></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td><td></td><td></td><td></td></tr><tr><td>1</td><td></td><td></td><td>1</td><td>75</td><td></td><td></td></tr><tr><td>2</td><td></td><td></td><td>2</td><td>78</td><td></td><td></td></tr><tr><td>3</td><td></td><td></td><td>3</td><td>84</td><td></td><td></td></tr><tr><td>4</td><td>1</td><td></td><td></td><td>87</td><td></td><td>Too Hot</td></tr><tr><td>5</td><td>2</td><td></td><td></td><td>91</td><td></td><td>Too Hot</td></tr><tr><td>6</td><td></td><td></td><td>4</td><td>80</td><td></td><td></td></tr><tr><td>7</td><td></td><td></td><td>5</td><td>75</td><td></td><td></td></tr><tr><td>8</td><td></td><td></td><td>6</td><td>70</td><td></td><td></td></tr><tr><td>9</td><td></td><td></td><td>7</td><td>65</td><td></td><td></td></tr><tr><td>10</td><td></td><td>1</td><td></td><td>62</td><td></td><td>Too Cold</td></tr><tr><td></td><td></td><td></td><td></td><td>-1</td><td>30</td><td>30</td></tr></table>	Counter	Hot	Cold	Serve	Temp	Error	OUTPUT	0	0	0	0				1			1	75			2			2	78			3			3	84			4	1			87		Too Hot	5	2			91		Too Hot	6			4	80			7			5	75			8			6	70			9			7	65			10		1		62		Too Cold					-1	30	30	7
Counter	Hot	Cold	Serve	Temp	Error	OUTPUT																																																																																							
0	0	0	0																																																																																										
1			1	75																																																																																									
2			2	78																																																																																									
3			3	84																																																																																									
4	1			87		Too Hot																																																																																							
5	2			91		Too Hot																																																																																							
6			4	80																																																																																									
7			5	75																																																																																									
8			6	70																																																																																									
9			7	65																																																																																									
10		1		62		Too Cold																																																																																							
				-1	30	30																																																																																							

Past paper 2 Exam Questions and Answers:

Question 14:

5 The pseudocode represents an algorithm.

The pre-defined function `DIV` gives the value of the result of integer division.
For example, `Y = 9 DIV 4` gives the value `Y = 2`

The pre-defined function `MOD` gives the value of the remainder of integer division.
For example, `R = 9 MOD 4` gives the value `R = 1`

```
First ← 0
Last ← 0
INPUT Limit
FOR Counter ← 1 TO Limit
    INPUT Value
    IF Value >= 100
        THEN
            IF Value < 1000
                THEN
                    First ← Value DIV 100
                    Last ← Value MOD 10
                    IF First = Last
                        THEN
                            OUTPUT Value
                        ENDIF
                    ENDIF
                ENDIF
            ENDIF
        NEXT Counter
```

Past paper 2 Exam Questions and Answers:

Question 14:

(a) Complete the trace table for the algorithm using this input data:

8, 66, 606, 6226, 8448, 642, 747, 77, 121

Counter	Value	First	Last	Limit	OUTPUT

[5]

Past paper 2 Exam Questions and Answers:

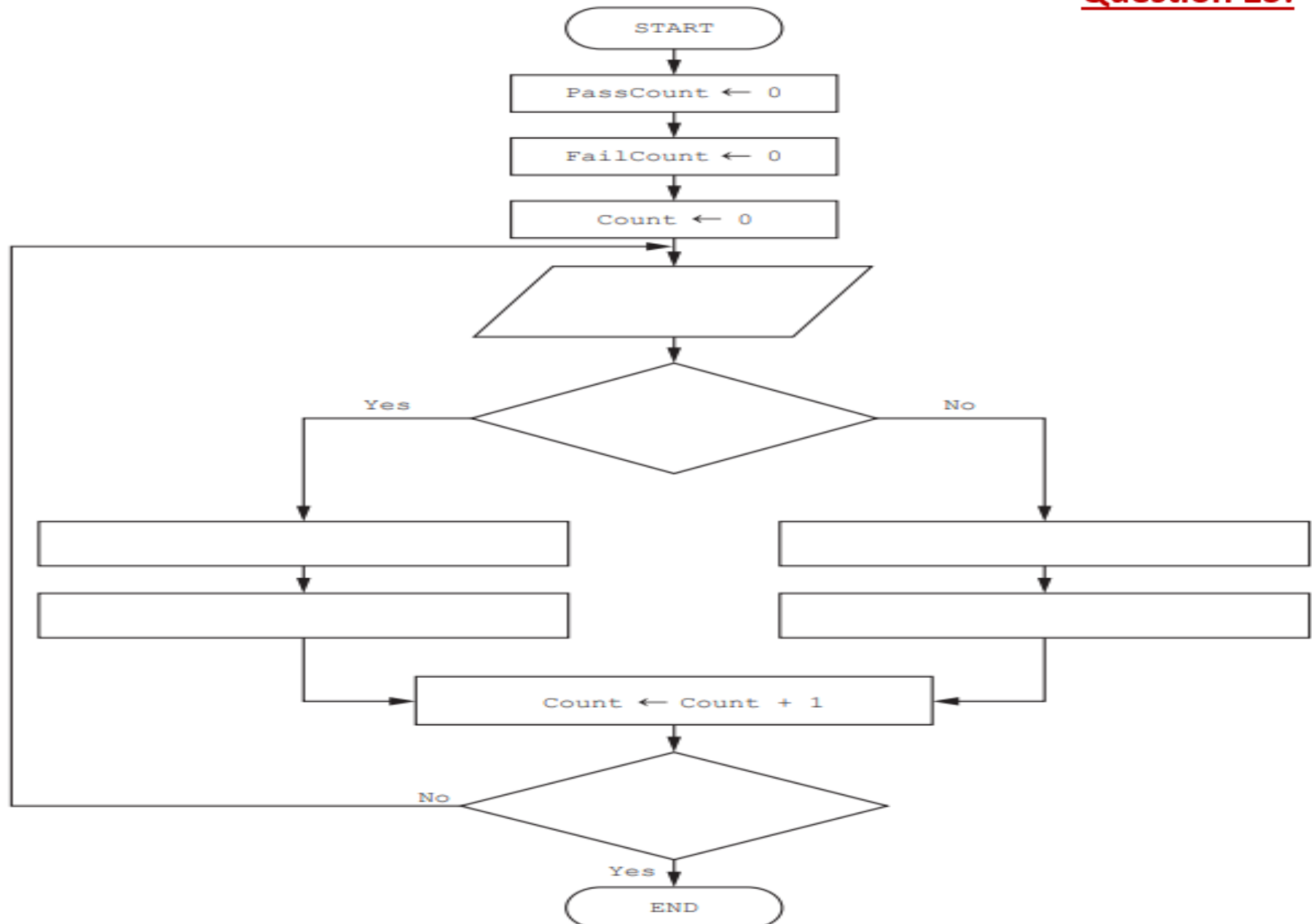
Answer 14:

Question	Answer	Marks																																																																														
5(a)	One mark per mark point, max five MP1 correct Counter and Limit columns MP2 correct Value column MP3 correct First column MP4 correct Last column MP5 correct OUTPUT <table><tr><th>Counter</th><th>Value</th><th>First</th><th>Last</th><th>Limit</th><th>OUTPUT</th></tr><tr><td></td><td></td><td>0</td><td>0</td><td>8</td><td></td></tr><tr><td>1</td><td>66</td><td></td><td></td><td></td><td></td></tr><tr><td>2</td><td>606</td><td>6</td><td>6</td><td></td><td>606</td></tr><tr><td>3</td><td>6226</td><td></td><td></td><td></td><td></td></tr><tr><td>4</td><td>8448</td><td></td><td></td><td></td><td></td></tr><tr><td>5</td><td>642</td><td>6</td><td>2</td><td></td><td></td></tr><tr><td>6</td><td>747</td><td>7</td><td>7</td><td></td><td>747</td></tr><tr><td>7</td><td>77</td><td></td><td></td><td></td><td></td></tr><tr><td>8</td><td>121</td><td>1</td><td>1</td><td></td><td>121</td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td></tr><tr><td></td><td></td><td></td><td></td><td></td><td></td></tr></table>	Counter	Value	First	Last	Limit	OUTPUT			0	0	8		1	66					2	606	6	6		606	3	6226					4	8448					5	642	6	2			6	747	7	7		747	7	77					8	121	1	1		121																			5
Counter	Value	First	Last	Limit	OUTPUT																																																																											
		0	0	8																																																																												
1	66																																																																															
2	606	6	6		606																																																																											
3	6226																																																																															
4	8448																																																																															
5	642	6	2																																																																													
6	747	7	7		747																																																																											
7	77																																																																															
8	121	1	1		121																																																																											

- 4 The flowchart shows an algorithm that should allow 60 test results to be entered into the variable *Score*. Each test result is checked to see if it is 50 or more. If it is, the test result is assigned to the *Pass* array. Otherwise, it is assigned to the *Fail* array.

(a) Complete this flowchart:

Question 15:



Past paper 2 Exam Questions and Answers:

Answer 15:

4(a)

One mark per mark point, max six

MP1 input box
MP2 correct check of Score
MP3 assign Score to Pass correctly
MP4 assign Score to Fail correctly
MP5 increment **both** array counters
MP6 correct check of number of scores

